FHIR-STU3 — Implementation Support Module (sections 7 – 7.15.5)

Source: HL7 FHIR STU3 (v3.0.2). Text-only edition for AI ingestion (NotebookLM); pages follow the source order (its table of contents when one exists).

7 Implementation Support Module

URL: https://hl7.org/fhir/STU3/implsupport-module.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

Work Group FHIR Infrastructure & Application Implementation and Design | Ballot Status: Informative

7.0 Implementation Support Module

7.0.1 Introduction

This section provides information which will be useful for FHIR implementers, including information about available libraries, tools, and other similar resources, as well as where to seek help.

7.0.2 Index

In addition to the content below, a number of implementation resources can be found on the Downloads Page.

This module also contains some specific documentation that relates to issues commonly encountered by developers:

Testing FHIR + TestScript + TestReport Validating Resources Mapping Language (Tutorial & StructureMap) Common Usages Version Management Policy | Clinical Safety Considerations How FHIR fits into an EHR Managing Resource Identity Interaction Patterns Update Rules Clinical Examples | Comparisons: v2 & v3 Messaging CDA + CDA on FHIR Other Specifications

7.0.3 Security and Privacy

For more general considerations, see the Security and Privacy module.

7.0.4 Common Use Cases

7.0.4.1 For Client Developers and Testers: Reference Servers

The following reference servers have been created by the FHIR team and made available to help implementers test their code. While the reference servers are not considered to be a normative part of the FHIR specification, the maintainers make every effort to ensure that they are fully compliant.

Note that there are a large number of servers available for testing that are not listed here. A full list is available on the HL7 wiki here .

Server Name | Maintainer | Link
Healthintersections | Grahame Grieve | http://fhir3.healthintersections.com.au/
Spark | Furore Informatica | http://spark.furore.com/
HAPI | University Health Network / James Agnew | http://fhirtest.uhn.ca/
sqlonfhir | Telstra Health / Brian Postlethwaite | http://sqlonfhir-stu3.azurewebsites.net/fhir

7.0.4.2 For Developers: Reference Implementations (Libraries)

The following reference implementations are made available under an open-source license. These libraries may be used by developers to quickly add FHIR capabilities to their applications.

Language | Library | Link | License
.NET / C# | FHIR .NET API | https://github.com/ewoutkramer/fhir-net-api | BSD-3
Java | HAPI FHIR | http://hapifhir.io | Apache 2.0
Swift | Swift FHIR | https://github.com/smart-on-fhir/Swift-FHIR | Apache 2.0
JavaScript | fhir.js | https://github.com/smart-on-fhir/fhir.js | MIT
Python | Client Py | https://github.com/smart-on-fhir/client-py | Apache 2.0
Pascal | FHIR Pascal | http://hl7.org/fhir/downloads.html | BSD-3

7.0.4.3 For Profilers

A number of tools are available to profilers wishing to create profiles for use in their implementations. A current list of tools can be found here on the HL7 wiki. (See the conformance module for information on profiling.)

7.0.4.4 For Testers

A number of tools are available to solution testers who want to test FHIR implementations for conformance to the FHIR specification. A current list of such tools can be found here .

7.0.5 Getting Help

The following are a few ways that implementers can seek help as they work with FHIR:

* FHIR Chat Channel / Zulip (maintained by fhir.org)

* FHIR Community Forum (maintained by fhir.org)

* StackOverflow (General tech community, use the tag hl7_fhir)

7.0.6 Developmental Roadmap

The reference servers and reference implementations generally try to keep up to date with recent changes to the FHIR specification. Each server may have multiple endpoints which are held to a specific version of the specification, but generally there will also be endpoints available for testing which conform to a very recent build.

Efforts are now underway to create a curated collection of quality test data which can be used by FHIR implementers to help test their applications. This collection will be made available when it is ready.

7.1 Downloads

URL: https://hl7.org/fhir/STU3/downloads.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

Specification Downloads

FHIR Definitions | All the value sets, profiles, etc defined as part of the FHIR specification, and the included implementation guides: XML JSON

Forge: Special version of definitions for Forge (temporary) These is the master set of definitions that should be the first choice whenever generating any implementation artifacts. All the other forms below include only subsets of the information available in these definition files, and do not contain all of the rules about what makes resources valid. Implementers will still need to be familiar with the content of the specification and with any profiles that apply to the resources in order to make a conformant implementation.

XML | Examples - all the example resources in XML format Validation Schemas (includes support schemas, resource schemas, modular & combined schemas, and Schematrons) Code Generation Schemas (See notes about code-generation schemas)

JSON | Examples - all the example resources in JSON format JSON Schemas (includes both modular and combined schema)

RDF | Turtle Examples - all the example resources in Turtle format JSON-LD Examples - all the example resources in JSON-LD format (experimental) ShEx Schemas - ShEx definitions for validating RDF resources Definitions - the formal definitions that define the predicates and classes used in the RDF format (not up to date) JSON-LD Definitions - the JSON-LD @context for this version of FHIR

FHIR Specification | The whole specification so that you can host your own local copy (does not include the downloads)

Implementation Tools

Validator | The official FHIR validator - a Java jar file that can be used to validate resources. See Validation Tools for further information

IG Publisher | The Implementation Guide Publishing Tool (see IG Publishing documentation) (note: use the current build version, and set the version in your control file)

IG Pack | The Definitions File used by the IG Publisher. This is a zip file that contains all the conformance & example resources, a version marker, and a json file that lists all the pages and URLs in the specification itself. This file can also be used by the validator

Translation File | Translations of common FHIR names and messages into multiple languages (see wiki for instructions on how to add to this)

Icon Pack | The FHIR Icon at various resolutions. Any FHIR Implementation created by an organization that has attended a connectathon is allowed to use the FHIR icon in association with the application (this policy will be reviewed in the future).

Test Cases | A Collection of Test Cases. These are XML or JSON files that provide test cases for the various FHIR reference implementations to ensure correct functioning

Reference Implementations

There are many open source reference implementations available to help implementers. Here are a list of the more common implementations used by implementers:

Java | HAPI-FHIR : Object Models, Parsers, Client + Server Framework, FHIR Validator, & Utilities. The specification is built with this Java code

C# | HL7.FHIR : Object models, Parsers/Serialisers, Utilities, and a Client. Source code on GitHub at http://github.com/ewoutkramer/fhir-net-api

Pascal | FhirServer : Object models, Parsers/Serialisers, Validator, Utilities, Client, and the FHIR Reference server. Requires Delphi (unicode versions)

XML | XML Tools: Document Rendering Stylesheet, supplementary implementation schemas and transforms

Javascript | FHIR.js : Javascript Client and Utilities

Swift | Swift-FHIR : Object Model, Client and Utilities

Note: These reference implementations are provided for implementer interest and assistance. While they may be used (and are) in production systems, HL7 and their various contributors accept no liability for their use. Note that these reference implementations are provided to assist to implementers to adopt the specification, and some are maintained by the FHIR project team, but are not part of the specification, and implementations are not required to conform to these, nor are they subject to the formal standards process.

Full blown open source implementations for FHIR, some of which use these reference implementations, are listed on the HL7 wiki .

It is not necessary to use these particular implementations in order to be conformant. Any other approach may be used, including code generated from the schemas.

7.2 Testing FHIR

URL: https://hl7.org/fhir/STU3/testing.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.2 Testing FHIR

FHIR Infrastructure Work Group | Maturity Level: 2 | Ballot Status: Draft

The FHIR specification describes a set of resources, and several different frameworks for exchanging resources between different systems. Because of its general nature and wide applicability, the rules made in this specification are generally fairly loose. As a consequence and in order to insure interoperability between applications claiming conformance to this specification, a testing framework has been established within the FHIR specification. To this end, the TestScript resource provides an implementation-agnostic description of tests that allows test execution engines to evaluate if a FHIR implementation conforms with the FHIR specification. Providing a clear and concise test methodology for the FHIR specification through the TestScript resource helps to enable interoperability among various FHIR server and client implementations.

Furthermore, the TestScript resource provides clear examples of the appropriate use of the FHIR specification through test-based documentation. The TestScript resource stands as a form of executable documentation allowing developers to examine the operations defined by the tests in order to understand how various RESTful API interactions and resources should be used in coordination. The tests can also be automatically executed against systems under development to determine how well the systems adhere to the specification.

The TestScript resource contains:

* Name and description detailing the purpose of the test suite

* Links describing how the test suite relates to the FHIR specification

* A list of server interactions required to execute the test suite

* A list of server interactions that the test suite validates the correctness of

* The fixtures (required data or resources) the tests use during execution

* A set of operations to set up the test suite environment

* A list of tests each containing

* Name and description of the test

* Links describing how the test relates to the FHIR specification

* A list of server interactions required to execute the test

* A list of server interactions that the test validates the correctness of

* A list of operations that provide the execution logic of the test

* A list of assertions that provide the verification logic of the test

* A set of operations to tear down the test environment

7.2.1 Execution

7.2.1.1 Workflow

Pre-Processing

The TestScript execution workflow begins by determining if the test suite is appropriate for the server under test. This can be determined by evaluating if the interactions listed in the TestScript metadata "capabilities" section are supported by the server's conformance resource. If the capabilities are supported by the server, then the TestScript can be executed. Otherwise, a test engine will provide a comparison of the system under test?s conformance capabilities and allow a TestScript as a whole to be skipped based on this comparison.. See How to specify metadata capabilities.

If the server supports the requirements of the TestScript instance, any specified fixtures are loaded or retrieved. If the fixtures are marked as 'autocreate' then they are automatically created on the server(s) under test using 'create' operations. If any of the autocreate operations fail, then the tests in the TestScript are skipped.

Setup Execution

After the fixtures are loaded and autocreates are executed, then the setup section is executed to establish the testing environment. The purpose of the setup section is typically to pre-load data (if it was not autocreated) or delete data required for the execution of the tests against the FHIR system(s) under test. The setup operations are executed once before all the tests are run (see Operation Execution). All operations in a setup section (including assertions) must complete successfully for the subsequent tests to be executed. If an assertion operation in the setup section fails, then execution and evaluation of the tests in the TestScript should be skipped. Technically, any operation (see the operations table for a complete listing) can be included in the setup section, but typical operations will be create, update, read, and vread.

Due to the possibility that the setup actions are not required on the server under test, the TestScript execution workflow MAY provide the capability of skipping or ignoring the setup section of the TestScript.

Test Execution

Once setup is complete, each test is executed. Tests contain a set of operations, and executing a test involves the evaluation of each operation listed in the test in the order defined by the test (see Operation Execution and the list of operations).

Teardown Execution

After all the tests have completed execution, the teardown section is executed. The purpose of the teardown section is to revert the FHIR server under test to a pre-test clean state. This requires removing any resources or artifacts generated as part of test suite setup or test execution. Technically, any operation (see the operations table for a complete listing) can be included in the teardown section, but the most often used operation will be delete. Assertions are not supported in the teardown section.

Due to the possibility that the teardown actions are not required on the server under test, the TestScript execution workflow MAY provide the capability of skipping or ignoring the teardown section of the TestScript.

Post-Processing

After the teardown section is executed, any fixtures that were marked 'autodelete' are removed from the server(s) under test. After this final stage, the execution of the TestScript is complete.

7.2.1.2 Fixtures

The fixtures section of the TestScript defines a set of resource instances that will be used as part of the setup, test, and teardown sections during TestScript execution. All defined fixtures are expected to be required in order for the test script to execute. Each fixture defines a resource instance by URI, and must be identified by an ID. The URI can be local or remote (i.e. another server than the one the TestScript resource resides), absolute or relative. The ID on the fixture is considered the "source" identifier of the fixture -- it is not the same thing as the resource ID on the server where it was hosted. The "source" identifier is used to define the fixture instance within the context of the TestScript. Operations reference the ID of a fixture to uniquely identify the fixture instance the operation is using ("sourceId") or acting against ("targetId"). Once a fixture has been instantiated on a server (typically by the use of a create operation), the fixture ID is mapped to the ID of the corresponding resource instance on the server. TestScript execution engines must maintain this relationship between fixture IDs and server resource IDs. The TestScript execution engine is responsible for translating the fixture IDs (whether provided to the operation as "source" or "target") to the ID of the resource on the server during execution.

Using the optional "autocreate" and "autodelete" elements (missing values default to false), fixtures can be configured to automatically be created during TestScript setup and automatically deleted during TestScript teardown. This means that additional "create" and "delete" operations in the TestScript.setup and TestScript.teardown sections are unnecessary.

7.2.1.3 Profiles

The profiles section of the TestScript defines a set of FHIR profiles (see StructureDefinition resource) that will be used as part of the setup or test sections during TestScript execution. Each profile defines a StructureDefinition instance by URI, and must be identified by an ID. The URI SHALL reference a known FHIR profile corresponding to the FHIR specification version being tested. As with Fixtures, the ID on the profile is considered the "source" identifier of the profile. Operations reference the ID of a profile to uniquely identify the profile instance the operation is using ("validateProfileId").

See Use Profiles for more information.

7.2.1.4 Variables

The variables section of the TestScript defines a set of expressions whose evaluations will be used in substitutions. These substitutions are performed in operation request headers and URL paths as well as assertion values.

Without variables, search parameters and request headers (such as If-Modified-Since) would be specified in outgoing requests as literal values. Variables allow the values to be managed externally in fixtures or dynamically in server response fixtures. They would be defined to hold path expressions against the fixtures. The path expressions would not change from one server to another but the fixture data could.

Using variables allows for the same test scripts to be executed against the same servers by different clients at the same time. Each client would change the fixture data (external to the test script) to make the data unique to that client. This ensures that the same delete/create/read operations executed concurrently by one client does not interfere with those of another client. That can be important within the context of a testing event such as a Connectathon. It can be very useful in year-round testing against public servers as well.

See Use Variables for more information.

7.2.1.5 Operation Execution

The setup, test, and teardown sections of a TestScript can contain operation elements. Operations are intended to be executed in sequence and they represent the logic defined by the TestScript. Operations define a type, sourceId, targetId, destination, responseId, contentType, and other parameters. The type of the operation aligns with a corresponding RESTful API interaction. The sourceId and targetId of an operation define the context of the fixture data the operation is acting against (see Fixtures). The destination defines the server the operation is executed on and is only required for tests that include multiple servers. The responseId specifies a fixture ID to use to map to the server response. The contentType defines the format (XML or JSON) and the corresponding mime-type (application/fhir+xml and application/fhir+json) to use in the RESTful operation (defaulting to XML). The parameters of an operation allow providing additional data required for execution.

TestScript execution engines must load the operation details and execute the operation against the server(s) under test. Operations that are expected to result in an error or exception, shall immediately be followed by one or more "assertion" operations (that test for those error conditions), otherwise the test fails. This allows for "negative" testing (for example, the test script may perform operations that should return a 4XX or 5XX HTTP response code). Test execution is halted as soon as an operation or assertion fails. If an operation or assertion fails, then the test ends in failure and the test script execution proceeds to the next test. Once all tests have completed execution, the teardown section is executed. Once teardown completes, the suite execution is complete. If any setup or test operation or assertion failed, the test script is marked in failure. Failures in teardown are ignored.

7.2.1.6 Assertion Execution

The "assertion" evaluates the results of previous operations to determine if the server under test behaves appropriately. In order to evaluate an assertion, the request, response, and results of the most recently executed operation must always be maintained by the TestScript execution engine.

The TestScript execution engine must implement the behavior of each assertion to determine if the response of the last operation meets the conditions of the assertion.

If the conditions of the assertion are met execution of the test continues. If the conditions of the assertion are not met, the test being executed fails and execution of the test is halted.

7.2.2 How Tos

7.2.2.1 Test create operation

To test if create operation is properly supported on a server, run the operation as part of TestScript.test.

First, define the fixture as a reference at the top of the test script. The fixture will hold the body of the POST request:

Note that it is illegal for the fixture to contain a resource id in a create operation.

Point the sourceId element of the create operation to the fixture id just defined:

There are two ways to verify that the create operation returned the right status code:

* See for complete list.

Use assert.response:

response codes

Use assert.responseCode explicitly:

7.2.2.2 Test search operation

To test if search operation is properly supported on a server, run the operation as part of TestScript.test.

Use the resource element to specify the resource type and the params element to specify the search parameters:

The contentType element is optional and will default to "xml" which will translate to HTTP request header "Content-Type" being set to "application/fhir+xml" by test engines. In this case, though, it was used to set it to "application/fhir+json".

The responseId element was used to store the response in a reference called "R1". This reference will hold both the response headers and the response body.

Verify that the search operation returned the right status code:

See response codes for complete list.

Verify that the search operation returned the right resource type:

There are many ways to verify that the search operation returned the right Patient:

Explicitly compare the elements to known value:

The sourceId element is pointed to the responseId value of the search operation. If no sourceId is specified, then test engines will use the response of the last operation in the test script even if responseId was not specified in the operation.

The path element holds an XPath or JSONPath expression against the response body contents.

Compare the elements in response to elements in another fixture that is either dynamically set by responseId or defined statically by the fixture element at the top of the script:

This time the birthDate value in the response is compared to the birthDate value in a fixture called 'F1'.

Verify that the response contains all the element/content in another fixture pointed to by minimumId.

Test engines will parse the 'body' of the F1 fixture and verify that each element and its value matches the corresponding element in the R1 response body. In other words, R1 is verified to be a 'superset' of F1. The resource id element in the body will be ignored during comparison. The headers will also be ignored.

F1 can be statically defined or it can be the responseId for another operation. If sourceId is not specified, then test engines will use the response of the last operation. So the previous assertion could have been defined as:

7.2.2.3 Perform delete operation in teardown

Test scripts should clean up resources created as part of execution. The TestScript.teardown operations will get executed once before the test script execution completes.

Here are a couple of ways to run delete operation in TestScript.teardown:

* Use conditional delete operation in TestScript.teardown:

Use delete operation with targetId fixture.

To do that, the resource must have been created during TestScript.setup or TestScript.test:

As part of TestScript.teardown, run the delete operation with targetId value pointed to sourceId value of the create operation:

Test engines will keep track of response headers and body of all operations.

The delete operation's targetId value is expected to correspond to the responseId of a GET operation (such as search or read) or the sourceId of a POST/PUT operation (such as create).

For targetId value corresponding to responseId of GET operations (such as search or read), test engines will use the resource type and id returned in the GET response body's resource to set the [type] and [id] in delete operation's URL, respectively.

For targetId value corresponding to responseId of POST/PUT operations (such as create), test engines will use the resource type and id returned in the POST/PUT response "Location" header to set the [type] and [id] in delete operation's URL, respectively. This is the case in the example above.

The targetId value cannot point to a statically defined fixture as the id in the fixture cannot be relied upon.

7.2.2.4 Perform delete operation in setup

Deletion of resources created during test script execution should be done using delete operation in TestScript.teardown. See Perform delete operation in teardown for details.

There might be left-over resource instances though on the server from prior executions of the script that terminated prematurely through an error. Resources can be deleted in TestScript.setup as well to ensure reliable test execution.

To delete a resource in setup, the server is required to support Conditional Delete operation.

Use the params element to specify the search criteria for the delete operation:

Test engines will append the contents of the params element to url after [type]: "[base]/[type]?[search parameters]". The resource element value ("Patient") will be used to replace [type] in the url.

7.2.2.5 Test conditional delete operation

To test if a server supports conditional delete operation, run a create operation prior to the delete using a sourceId that points to a fixture defined at the top of the script:

Then use the params element to specify the search criteria for the delete operation:

Test engines will append the contents of the params element to url after [type]: "[base]/[type]?[search parameters]". The resource element value ("Patient") will be used to replace [type] in the url.

To verify that the delete operation returned the right status code:

To verify that the resource was indeed deleted on the server, run a search using the same parameters and verify that the status code is 404 (not found):

7.2.2.6 Test conditional create operation

To test if a server supports conditional create operation, use the 'If-None-Exist' request header:

The response code of 200 verifies that the resource already exists and did not get created:

7.2.2.7 Test update operation

Update operations require a resource id. The id must be present in the fixture (PUT body contents) as well as the URL. The values must match.

Because resource ids cannot be predicted on the server, it is best to retrieve the id on a resource freshly created as part of the script

There are many ways to do that. Below is a couple:

Use update operation with targetId fixture pointing to create operation's responseId:

Test engines will keep track of response headers and body of all operations.

The update operation's targetId value is expected to correspond to the responseId of a GET operation (such as search or read) or the sourceId of a POST/PUT operation (such as create).

For targetId value corresponding to responseId of GET operations (such as search or read), test engines will use the resource type and id returned in the GET response body's resource to set the [type] and [id] in update operation's URL, respectively. This is the case in the next example below.

For targetId value corresponding to responseId of POST/PUT operations (such as create and update), test engines will use the resource type and id returned in the POST/PUT response "Location" header to set the [type] and [id] in update operation's URL, respectively. This is the case in the example above.

The targetId value cannot point to a statically defined fixture as the id in the fixture cannot be relied upon.

Use update operation with targetId fixture pointing to search operation's responseId:

After the update operation, test scripts would perform at least one more read/search operation to retrieve the contents of the updated resource and then perform assertions to verify that the data did indeed get updated on the server:

Verify that the birthdate got updated and is being returned properly:

7.2.2.8 Test conditional update operation

Unlike a regular update operation, a conditional update operation does not require a resource id in the URL (or the body of the PUT).

To test conditional update, use params element in the operation instead of targetId. The resource element will be required in this case.

Test engines will append the contents of the params element to url after [type]: "PUT [base]/[type]?[search parameters]". The resource element value ("Patient") will be used to replace [type] in the URL.

Verify that the birthdate got updated and is being returned properly:

7.2.2.9 Test read operation

The read operation requires the resource id in the URL. Since resource ids are unpredictable on servers, it's best to create the resource within the test script prior to executing the read operation:

One way to execute the read operation is to run the read operation with targetId value pointed to responseId value of the create operation:

Test engines will keep track of response headers and body of all operations.

The read operation's targetId value is expected to correspond to the responseId of a GET operation (such as search or read) or the sourceId of a POST/PUT operation (such as create).

For targetId value corresponding to responseId of GET operations (such as search or read), test engines will use the resource type and id returned in the GET response body's resource to set the [type] and [id] in read operation's URL, respectively.

For targetId value corresponding to responseId of POST/PUT operations (such as create), test engines will use the resource type and id returned in the POST/PUT response "Location" header to set the [type] and [id] in read operation's URL, respectively. This is the case in the example above.

The targetId value cannot point to a statically defined fixture as the id in the fixture cannot be relied upon.

7.2.2.10 Test vread operation

The vread operation requires the resource id as well as the resource version id in the URL. Since resource ids and version ids are unpredictable on servers, it's best to create the resource within the test script prior to executing the vread operation:

One way to execute the vread operation is to run the vread operation with targetId value pointed to responseId value of the create operation:

Test engines will keep track of response headers and body of all operations.

The vread operation's targetId value is expected to correspond to the responseId of a GET operation (such as search or read) or the sourceId of a POST/PUT operation (such as create).

For targetId value corresponding to responseId of GET operations (such as search or read), test engines will use the resource type, id and version id returned in the GET response body's resource to set the [type], [id] and [vid] in vread operation's URL, respectively.

For targetId value corresponding to responseId of POST/PUT operations (such as create), test engines will use the resource type, id and version id returned in the POST/PUT response "Location" header to set the [type], [id] and [vid] in vread operation's URL, respectively. This is the case in the example above.

The targetId value cannot point to a statically defined fixture as the id in the fixture cannot be relied upon.

7.2.2.11 Test history operation

The history operation can be executed in the following ways:

GET [base]/[type]/[id]/_history{?[parameters]&_format=[mime-type]}

Here the resource id is required in the URL. This is similar to read operation if targetId elemet is used. See Test read operation for details.

* GET [base]/[type]/_history{?[parameters]&_format=[mime-type]}

Here the resource id is not required in the URL.

Instead of targetId element, the params element can be used to specify the search criteria for the history operation.

In the following example, all history entries for John Doe patient would be returned by server:

* GET [base]/_history{?[parameters]&_format=[mime-type]}

Here neither the resource type nor the resource id is required in the URL. In the following example, no more than 50 history entries would be returned by server:

7.2.2.12 Specify Accept header in request

The default "Accept" header that will be set on all GET operations (such as read, vread, search, history, etc.) will be "application/fhir+xml".

There are two ways to change the default "Accept" header:

Use the accept element:

Test engines will set the Accept header to "application/fhir+json" if "json" is specified and will use "application/fhir+xml" if "xml" is specified.

Use the requestHeader element to set "Accept" explicitly:

Test engines will take values specified for requestHeader "as-is" and not transform them. This might be useful for negative testing, e.g. the value can be set explicitly to "applcation/xml" or an invalid value and verify server response.

7.2.2.13 Specify Content-Type header in request

The default "Content-Type" header that will be set on all POST/PUT operations (such as create, update, etc.) will be "application/fhir+xml".

There are two ways to change the default "Content-Type" header:

Use the contentType element:

Test engines will set the Content-Type header to "application/fhir+json" if "json" is specified and will use "application/fhir+xml" if "xml" is specified.

Use the requestHeader element to set Content-Type explicitly:

Test engines will take values specified for requestHeader "as-is" and not transform them. This might be useful for negative testing e.g. the value can be set explicitly to "applcation/xml" or an invalid value and verify server response.

7.2.2.14 Verify Content-Type header in response

There are two ways to verify the "Content-Type" header in response:

Use the contentType element:

Test engines will verify that "application/fhir+json" is present in Content-Type header if "json" is specified and will verify that "application/fhir+xml" is present if "xml" is specified.

Use the requestHeader element to verify Content-Type explicitly:

Test engines will take values specified for headerField "as-is" and not interpret them.

Note that test engines will not verify contentType in response if assertions for contentType are missing.

7.2.2.15 Use Profiles

Profiles are defined against known FHIR profiles (see StructureDefinition resource) corresponding to the FHIR specification version being tested. They are used in the "assert.validateProfileId" and, when present in an assert, will invoke validation of the FHIR resource payload in either the request or response message.

Profiles would be defined at the top of the script.

Below is a profile that is defined against the FHIR Patient resource type and referenced by "P1":

Test engines will not evaluate this at this point. They will store the profile reference in "P1" and will look for "P1" in an "assert.validateProfileId" element values during operation calls.

Here is a read operation for a Patient with a resource id of "example" followed by an assert that performs a FHIR validation using the base FHIR profile for the Patient resource type referenced by the "P1" profile. The validation will be performed against the response payload by default if the "assert.direction" is not present. If the validation returns any "fatal" or "error" messages, then this assert will be failed. Otherwise, if the validation returns any "warning" messages, then this assert will pass but with warnings. Finally, if the validation returns only "information" messages or no messages, then this assert will pass.

7.2.2.16 Use Variables

Variables can be defined against static fixtures and dynamic operation responses. They can be used in "operation.params", "operation.requestHeader.value", "operation.url", and "assert.value" element values. As such they allow for the data used in operations and assertions to be externally defined. The data could be unique to each client involved in interactions with a server or could be unique to a given server database instance. This allows for multiple clients to execute the same test scripts concurrently against the same server.

Variables would be defined at the top of the script.

Below is a variable that is defined as the Location header to the response referenced by "R1":

Test engines will not evaluate this at this point. They will store the expresssion in "V1" and will look for "${}" in "operation.params", "operation.requestHeader.value", and "operation.url" element values during operation calls.

Here is a read operation that will use the V1 variable. The variable expression was "Location against R1 response" (defined above). If a prior operation has not set R1 to be the responseId of the operation, then test engine will error out. Otherwise, V1 will be set to the Location header value of R1 response and that value will be substituted for ${V1} below. In other words, the read will be performed against the Location header value of R1 response.

Below are three variables defined as path expressions against the static fixture referenced by "F1". The expressions are against the given name, family name, and birthDate of a patient resource. The resource data will be managed external to the test script.

Again, test engines will not evaluate the path expression at this point. They will look for anything wrapped in '${}' in "operation.params", "operation.requestHeader.value", "operation.url", and "assert.value" element values and substitute the placeholders with the evaluated expressions.

Here is a conditional create operation that will set the requestHeader using the PatientGivenName, PatientFamilyName, and PatientDOB variables defined above. The variable expressions were path expressions against the statically defined F1 fixture. They will be evaluated against the fixture body (containing resource) and the corresponding values will be extracted from the fixtures and used to substitute the variables in the requestHeader value below.

Here is a search operation that will perform a search using the PatientGivenName, PatientFamilyName, and PatientDOB variables defined above. The variable expressions were path expressions against the statically defined F1 fixture. They will be evaluated against the fixture body (containing resource) and the corresponding values will be extracted from the fixtures and used to substitute the variables in the params value below.

Here are the assertions that verify that the search was successful:

7.2.2.17 Test server support for '_format'

Servers are required to support "_format" in the request url to determine the response mime-type. See Content Type and Encodings

Use the params element to specify the _format:

Use the requestHeader element to verify Content-Type explicitly:

7.2.2.18 How to specify metadata capabilities

If the capabilities are supported by the server, then the TestScript can be executed. Otherwise, a test engine will provide a comparison of the system under test?s conformance capabilities and allow a TestScript as a whole to be skipped based on this comparison..

Here's how to specify that the test script requires the server to support Patient create and delete operations:

The contents of PatientCreateDelete.xml would be a minimal capability statement to indicate what sections need to be present in server capability statement:

When the metadata capabilities section is defined at TestScript.metadata and the server's capability statement does not contain the elements defined in the minimal capability statement, then all the tests in the TestScript are skipped.

The "metadata.capabilities.required" and "metadata.capabilities.validated" elements only indicate whether the capabilities are the primary focus of the test script or not. The do not impact the skipping logic. Capabilities whose "metadata.capabilities.validated" flag is true are the primary focus of the test script.

7.2 Operations

This table presents a summary of the constraints applicable to TestScript.setup.action.operation, TestScript.test.action.operation, and TestScript.teardown.action.operation elements. The operation elements should be configured consistently with the FHIR RESTful API summary.

| read | vread | search | history | create | update | transaction | conformance | delete

resource | The resource element is required to specify the resource ([type]) in the request URL when params element is used. Will be ignored if targetId or url are specified. In the case of targetId, the resource type will be extracted from the fixture. | If targetId is specified, then [type] for request URL will be determined from targetId's fixture and resource element will be ignored. Otherwise, resource type will be extracted from sourceId's fixture if specified. For conditional updates, resource is required. | N/A | N/A | If targetId is specified, then [type] for request URL will be determined from targetId and resource element will be ignored. For conditional deletes, resource is required.

accept | The accept element can be used to specify the "Accept" header in the outgoing HTTP request. If "json" is specified, then "Accept" value of "application/fhir+json" will be set in the request header. If "xml" is specified, then "application/fhir+xml" will be used. | N/A

contentType | The contentType element can be used to specify the "Content-Type" header in the outgoing HTTP request. If "json" is specified, then "Content-Type" value of "application/fhir+json" will be set in the request header. If "xml" is specified, then "application/fhir+xml" will be used. | N/A

destination | If the TestScript is testing more than one FHIR server simultaneously, the destination identifies which test system the operation will receive this request message using zero-based indexing. The destination value MUST be equal to a defined TestScript.destination".

encodeRequestUrl | The encodeRequestUrl element, if "true", can be used to specify that the destination test system will receive this request message URL query parameters in HTML (Percent) encoded UTF-8 format. The default value for this element if not present is "false".

origin | If the TestScript is testing more than one FHIR server simultaneously, the origin identifies which test system the operation will send this request message using zero-based indexing. The destination value MUST be equal to a defined TestScript.origin".

params | The params element can be used to specify the [id] using variable substitutions and the rest of the highlighted portion in the request URL: [base]/[type]/[id] {?_format=[mime-type]} If used, then resource is required and targetId and url must not be specified. | The params element can be used to specify the [id] and [vid] using variable substitutions and the rest of the highlighted portion in the request URL: [base]/[type]/[id]/_history/[vid] {?_format=[mime-type]} If used, then resource is required and targetId and url must not be specified. | The params element can be used to specify the highlighted portion in the request URL: [base]/[type]{?[parameters]{&_format=[mime-type]}} If used, then resource is optional and targetId and url must not be specified. | The params element can be used to specify the [id] using variable substitutions and the rest of the highlighted portion in the following request URLs: [base]/[type]/[id]/_history{?[parameters]&_format=[mime-type]} [base]/[type]/_history{?[parameters]&_format=[mime-type]} [base]/_history{?[parameters]&_format=[mime-type]} If used, then resource is optional and targetId and url must not be specified. | N/A | The params element can be used in conditional update operation to specify the highlighted portion of the request URL: [base]/[type]?[search parameters] If used, then resource is required and targetId and url must not be specified. | N/A | The params element can be used to specify the highlighted portion in the request URL: [base]/metadata {?_format=[mime-type]} If used, then resource is ignored and targetId and url must not be specified. | The params element can be used to specify the [id] using variable substitutions in the request URL: [base]/[type]/[id] If used, then resource is required and targetId and url must not be specified.

requestHeader | The requestHeader element allows for request headers to be specified explicitly. Test engines will take values specified for requestHeader "as-is" and not transform them. This allows for testing using: "If-Modified-Since" and "If-None-Match" headers. See Conditional read. "If-Match" header. See Managing resource contention. Conditional Create using "If-None-Exist". See Conditional Create. Invalid "Content-Type" header for negative testing. etc.

requestId | N/A | N/A | N/A | N/A | The requestId element can be used to reference the operation request containing response body and headers. If specified, the value can later be used in assertion sourceId to evaluate path (XPath/JSONPath) and headerFields against the request sent for an operation. | N/A | N/A

responseId | The responseId element can be used to reference the operation response containing response body and headers. If specified, the value can later be used in assertion sourceId to evaluate path (XPath/JSONPath) and headerFields against the response received for an operation. | N/A

sourceId | N/A | N/A | N/A | N/A | The sourceId element points to a fixture to be used for the created resource. The fixture cannot contain the id element. | The sourceId element points to a fixture to be used for the updated resource. Has to correspond to the responseId of an operation executed upstream in the test script. The response body must contain a resource with a resource id. The sourceId fixture cannot be statically defined

because the id cannot be relied upon. | Fixture to be used for the transaction. Has to be a Bundle. | N/A | N/A

targetId | The targetId element can be used to specify the [type] and [id] in the request URL. If used, then params and url must not be specified. The targetId value has to correspond to the responseId of an operation executed upstream in the test script. The response body must contain a reosurce with a resource id. The targetId fixture cannot be statically defined because the id cannot be relied upon. | The targetId element can be used to specify the [type], [id], and [vid] in the request URL. If used, then params and url must not be specified. The targetId value has to correspond to the responseId of an operation executed upstream in the test script. The response body must contain a reosurce with a resource id. The targetId fixture cannot be statically defined because the id and vid cannot be relied upon. | The targetId element cannot be used as it's not allowed with params element | The targetId element can be used to specify the [type], [id], and [vid] in the request URL. If used, then params and url must not be specified. The targetId value has to correspond to the responseId of an operation executed upstream in the test script. The response body must contain a reosurce with a resource id. The targetId fixture cannot be statically defined because the id and vid cannot be relied upon. | N/A. The [type] for the request URL will be extracted from sourceId. | N/A. The [type] for the request URL will be extracted from sourceId. | N/A. | N/A. | The targetId element can be used to specify the [type] and [id] in the request URL. If used, then params and url must not be specified. The targetId value has to correspond to the responseId of an operation executed upstream in the test script. The response body must contain a reosurce with a resource id. The targetId fixture cannot be statically defined because the id cannot be relied upon.

url | The url element will contain the full HTTP URL for the operation. This should rarely be used in test scripts. One possible application would be to test if the Location header returned in a response is pointing to an expected resource. See testscript-example-search example.

7.2 Assertions

Assertion | Valid operator values | Description

compareToSourcePath | equals | notEquals | Asserts that compareToSourcePath against the response body of compareToSourceId fixture evaluates to a value that is equal or notEqual to the evaluated value of path which must be present also.

contentType | equals | notEquals | contains | notContains | Asserts that the "Content-Type" in response header is or is not the specified value for contentType element depending on the operator used.

headerField | equals | notEquals | in | notIn | greaterThan | lessThan | empty | notEmpty | contains | notContains | Asserts that the header specified for headerField element in the response contains, not contains, is equal, not equal, in, not in, greater than, or less than the value specified for value element if present. If the operator is "empty" or "notEmpty" then value will be ignored. If sourceId is not specified, then headerField will be evaluated against the last operation's response headers.

minimumId | N/A | Asserts that the response contains all the element/content in another fixture pointed to by minimumId element. This can be a statically defined fixture or one that is dynamically set via responseId.

navigationLinks | N/A | Asserts that the response Bundle contains or does NOT contain first, last, and next links depending on whether or not navigationLinks element is set to true or false.

path | equals | notEquals | in | notIn | greaterThan | lessThan | empty | notEmpty | contains | notContains | Asserts that path against the response body evaluates to a value that contains, not contains, is equal, not equal, in, not in, greater than, or less than the value specified for value element if present. If the operator is "empty" or "notEmpty" then value will be ignored. If sourceId is not specified, then path will be evaluated against the last operation's response body.

resource | equals | notEquals | Asserts that the resource returned in the response body is or is not of the specified value for resource element.

response | equals | notEquals | Asserts that status code in the response is or is not one of the enumerated values in response abbreviations.

responseCode | equals | notEquals | in | notIn | greaterThan | lessThan | Asserts that status code in the response is equal, notEqual, in, not in, greater than, or less than the specified value(s) for responseCode element

validateProfileId | N/A | Asserts that the response is valid according to the profile specified by validateProfileId element.

7.3 Resource TestScript - Content

URL: https://hl7.org/fhir/STU3/testscript.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

FHIR Infrastructure Work Group | Maturity Level: 2 | Trial Use | Compartments: Not linked to any defined compartments

A structured set of tests against a FHIR server implementation to determine compliance against the FHIR specification.

It may not be possible to fully automate the latter goal (especially with regards to business rules that might ride on top of different implementations), however the tests should be able to determine whether two servers support the operations, value sets, profiles, and extensions to exchange Bundles of Resources (such as returned from the).

7.3.1 Scope and Usage

The TestScript resource is used to define tests that can be executed on one or more FHIR servers. The TestScript resource would typically contain

* a list of fixtures (required resources used in the tests)

* setup procedures

* a suite of thematically related tests

* teardown procedures

For example, one TestScript might feature a set of tests focusing on searching Patients and validating the Bundle responses. The fixtures for such a test would contain a list of Patient resources that are required for the test to complete successfully. The setup procedures create the fixtures on the FHIR server being tested. A series of tests execute various search parameters and search for the fixtures in the results. The teardown procedures would then clean up (delete) the fixtures on FHIR server that were created during the setup procedures.

The purpose of the TestScript is to encode in an executable representation tests that can be used to

* determine whether a given FHIR server adheres to the FHIR specification and

* determine whether two or more FHIR servers implement capabilities in a compatible or interoperable manner.

Patient $everything operation

7.3.2 Boundaries and Relationships

The TestScript resource should NOT be used to represent Clinical tests, Prescriptions, or any other Healthcare related concept. The TestScript resource is an infrastructure support resource used to represent standardized tests to determine an implementation's level of adherence to the FHIR specification.

TestScript is a part of the conformance framework and is used to validate the behavior of FHIR systems, specifically their correct implementation of StructureDefinition, ValueSet, OperationDefinition, CapabilityStatement and other FHIR resources that govern system behavior. TestScript instances may be included as part of ImplementationGuides to help define and test the desired behavior of systems that choose to comply with the implementation guide.

The following resources represent the FHIR conformance framework that are used to express the expected behavior of a FHIR compliant system:

* ValueSet

* ConceptMap

* StructureDefinition

* DataElement

* CapabilityStatement

* OperationDefinition

* SearchParameter

* ImplementationGuide

7.3.3 Background and Context

7.3.3.1 Background

The TestScript resource is designed to establish testing as a first class artifact within the FHIR specification. This resource allows defining a suite of tests that can be executed on one or more FHIR servers and clients.

Implementers should be familiar with the testing concepts and descriptions found in Testing FHIR before working with this resource.

This resource is referenced by testreport

7.3.4 Resource Content

* Structure

* UML

* XML

* JSON

* Turtle

* R2 Diff

* All

Structure

Name | Flags | Card. | Type | Description & Constraints
TestScript | | | DomainResource | Describes a set of tests Elements defined in Ancestors: id, meta, implicitRules, language, text, contained, extension, modifierExtension
url | ? | 1..1 | uri | Logical URI to reference this test script (globally unique)
identifier | ? | 0..1 | Identifier | Additional identifier for the test script
version | ? | 0..1 | string | Business version of the test script
name | ? | 1..1 | string | Name for this test script (computer friendly)
title | ? | 0..1 | string | Name for this test script (human friendly)
status | ?!? | 1..1 | code | draft | active | retired | unknown PublicationStatus (Required)
experimental | ?!? | 0..1 | boolean | For testing purposes, not real usage
date | ? | 0..1 | dateTime | Date this was last changed
publisher | ? | 0..1 | string | Name of the publisher (organization or individual)
contact | ? | 0..* | ContactDetail | Contact details for the publisher
description | | 0..1 | markdown | Natural language description of the test script
useContext | ? | 0..* | UsageContext | Context the content is intended to support
jurisdiction | ? | 0..* | CodeableConcept | Intended jurisdiction for test script (if applicable) Jurisdiction ValueSet (Extensible)
purpose | | 0..1 | markdown | Why this test script is defined
copyright | | 0..1 | markdown | Use and/or publishing restrictions
origin | | 0..* | BackboneElement | An abstract server representing a client or sender in a message exchange
index | | 1..1 | integer | The index of the abstract origin server starting at 1
profile | | 1..1 | Coding | FHIR-Client | FHIR-SDC-FormFiller TestScriptProfileOriginType (Extensible)
destination | | 0..* | BackboneElement | An abstract server representing a destination or receiver in a message exchange
index | | 1..1 | integer | The index of the abstract destination server starting at 1

profile | | 1..1 | Coding | FHIR-Server | FHIR-SDC-FormManager | FHIR-SDC-FormReceiver | FHIR-SDC-FormProcessor TestScriptProfileDestinationType (Extensible)
metadata | I | 0..1 | BackboneElement | Required capability that is assumed to function correctly on the FHIR server being tested + TestScript metadata capability SHALL contain required or validated or both.
link | | 0..* | BackboneElement | Links to the FHIR specification
url | | 1..1 | uri | URL to the specification
description | | 0..1 | string | Short description
capability | | 1..* | BackboneElement | Capabilities that are assumed to function correctly on the FHIR server being tested
required | | 0..1 | boolean | Are the capabilities required?
validated | | 0..1 | boolean | Are the capabilities validated?
description | | 0..1 | string | The expected capabilities of the server
origin | | 0..* | integer | Which origin server these requirements apply to
destination | | 0..1 | integer | Which server these requirements apply to
link | | 0..* | uri | Links to the FHIR specification
capabilities | | 1..1 | Reference(CapabilityStatement) | Required Capability Statement
fixture | | 0..* | BackboneElement | Fixture in the test script - by reference (uri)
autocreate | | 0..1 | boolean | Whether or not to implicitly create the fixture during setup
autodelete | | 0..1 | boolean | Whether or not to implicitly delete the fixture during teardown
resource | | 0..1 | Reference(Any) | Reference of the resource
profile | | 0..* | Reference(Any) | Reference of the validation profile
variable | I | 0..* | BackboneElement | Placeholder for evaluated elements + Variable can only contain one of expression, headerField or path.
name | | 1..1 | string | Descriptive name for this variable
defaultValue | | 0..1 | string | Default, hard-coded, or user-defined value for this variable
description | | 0..1 | string | Natural language description of the variable
expression | | 0..1 | string | The fluentpath expression against the fixture body
headerField | | 0..1 | string | HTTP header field name for source
hint | | 0..1 | string | Hint help text for default value to enter
path | | 0..1 | string | XPath or JSONPath against the fixture body
sourceId | | 0..1 | id | Fixture Id of source expression or headerField within this variable
rule | | 0..* | BackboneElement | Assert rule used within the test script
resource | | 1..1 | Reference(Any) | Assert rule resource reference
param | | 0..* | BackboneElement | Rule parameter template
name | | 1..1 | string | Parameter name matching external assert rule parameter
value | | 0..1 | string | Parameter value defined either explicitly or dynamically
ruleset | | 0..* | BackboneElement | Assert ruleset used within the test script
resource | | 1..1 | Reference(Any) | Assert ruleset resource reference
rule | | 1..* | BackboneElement | The referenced rule within the ruleset
ruleId | | 1..1 | id | Id of referenced rule within the ruleset
param | | 0..* | BackboneElement | Ruleset rule parameter template
name | | 1..1 | string | Parameter name matching external assert ruleset rule parameter
value | | 0..1 | string | Parameter value defined either explicitly or dynamically
setup | | 0..1 | BackboneElement | A series of required setup operations before tests are executed
action | I | 1..* | BackboneElement | A setup operation or assert to perform + Setup action SHALL contain either an operation or assert but not both.
operation | I | 0..1 | BackboneElement | The setup operation to perform + Setup operation SHALL contain either sourceId or targetId or params or url.
type | | 0..1 | Coding | The operation code type that will be executed TestScriptOperationCode (Extensible)
resource | | 0..1 | code | Resource type FHIRDefinedType (Required)
label | | 0..1 | string | Tracking/logging operation label
description | | 0..1 | string | Tracking/reporting operation description
accept | | 0..1 | code | xml | json | ttl | none ContentType (Required)
contentType | | 0..1 | code | xml | json | ttl | none ContentType (Required)
destination | | 0..1 | integer | Server responding to the request
encodeRequestUrl | | 0..1 | boolean | Whether or not to send the request url in encoded format
origin | | 0..1 | integer | Server initiating the request
params | | 0..1 | string | Explicitly defined path parameters
requestHeader | | 0..* | BackboneElement | Each operation can have one or more header elements
field | | 1..1 | string | HTTP header field name
value | | 1..1 | string | HTTP headerfield value
requestId | | 0..1 | id | Fixture Id of mapped request
responseId | | 0..1 | id | Fixture Id of mapped response
sourceId | | 0..1 | id | Fixture Id of body for PUT and POST requests
targetId | | 0..1 | id | Id of fixture used for extracting the [id], [type], and [vid] for GET requests
url | | 0..1 | string | Request URL
assert | I | 0..1 | BackboneElement | The assertion to perform + Only a single assertion SHALL be present within setup action assert element. + Setup action assert SHALL contain either compareToSourceId and compareToSourceExpression, compareToSourceId and compareToSourcePath or neither. + Setup action assert response and responseCode SHALL be empty when direction equals request
label | | 0..1 | string | Tracking/logging assertion label
description | | 0..1 | string | Tracking/reporting assertion description
direction | | 0..1 | code | response | request AssertionDirectionType (Required)
compareToSourceId | | 0..1 | string | Id of the source fixture to be evaluated
compareToSourceExpression | | 0..1 | string | The fluentpath expression to evaluate against the source fixture
compareToSourcePath | | 0..1 | string | XPath or JSONPath expression to evaluate against the source fixture
contentType | | 0..1 | code | xml | json | ttl | none ContentType (Required)
expression | | 0..1 | string | The fluentpath expression to be evaluated

headerField | | 0..1 | string | HTTP header field name
minimumId | | 0..1 | string | Fixture Id of minimum content resource
navigationLinks | | 0..1 | boolean | Perform validation on navigation links?
operator | | 0..1 | code | equals | notEquals | in | notIn | greaterThan | lessThan | empty | notEmpty | contains | notContains | eval AssertionOperatorType (Required)
path | | 0..1 | string | XPath or JSONPath expression
requestMethod | | 0..1 | code | delete | get | options | patch | post | put TestScriptRequestMethodCode (Required)
requestURL | | 0..1 | string | Request URL comparison value
resource | | 0..1 | code | Resource type FHIRDefinedType (Required)
response | | 0..1 | code | okay | created | noContent | notModified | bad | forbidden | notFound | methodNotAllowed | conflict | gone | preconditionFailed | unprocessable AssertionResponseTypes (Required)
responseCode | | 0..1 | string | HTTP response code to test
rule | | 0..1 | BackboneElement | The reference to a TestScript.rule
ruleId | | 1..1 | id | Id of the TestScript.rule
param | | 0..* | BackboneElement | Rule parameter template
name | | 1..1 | string | Parameter name matching external assert rule parameter
value | | 1..1 | string | Parameter value defined either explicitly or dynamically
ruleset | | 0..1 | BackboneElement | The reference to a TestScript.ruleset
rulesetId | | 1..1 | id | Id of the TestScript.ruleset
rule | | 0..* | BackboneElement | The referenced rule within the ruleset
ruleId | | 1..1 | id | Id of referenced rule within the ruleset
param | | 0..* | BackboneElement | Rule parameter template
name | | 1..1 | string | Parameter name matching external assert ruleset rule parameter
value | | 1..1 | string | Parameter value defined either explicitly or dynamically
sourceId | | 0..1 | id | Fixture Id of source expression or headerField
validateProfileId | | 0..1 | id | Profile Id of validation profile reference
value | | 0..1 | string | The value to compare to
warningOnly | | 0..1 | boolean | Will this assert produce a warning only on error?
test | | 0..* | BackboneElement | A test in this script
name | | 0..1 | string | Tracking/logging name of this test
description | | 0..1 | string | Tracking/reporting short description of the test
action | I | 1..* | BackboneElement | A test operation or assert to perform + Test action SHALL contain either an operation or assert but not both.
operation | I | 0..1 | see operation | The setup operation to perform + Test operation SHALL contain either sourceId or targetId or params or url.
assert | I | 0..1 | see assert | The setup assertion to perform + Only a single assertion SHALL be present within test action assert element. + Test action assert SHALL contain either compareToSourceId and compareToSourceExpression, compareToSourceId and compareToSourcePath or neither. + Test action assert response and response and responseCode SHALL be empty when direction equals request
teardown | | 0..1 | BackboneElement | A series of required clean up steps
action | | 1..* | BackboneElement | One or more teardown operations to perform
operation | I | 1..1 | see operation | The teardown operation to perform + Teardown operation SHALL contain either sourceId or targetId or params or url.
Documentation for this format

UML Diagram (Legend)

XML Template

```
<TestScript xmlns="http://hl7.org/fhir">
 <!-- from Resource: id, meta, implicitRules, and language -->
 <!-- from DomainResource: text, contained, extension, and modifierExtension -->
 <url value="[uri]"/><!-- 1..1 Logical URI to reference this test script (globally unique) -->
 <identifier><!-- 0..1 Identifier Additional identifier for the test script --></identifier>
 <version value="[string]"/><!-- 0..1 Business version of the test script -->
 <name value="[string]"/><!-- 1..1 Name for this test script (computer friendly) -->
 <title value="[string]"/><!-- 0..1 Name for this test script (human friendly) -->
 <status value="[code]"/><!-- 1..1 draft | active | retired | unknown -->
 <experimental value="[boolean]"/><!-- 0..1 For testing purposes, not real usage -->
 <date value="[dateTime]"/><!-- 0..1 Date this was last changed -->
 <publisher value="[string]"/><!-- 0..1 Name of the publisher (organization or individual) -->
 <contact><!-- 0..* ContactDetail Contact details for the publisher --></contact>
 <description value="[markdown]"/><!-- 0..1 Natural language description of the test script -->
 <useContext><!-- 0..* UsageContext Context the content is intended to support --></useContext>
 <jurisdiction><!-- 0..* CodeableConcept Intended jurisdiction for test script (if applicable) --></jurisdiction>
 <purpose value="[markdown]"/><!-- 0..1 Why this test script is defined -->
 <copyright value="[markdown]"/><!-- 0..1 Use and/or publishing restrictions -->
 <origin>  <!-- 0..* An abstract server representing a client or sender in a message exchange -->
  <index value="[integer]"/><!-- 1..1 The index of the abstract origin server starting at 1 -->
  <profile><!-- 1..1 Coding FHIR-Client | FHIR-SDC-FormFiller --></profile>
 </origin>
 <destination>  <!-- 0..* An abstract server representing a destination or receiver in a message exchange -->
  <index value="[integer]"/><!-- 1..1 The index of the abstract destination server starting at 1 -->
  <profile><!-- 1..1 Coding FHIR-Server | FHIR-SDC-FormManager | FHIR-SDC-FormReceiver | FHIR-SDC-FormProcessor
    --></profile>
 </destination>
 <metadata>  <!-- 0..1 Required capability that is assumed to function correctly on the FHIR server being tested -->
  <link>  <!-- 0..* Links to the FHIR specification -->
   <url value="[uri]"/><!-- 1..1 URL to the specification -->
   <description value="[string]"/><!-- 0..1 Short description -->
  </link>
  <capability>  <!-- 1..* Capabilities  that are assumed to function correctly on the FHIR server being tested -->
   <required value="[boolean]"/><!-- 0..1 Are the capabilities required? -->
   <validated value="[boolean]"/><!-- 0..1 Are the capabilities validated? -->
```

```
  <description value="[string]"/><!-- 0..1 The expected capabilities of the server -->
  <origin value="[integer]"/><!-- 0..* Which origin server these requirements apply to -->
  <destination value="[integer]"/><!-- 0..1 Which server these requirements apply to -->
  <link value="[uri]"/><!-- 0..* Links to the FHIR specification -->
  <capabilities><!-- 1..1 Reference(CapabilityStatement) Required Capability Statement --></capabilities>
 </capability>
</metadata>
<fixture>  <!-- 0..* Fixture in the test script - by reference (uri) -->
 <autocreate value="[boolean]"/><!-- 0..1 Whether or not to implicitly create the fixture during setup -->
 <autodelete value="[boolean]"/><!-- 0..1 Whether or not to implicitly delete the fixture during teardown -->
 <resource><!-- 0..1 Reference(Any) Reference of the resource --></resource>
</fixture>
<profile><!-- 0..* Reference(Any) Reference of the validation profile --></profile>
<variable>  <!-- 0..* Placeholder for evaluated elements -->
 <name value="[string]"/><!-- 1..1 Descriptive name for this variable -->
 <defaultValue value="[string]"/><!-- 0..1 Default, hard-coded, or user-defined value for this variable -->
 <description value="[string]"/><!-- 0..1 Natural language description of the variable -->
 <expression value="[string]"/><!-- 0..1 The fluentpath expression against the fixture body -->
 <headerField value="[string]"/><!-- 0..1 HTTP header field name for source -->
 <hint value="[string]"/><!-- 0..1 Hint help text for default value to enter -->
 <path value="[string]"/><!-- 0..1 XPath or JSONPath against the fixture body -->
 <sourceId value="[id]"/><!-- 0..1 Fixture Id of source expression or headerField within this variable -->
</variable>
<rule>  <!-- 0..* Assert rule used within the test script -->
 <resource><!-- 1..1 Reference(Any) Assert rule resource reference --></resource>
 <param>  <!-- 0..* Rule parameter template -->
  <name value="[string]"/><!-- 1..1 Parameter name matching external assert rule parameter -->
  <value value="[string]"/><!-- 0..1 Parameter value defined either explicitly or dynamically -->
 </param>
</rule>
<ruleset>  <!-- 0..* Assert ruleset used within the test script -->
 <resource><!-- 1..1 Reference(Any) Assert ruleset resource reference --></resource>
 <rule>  <!-- 1..* The referenced rule within the ruleset -->
  <ruleId value="[id]"/><!-- 1..1 Id of referenced rule within the ruleset -->
  <param>  <!-- 0..* Ruleset rule parameter template -->
   <name value="[string]"/><!-- 1..1 Parameter name matching external assert ruleset rule parameter -->
   <value value="[string]"/><!-- 0..1 Parameter value defined either explicitly or dynamically -->
  </param>
 </rule>
</ruleset>
<setup>  <!-- 0..1 A series of required setup operations before tests are executed -->
 <action>  <!-- 1..* A setup operation or assert to perform -->
  <operation>  <!-- 0..1 The setup operation to perform -->
   <type><!-- 0..1 Coding The operation code type that will be executed --></type>
   <resource value="[code]"/><!-- 0..1 Resource type -->
   <label value="[string]"/><!-- 0..1 Tracking/logging operation label -->
   <description value="[string]"/><!-- 0..1 Tracking/reporting operation description -->
   <accept value="[code]"/><!-- 0..1 xml | json | ttl | none -->
   <contentType value="[code]"/><!-- 0..1 xml | json | ttl | none -->
   <destination value="[integer]"/><!-- 0..1 Server responding to the request -->
   <encodeRequestUrl value="[boolean]"/><!-- 0..1 Whether or not to send the request url in encoded format -->
   <origin value="[integer]"/><!-- 0..1 Server initiating the request -->
   <params value="[string]"/><!-- 0..1 Explicitly defined path parameters -->
   <requestHeader>  <!-- 0..* Each operation can have one or more header elements -->
    <field value="[string]"/><!-- 1..1 HTTP header field name -->
    <value value="[string]"/><!-- 1..1 HTTP headerfield value -->
   </requestHeader>
   <requestId value="[id]"/><!-- 0..1 Fixture Id of mapped request -->
   <responseId value="[id]"/><!-- 0..1 Fixture Id of mapped response -->
   <sourceId value="[id]"/><!-- 0..1 Fixture Id of body for PUT and POST requests -->
   <targetId value="[id]"/><!-- 0..1 Id of fixture used for extracting the [id],  [type], and [vid] for GET requests
   -->
   <url value="[string]"/><!-- 0..1 Request URL -->
  </operation>
  <assert>  <!-- 0..1 The assertion to perform -->
   <label value="[string]"/><!-- 0..1 Tracking/logging assertion label -->
   <description value="[string]"/><!-- 0..1 Tracking/reporting assertion description -->
   <direction value="[code]"/><!-- 0..1 response | request -->
   <compareToSourceId value="[string]"/><!-- 0..1 Id of the source fixture to be evaluated -->
   <compareToSourceExpression value="[string]"/><!-- 0..1 The fluentpath expression to evaluate against the source
   fixture -->
   <compareToSourcePath value="[string]"/><!-- 0..1 XPath or JSONPath expression to evaluate against the source fixture
    -->
   <contentType value="[code]"/><!-- 0..1 xml | json | ttl | none -->
   <expression value="[string]"/><!-- 0..1 The fluentpath expression to be evaluated -->
   <headerField value="[string]"/><!-- 0..1 HTTP header field name -->
   <minimumId value="[string]"/><!-- 0..1 Fixture Id of minimum content resource -->
   <navigationLinks value="[boolean]"/><!-- 0..1 Perform validation on navigation links? -->
   <operator value="[code]"/><!-- 0..1 equals | notEquals | in | notIn | greaterThan | lessThan | empty | notEmpty |
   contains | notContains | eval -->
   <path value="[string]"/><!-- 0..1 XPath or JSONPath expression -->
   <requestMethod value="[code]"/><!-- 0..1 delete | get | options | patch | post | put -->
   <requestURL value="[string]"/><!-- 0..1 Request URL comparison value -->
   <resource value="[code]"/><!-- 0..1 Resource type -->
   <response value="[code]"/><!-- 0..1 okay | created | noContent | notModified | bad | forbidden | notFound |
   methodNotAllowed | conflict | gone | preconditionFailed | unprocessable -->
   <responseCode value="[string]"/><!-- 0..1 HTTP response code to test -->
   <rule>  <!-- 0..1 The reference to a TestScript.rule -->
    <ruleId value="[id]"/><!-- 1..1 Id of the TestScript.rule -->
```

```
    <param>  <!-- 0..* Rule parameter template -->
     <name value="[string]"/><!-- 1..1 Parameter name matching external assert rule parameter -->
     <value value="[string]"/><!-- 1..1 Parameter value defined either explicitly or dynamically -->
    </param>
   </rule>
   <ruleset>  <!-- 0..1 The reference to a TestScript.ruleset -->
    <rulesetId value="[id]"/><!-- 1..1 Id of the TestScript.ruleset -->
    <rule>  <!-- 0..* The referenced rule within the ruleset -->
     <ruleId value="[id]"/><!-- 1..1 Id of referenced rule within the ruleset -->
     <param>  <!-- 0..* Rule parameter template -->
      <name value="[string]"/><!-- 1..1 Parameter name matching external assert ruleset rule parameter -->
      <value value="[string]"/><!-- 1..1 Parameter value defined either explicitly or dynamically -->
     </param>
    </rule>
   </ruleset>
   <sourceId value="[id]"/><!-- 0..1 Fixture Id of source expression or headerField -->
   <validateProfileId value="[id]"/><!-- 0..1 Profile Id of validation profile reference -->
   <value value="[string]"/><!-- 0..1 The value to compare to -->
   <warningOnly value="[boolean]"/><!-- 0..1 Will this assert produce a warning only on error? -->
  </assert>
 </action>
</setup>
<test>  <!-- 0..* A test in this script -->
 <name value="[string]"/><!-- 0..1 Tracking/logging name of this test -->
 <description value="[string]"/><!-- 0..1 Tracking/reporting short description of the test -->
 <action>  <!-- 1..* A test operation or assert to perform -->
  <operation><!-- 0..1 Content as for TestScript.setup.action.operation The setup operation to perform --></operation>
  <assert><!-- 0..1 Content as for TestScript.setup.action.assert The setup assertion to perform --></assert>
 </action>
</test>
<teardown>  <!-- 0..1 A series of required clean up steps -->
 <action>  <!-- 1..* One or more teardown operations to perform -->
  <operation><!-- 1..1 Content as for TestScript.setup.action.operation The teardown operation to perform
   --></operation>
 </action>
</teardown>
</TestScript>
```

JSON Template

```
{
  "resourceType" : "TestScript",
  // from Resource: id, meta, implicitRules, and language
  // from DomainResource: text, contained, extension, and modifierExtension
  "url" : "<uri>", // R!  Logical URI to reference this test script (globally unique)
  "identifier" : { Identifier }, // Additional identifier for the test script
  "version" : "<string>", // Business version of the test script
  "name" : "<string>", // R!  Name for this test script (computer friendly)
  "title" : "<string>", // Name for this test script (human friendly)
  "status" : "<code>", // R!  draft | active | retired | unknown
  "experimental" : <boolean>, // For testing purposes, not real usage
  "date" : "<dateTime>", // Date this was last changed
  "publisher" : "<string>", // Name of the publisher (organization or individual)
  "contact" : [{ ContactDetail }], // Contact details for the publisher
  "description" : "<markdown>", // Natural language description of the test script
  "useContext" : [{ UsageContext }], // Context the content is intended to support
  "jurisdiction" : [{ CodeableConcept }], // Intended jurisdiction for test script (if applicable)
  "purpose" : "<markdown>", // Why this test script is defined
  "copyright" : "<markdown>", // Use and/or publishing restrictions
  "origin" : [{ // An abstract server representing a client or sender in a message exchange
    "index" : <integer>, // R!  The index of the abstract origin server starting at 1
    "profile" : { Coding } // R!  FHIR-Client | FHIR-SDC-FormFiller
  }],
  "destination" : [{ // An abstract server representing a destination or receiver in a message exchange
    "index" : <integer>, // R!  The index of the abstract destination server starting at 1
    "profile" : { Coding } // R!  FHIR-Server | FHIR-SDC-FormManager | FHIR-SDC-FormReceiver | FHIR-SDC-FormProcessor
  }],
  "metadata" : { // Required capability that is assumed to function correctly on the FHIR server being tested
    "link" : [{ // Links to the FHIR specification
      "url" : "<uri>", // R!  URL to the specification
      "description" : "<string>" // Short description
    }],
    "capability" : [{ // R!  Capabilities  that are assumed to function correctly on the FHIR server being tested
      "required" : <boolean>, // Are the capabilities required?
      "validated" : <boolean>, // Are the capabilities validated?
      "description" : "<string>", // The expected capabilities of the server
      "origin" : [<integer>], // Which origin server these requirements apply to
      "destination" : <integer>, // Which server these requirements apply to
      "link" : ["<uri>"], // Links to the FHIR specification
      "capabilities" : { Reference(CapabilityStatement) } // R!  Required Capability Statement
    }]
  },
  "fixture" : [{ // Fixture in the test script - by reference (uri)
    "autocreate" : <boolean>, // Whether or not to implicitly create the fixture during setup
    "autodelete" : <boolean>, // Whether or not to implicitly delete the fixture during teardown
    "resource" : { Reference(Any) } // Reference of the resource
  }],
  "profile" : [{ Reference(Any) }], // Reference of the validation profile
  "variable" : [{ // Placeholder for evaluated elements
    "name" : "<string>", // R!  Descriptive name for this variable
```

```
  "defaultValue" : "<string>", // Default, hard-coded, or user-defined value for this variable
  "description" : "<string>", // Natural language description of the variable
  "expression" : "<string>", // The fluentpath expression against the fixture body
  "headerField" : "<string>", // HTTP header field name for source
  "hint" : "<string>", // Hint help text for default value to enter
  "path" : "<string>", // XPath or JSONPath against the fixture body
  "sourceId" : "<id>" // Fixture Id of source expression or headerField within this variable
}],
"rule" : [{ // Assert rule used within the test script
  "resource" : { Reference(Any) }, // R!  Assert rule resource reference
  "param" : [{ // Rule parameter template
    "name" : "<string>", // R!  Parameter name matching external assert rule parameter
    "value" : "<string>" // Parameter value defined either explicitly or dynamically
  }]
}],
"ruleset" : [{ // Assert ruleset used within the test script
  "resource" : { Reference(Any) }, // R!  Assert ruleset resource reference
  "rule" : [{ // R!  The referenced rule within the ruleset
    "ruleId" : "<id>", // R!  Id of referenced rule within the ruleset
    "param" : [{ // Ruleset rule parameter template
      "name" : "<string>", // R!  Parameter name matching external assert ruleset rule parameter
      "value" : "<string>" // Parameter value defined either explicitly or dynamically
    }]
  }]
}],
"setup" : { // A series of required setup operations before tests are executed
  "action" : [{ // R!  A setup operation or assert to perform
    "operation" : { // The setup operation to perform
      "type" : { Coding }, // The operation code type that will be executed
      "resource" : "<code>", // Resource type
      "label" : "<string>", // Tracking/logging operation label
      "description" : "<string>", // Tracking/reporting operation description
      "accept" : "<code>", // xml | json | ttl | none
      "contentType" : "<code>", // xml | json | ttl | none
      "destination" : <integer>, // Server responding to the request
      "encodeRequestUrl" : <boolean>, // Whether or not to send the request url in encoded format
      "origin" : <integer>, // Server initiating the request
      "params" : "<string>", // Explicitly defined path parameters
      "requestHeader" : [{ // Each operation can have one or more header elements
        "field" : "<string>", // R!  HTTP header field name
        "value" : "<string>" // R!  HTTP headerfield value
      }],
      "requestId" : "<id>", // Fixture Id of mapped request
      "responseId" : "<id>", // Fixture Id of mapped response
      "sourceId" : "<id>", // Fixture Id of body for PUT and POST requests
      "targetId" : "<id>", // Id of fixture used for extracting the [id],  [type], and [vid] for GET requests
      "url" : "<string>" // Request URL
    },
    "assert" : { // The assertion to perform
      "label" : "<string>", // Tracking/logging assertion label
      "description" : "<string>", // Tracking/reporting assertion description
      "direction" : "<code>", // response | request
      "compareToSourceId" : "<string>", // Id of the source fixture to be evaluated
      "compareToSourceExpression" : "<string>", // The fluentpath expression to evaluate against the source fixture
      "compareToSourcePath" : "<string>", // XPath or JSONPath expression to evaluate against the source fixture
      "contentType" : "<code>", // xml | json | ttl | none
      "expression" : "<string>", // The fluentpath expression to be evaluated
      "headerField" : "<string>", // HTTP header field name
      "minimumId" : "<string>", // Fixture Id of minimum content resource
      "navigationLinks" : <boolean>, // Perform validation on navigation links?
      "operator" : "<code>", // equals | notEquals | in | notIn | greaterThan | lessThan | empty | notEmpty | contains
   | notContains | eval
      "path" : "<string>", // XPath or JSONPath expression
      "requestMethod" : "<code>", // delete | get | options | patch | post | put
      "requestURL" : "<string>", // Request URL comparison value
      "resource" : "<code>", // Resource type
      "response" : "<code>", // okay | created | noContent | notModified | bad | forbidden | notFound |
  methodNotAllowed | conflict | gone | preconditionFailed | unprocessable
      "responseCode" : "<string>", // HTTP response code to test

      "rule" : { // The reference to a TestScript.rule
        "ruleId" : "<id>", // R!  Id of the TestScript.rule
        "param" : [{ // Rule parameter template
          "name" : "<string>", // R!  Parameter name matching external assert rule parameter
          "value" : "<string>" // R!  Parameter value defined either explicitly or dynamically
        }]
      },
      "ruleset" : { // The reference to a TestScript.ruleset
        "rulesetId" : "<id>", // R!  Id of the TestScript.ruleset
        "rule" : [{ // The referenced rule within the ruleset
          "ruleId" : "<id>", // R!  Id of referenced rule within the ruleset
          "param" : [{ // Rule parameter template
            "name" : "<string>", // R!  Parameter name matching external assert ruleset rule parameter
            "value" : "<string>" // R!  Parameter value defined either explicitly or dynamically
          }]
        }]
      },
      "sourceId" : "<id>", // Fixture Id of source expression or headerField
      "validateProfileId" : "<id>", // Profile Id of validation profile reference
      "value" : "<string>", // The value to compare to
```

```
      "warningOnly" : <boolean> // Will this assert produce a warning only on error?
    }
   }]
  },
  "test" : [{ // A test in this script
   "name" : "<string>", // Tracking/logging name of this test
   "description" : "<string>", // Tracking/reporting short description of the test
   "action" : [{ // R!  A test operation or assert to perform
    "operation" : { Content as for TestScript.setup.action.operation }, // The setup operation to perform
    "assert" : { Content as for TestScript.setup.action.assert } // The setup assertion to perform
   }]
  }],
  "teardown" : { // A series of required clean up steps
   "action" : [{ // R!  One or more teardown operations to perform
    "operation" : { Content as for TestScript.setup.action.operation } // R!  The teardown operation to perform
   }]
  }
}
```

Turtle Template

```
@prefix fhir: <http://hl7.org/fhir/> .


[ a fhir:TestScript;
  fhir:nodeRole fhir:treeRoot; # if this is the parser root

  # from Resource: .id, .meta, .implicitRules, and .language
  # from DomainResource: .text, .contained, .extension, and .modifierExtension
  fhir:TestScript.url [ uri ]; # 1..1 Logical URI to reference this test script (globally unique)
  fhir:TestScript.identifier [ Identifier ]; # 0..1 Additional identifier for the test script
  fhir:TestScript.version [ string ]; # 0..1 Business version of the test script
  fhir:TestScript.name [ string ]; # 1..1 Name for this test script (computer friendly)
  fhir:TestScript.title [ string ]; # 0..1 Name for this test script (human friendly)
  fhir:TestScript.status [ code ]; # 1..1 draft | active | retired | unknown
  fhir:TestScript.experimental [ boolean ]; # 0..1 For testing purposes, not real usage
  fhir:TestScript.date [ dateTime ]; # 0..1 Date this was last changed
  fhir:TestScript.publisher [ string ]; # 0..1 Name of the publisher (organization or individual)
  fhir:TestScript.contact [ ContactDetail ], ... ; # 0..* Contact details for the publisher
  fhir:TestScript.description [ markdown ]; # 0..1 Natural language description of the test script
  fhir:TestScript.useContext [ UsageContext ], ... ; # 0..* Context the content is intended to support
  fhir:TestScript.jurisdiction [ CodeableConcept ], ... ; # 0..* Intended jurisdiction for test script (if applicable)
  fhir:TestScript.purpose [ markdown ]; # 0..1 Why this test script is defined
  fhir:TestScript.copyright [ markdown ]; # 0..1 Use and/or publishing restrictions
  fhir:TestScript.origin [ # 0..* An abstract server representing a client or sender in a message exchange
    fhir:TestScript.origin.index [ integer ]; # 1..1 The index of the abstract origin server starting at 1
    fhir:TestScript.origin.profile [ Coding ]; # 1..1 FHIR-Client | FHIR-SDC-FormFiller
  ], ...;
  fhir:TestScript.destination [ # 0..* An abstract server representing a destination or receiver in a message exchange
    fhir:TestScript.destination.index [ integer ]; # 1..1 The index of the abstract destination server starting at 1
    fhir:TestScript.destination.profile [ Coding ]; # 1..1 FHIR-Server | FHIR-SDC-FormManager | FHIR-SDC-FormReceiver |
    FHIR-SDC-FormProcessor
  ], ...;
  fhir:TestScript.metadata [ # 0..1 Required capability that is assumed to function correctly on the FHIR server being
    tested
    fhir:TestScript.metadata.link [ # 0..* Links to the FHIR specification
      fhir:TestScript.metadata.link.url [ uri ]; # 1..1 URL to the specification
      fhir:TestScript.metadata.link.description [ string ]; # 0..1 Short description
    ], ...;
    fhir:TestScript.metadata.capability [ # 1..* Capabilities  that are assumed to function correctly on the FHIR server
     being tested
      fhir:TestScript.metadata.capability.required [ boolean ]; # 0..1 Are the capabilities required?
      fhir:TestScript.metadata.capability.validated [ boolean ]; # 0..1 Are the capabilities validated?
      fhir:TestScript.metadata.capability.description [ string ]; # 0..1 The expected capabilities of the server
      fhir:TestScript.metadata.capability.origin [ integer ], ... ; # 0..* Which origin server these requirements apply
    to
      fhir:TestScript.metadata.capability.destination [ integer ]; # 0..1 Which server these requirements apply to
      fhir:TestScript.metadata.capability.link [ uri ], ... ; # 0..* Links to the FHIR specification
      fhir:TestScript.metadata.capability.capabilities [ Reference(CapabilityStatement) ]; # 1..1 Required Capability
    Statement
    ], ...;
  ];
  fhir:TestScript.fixture [ # 0..* Fixture in the test script - by reference (uri)
    fhir:TestScript.fixture.autocreate [ boolean ]; # 0..1 Whether or not to implicitly create the fixture during setup
    fhir:TestScript.fixture.autodelete [ boolean ]; # 0..1 Whether or not to implicitly delete the fixture during
    teardown
    fhir:TestScript.fixture.resource [ Reference(Any) ]; # 0..1 Reference of the resource
  ], ...;
  fhir:TestScript.profile [ Reference(Any) ], ... ; # 0..* Reference of the validation profile
  fhir:TestScript.variable [ # 0..* Placeholder for evaluated elements
    fhir:TestScript.variable.name [ string ]; # 1..1 Descriptive name for this variable
    fhir:TestScript.variable.defaultValue [ string ]; # 0..1 Default, hard-coded, or user-defined value for this
    variable
    fhir:TestScript.variable.description [ string ]; # 0..1 Natural language description of the variable
    fhir:TestScript.variable.expression [ string ]; # 0..1 The fluentpath expression against the fixture body
    fhir:TestScript.variable.headerField [ string ]; # 0..1 HTTP header field name for source
    fhir:TestScript.variable.hint [ string ]; # 0..1 Hint help text for default value to enter
    fhir:TestScript.variable.path [ string ]; # 0..1 XPath or JSONPath against the fixture body
    fhir:TestScript.variable.sourceId [ id ]; # 0..1 Fixture Id of source expression or headerField within this variable
  ], ...;
```

```
fhir:TestScript.rule [ # 0..* Assert rule used within the test script
  fhir:TestScript.rule.resource [ Reference(Any) ]; # 1..1 Assert rule resource reference
  fhir:TestScript.rule.param [ # 0..* Rule parameter template
    fhir:TestScript.rule.param.name [ string ]; # 1..1 Parameter name matching external assert rule parameter
    fhir:TestScript.rule.param.value [ string ]; # 0..1 Parameter value defined either explicitly or dynamically
  ], ...;
], ...;
fhir:TestScript.ruleset [ # 0..* Assert ruleset used within the test script
  fhir:TestScript.ruleset.resource [ Reference(Any) ]; # 1..1 Assert ruleset resource reference
  fhir:TestScript.ruleset.rule [ # 1..* The referenced rule within the ruleset
    fhir:TestScript.ruleset.rule.ruleId [ id ]; # 1..1 Id of referenced rule within the ruleset
    fhir:TestScript.ruleset.rule.param [ # 0..* Ruleset rule parameter template
      fhir:TestScript.ruleset.rule.param.name [ string ]; # 1..1 Parameter name matching external assert ruleset rule
  parameter
      fhir:TestScript.ruleset.rule.param.value [ string ]; # 0..1 Parameter value defined either explicitly or
  dynamically
    ], ...;
  ], ...;
], ...;
fhir:TestScript.setup [ # 0..1 A series of required setup operations before tests are executed
  fhir:TestScript.setup.action [ # 1..* A setup operation or assert to perform
    fhir:TestScript.setup.action.operation [ # 0..1 The setup operation to perform
      fhir:TestScript.setup.action.operation.type [ Coding ]; # 0..1 The operation code type that will be executed
      fhir:TestScript.setup.action.operation.resource [ code ]; # 0..1 Resource type
      fhir:TestScript.setup.action.operation.label [ string ]; # 0..1 Tracking/logging operation label
      fhir:TestScript.setup.action.operation.description [ string ]; # 0..1 Tracking/reporting operation description
      fhir:TestScript.setup.action.operation.accept [ code ]; # 0..1 xml | json | ttl | none
      fhir:TestScript.setup.action.operation.contentType [ code ]; # 0..1 xml | json | ttl | none
      fhir:TestScript.setup.action.operation.destination [ integer ]; # 0..1 Server responding to the request
      fhir:TestScript.setup.action.operation.encodeRequestUrl [ boolean ]; # 0..1 Whether or not to send the request
  url in encoded format
      fhir:TestScript.setup.action.operation.origin [ integer ]; # 0..1 Server initiating the request
      fhir:TestScript.setup.action.operation.params [ string ]; # 0..1 Explicitly defined path parameters
      fhir:TestScript.setup.action.operation.requestHeader [ # 0..* Each operation can have one or more header
  elements
        fhir:TestScript.setup.action.operation.requestHeader.field [ string ]; # 1..1 HTTP header field name
        fhir:TestScript.setup.action.operation.requestHeader.value [ string ]; # 1..1 HTTP headerfield value
      ], ...;
      fhir:TestScript.setup.action.operation.requestId [ id ]; # 0..1 Fixture Id of mapped request
      fhir:TestScript.setup.action.operation.responseId [ id ]; # 0..1 Fixture Id of mapped response
      fhir:TestScript.setup.action.operation.sourceId [ id ]; # 0..1 Fixture Id of body for PUT and POST requests
      fhir:TestScript.setup.action.operation.targetId [ id ]; # 0..1 Id of fixture used for extracting the [id],
  [type], and [vid] for GET requests
      fhir:TestScript.setup.action.operation.url [ string ]; # 0..1 Request URL
    ];
    fhir:TestScript.setup.action.assert [ # 0..1 The assertion to perform
      fhir:TestScript.setup.action.assert.label [ string ]; # 0..1 Tracking/logging assertion label
      fhir:TestScript.setup.action.assert.description [ string ]; # 0..1 Tracking/reporting assertion description
      fhir:TestScript.setup.action.assert.direction [ code ]; # 0..1 response | request
      fhir:TestScript.setup.action.assert.compareToSourceId [ string ]; # 0..1 Id of the source fixture to be
  evaluated
      fhir:TestScript.setup.action.assert.compareToSourceExpression [ string ]; # 0..1 The fluentpath expression to
  evaluate against the source fixture
      fhir:TestScript.setup.action.assert.compareToSourcePath [ string ]; # 0..1 XPath or JSONPath expression to
  evaluate against the source fixture
      fhir:TestScript.setup.action.assert.contentType [ code ]; # 0..1 xml | json | ttl | none
      fhir:TestScript.setup.action.assert.expression [ string ]; # 0..1 The fluentpath expression to be evaluated
      fhir:TestScript.setup.action.assert.headerField [ string ]; # 0..1 HTTP header field name
      fhir:TestScript.setup.action.assert.minimumId [ string ]; # 0..1 Fixture Id of minimum content resource
      fhir:TestScript.setup.action.assert.navigationLinks [ boolean ]; # 0..1 Perform validation on navigation links?
      fhir:TestScript.setup.action.assert.operator [ code ]; # 0..1 equals | notEquals | in | notIn | greaterThan |
  lessThan | empty | notEmpty | contains | notContains | eval
      fhir:TestScript.setup.action.assert.path [ string ]; # 0..1 XPath or JSONPath expression
      fhir:TestScript.setup.action.assert.requestMethod [ code ]; # 0..1 delete | get | options | patch | post | put
      fhir:TestScript.setup.action.assert.requestURL [ string ]; # 0..1 Request URL comparison value
      fhir:TestScript.setup.action.assert.resource [ code ]; # 0..1 Resource type
      fhir:TestScript.setup.action.assert.response [ code ]; # 0..1 okay | created | noContent | notModified | bad |
  forbidden | notFound | methodNotAllowed | conflict | gone | preconditionFailed | unprocessable
      fhir:TestScript.setup.action.assert.responseCode [ string ]; # 0..1 HTTP response code to test
      fhir:TestScript.setup.action.assert.rule [ # 0..1 The reference to a TestScript.rule
        fhir:TestScript.setup.action.assert.rule.ruleId [ id ]; # 1..1 Id of the TestScript.rule
        fhir:TestScript.setup.action.assert.rule.param [ # 0..* Rule parameter template
          fhir:TestScript.setup.action.assert.rule.param.name [ string ]; # 1..1 Parameter name matching external
  assert rule parameter
          fhir:TestScript.setup.action.assert.rule.param.value [ string ]; # 1..1 Parameter value defined either
  explicitly or dynamically
        ], ...;
      ];
      fhir:TestScript.setup.action.assert.ruleset [ # 0..1 The reference to a TestScript.ruleset
        fhir:TestScript.setup.action.assert.ruleset.rulesetId [ id ]; # 1..1 Id of the TestScript.ruleset
        fhir:TestScript.setup.action.assert.ruleset.rule [ # 0..* The referenced rule within the ruleset
          fhir:TestScript.setup.action.assert.ruleset.rule.ruleId [ id ]; # 1..1 Id of referenced rule within the
  ruleset
          fhir:TestScript.setup.action.assert.ruleset.rule.param [ # 0..* Rule parameter template
            fhir:TestScript.setup.action.assert.ruleset.rule.param.name [ string ]; # 1..1 Parameter name matching
  external assert ruleset rule parameter
            fhir:TestScript.setup.action.assert.ruleset.rule.param.value [ string ]; # 1..1 Parameter value defined
  either explicitly or dynamically
          ], ...;
```

```
        ], ...;
      ];
      fhir:TestScript.setup.action.assert.sourceId [ id ]; # 0..1 Fixture Id of source expression or headerField
      fhir:TestScript.setup.action.assert.validateProfileId [ id ]; # 0..1 Profile Id of validation profile reference
      fhir:TestScript.setup.action.assert.value [ string ]; # 0..1 The value to compare to
      fhir:TestScript.setup.action.assert.warningOnly [ boolean ]; # 0..1 Will this assert produce a warning only on
    error?
      ];
    ], ...;
  ];
  fhir:TestScript.test [ # 0..* A test in this script
    fhir:TestScript.test.name [ string ]; # 0..1 Tracking/logging name of this test
    fhir:TestScript.test.description [ string ]; # 0..1 Tracking/reporting short description of the test
    fhir:TestScript.test.action [ # 1..* A test operation or assert to perform
      fhir:TestScript.test.action.operation [ See TestScript.setup.action.operation ]; # 0..1 The setup operation to
    perform
      fhir:TestScript.test.action.assert [ See TestScript.setup.action.assert ]; # 0..1 The setup assertion to perform
    ], ...;
  ], ...;
  fhir:TestScript.teardown [ # 0..1 A series of required clean up steps
    fhir:TestScript.teardown.action [ # 1..* One or more teardown operations to perform
      fhir:TestScript.teardown.action.operation [ See TestScript.setup.action.operation ]; # 1..1 The teardown operation
     to perform
    ], ...;
  ];
]
```

Changes since DSTU2

TestScript
TestScript.title | Added Element
TestScript.status | Change value set from http://hl7.org/fhir/ValueSet/conformance-resource-status to http://hl7.org/fhir/ValueSet/publication-status
TestScript.experimental | Now marked as Modifier
TestScript.contact | Type changed from BackboneElement to ContactDetail
TestScript.description | Type changed from string to markdown
TestScript.useContext | Type changed from CodeableConcept to UsageContext Remove Binding http://hl7.org/fhir/ValueSet/use-context (extensible)
TestScript.jurisdiction | Added Element
TestScript.purpose | Added Element
TestScript.copyright | Type changed from string to markdown
TestScript.origin | Added Element
TestScript.origin.index | Added Element
TestScript.origin.profile | Added Element
TestScript.destination | Added Element
TestScript.destination.index | Added Element
TestScript.destination.profile | Added Element
TestScript.metadata.capability.origin | Added Element
TestScript.metadata.capability.capabilities | Added Element
TestScript.fixture.autocreate | Default Value "false" added
TestScript.fixture.autodelete | Default Value "false" added
TestScript.variable.defaultValue | Added Element
TestScript.variable.description | Added Element
TestScript.variable.expression | Added Element
TestScript.variable.hint | Added Element
TestScript.rule | Added Element
TestScript.rule.resource | Added Element
TestScript.rule.param | Added Element
TestScript.rule.param.name | Added Element
TestScript.rule.param.value | Added Element
TestScript.ruleset | Added Element
TestScript.ruleset.resource | Added Element
TestScript.ruleset.rule | Added Element
TestScript.ruleset.rule.ruleId | Added Element
TestScript.ruleset.rule.param | Added Element
TestScript.ruleset.rule.param.name | Added Element
TestScript.ruleset.rule.param.value | Added Element
TestScript.setup.action.operation.destination | Default Value "0" removed
TestScript.setup.action.operation.origin | Added Element
TestScript.setup.action.operation.requestId | Added Element
TestScript.setup.action.assert.compareToSourceExpression | Added Element
TestScript.setup.action.assert.expression | Added Element
TestScript.setup.action.assert.requestMethod | Added Element
TestScript.setup.action.assert.requestURL | Added Element
TestScript.setup.action.assert.rule | Added Element
TestScript.setup.action.assert.rule.ruleId | Added Element
TestScript.setup.action.assert.rule.param | Added Element
TestScript.setup.action.assert.rule.param.name | Added Element
TestScript.setup.action.assert.rule.param.value | Added Element
TestScript.setup.action.assert.ruleset | Added Element

TestScript.setup.action.assert.ruleset.rulesetId | Added Element
TestScript.setup.action.assert.ruleset.rule | Added Element
TestScript.setup.action.assert.ruleset.rule.ruleId | Added Element
TestScript.setup.action.assert.ruleset.rule.param | Added Element
TestScript.setup.action.assert.ruleset.rule.param.name | Added Element
TestScript.setup.action.assert.ruleset.rule.param.value | Added Element
TestScript.teardown.action.operation | Min Cardinality changed from 0 to 1
TestScript.contact.name | deleted
TestScript.contact.telecom | deleted
TestScript.requirements | deleted
TestScript.metadata.capability.conformance | deleted
TestScript.multiserver | deleted
TestScript.setup.metadata | deleted
TestScript.test.metadata | deleted

See the Full Difference for further information

This analysis is available as XML or JSON.

See R2 <--> R3 Conversion Maps (status = 6 tests that all execute ok. 6 fail round-trip testing and 6 r3 resources are invalid (11 errors).).

Structure

Name | Flags | Card. | Type | Description & Constraints
TestScript | | | DomainResource | Describes a set of tests Elements defined in Ancestors: id, meta, implicitRules, language, text, contained, extension, modifierExtension
url | ? | 1..1 | uri | Logical URI to reference this test script (globally unique)
identifier | ? | 0..1 | Identifier | Additional identifier for the test script
version | ? | 0..1 | string | Business version of the test script
name | ? | 1..1 | string | Name for this test script (computer friendly)
title | ? | 0..1 | string | Name for this test script (human friendly)
status | ?!? | 1..1 | code | draft | active | retired | unknown PublicationStatus (Required)
experimental | ?!? | 0..1 | boolean | For testing purposes, not real usage
date | ? | 0..1 | dateTime | Date this was last changed
publisher | ? | 0..1 | string | Name of the publisher (organization or individual)
contact | ? | 0..* | ContactDetail | Contact details for the publisher
description | | 0..1 | markdown | Natural language description of the test script
useContext | ? | 0..* | UsageContext | Context the content is intended to support
jurisdiction | ? | 0..* | CodeableConcept | Intended jurisdiction for test script (if applicable) Jurisdiction ValueSet (Extensible)
purpose | | 0..1 | markdown | Why this test script is defined
copyright | | 0..1 | markdown | Use and/or publishing restrictions
origin | | 0..* | BackboneElement | An abstract server representing a client or sender in a message exchange
index | | 1..1 | integer | The index of the abstract origin server starting at 1
profile | | 1..1 | Coding | FHIR-Client | FHIR-SDC-FormFiller TestScriptProfileOriginType (Extensible)
destination | | 0..* | BackboneElement | An abstract server representing a destination or receiver in a message exchange
index | | 1..1 | integer | The index of the abstract destination server starting at 1
profile | | 1..1 | Coding | FHIR-Server | FHIR-SDC-FormManager | FHIR-SDC-FormReceiver | FHIR-SDC-FormProcessor TestScriptProfileDestinationType (Extensible)
metadata | I | 0..1 | BackboneElement | Required capability that is assumed to function correctly on the FHIR server being tested + TestScript metadata capability SHALL contain required or validated or both.
link | | 0..* | BackboneElement | Links to the FHIR specification
url | | 1..1 | uri | URL to the specification
description | | 0..1 | string | Short description
capability | | 1..* | BackboneElement | Capabilities that are assumed to function correctly on the FHIR server being tested
required | | 0..1 | boolean | Are the capabilities required?
validated | | 0..1 | boolean | Are the capabilities validated?
description | | 0..1 | string | The expected capabilities of the server
origin | | 0..* | integer | Which origin server these requirements apply to
destination | | 0..1 | integer | Which server these requirements apply to
link | | 0..* | uri | Links to the FHIR specification
capabilities | | 1..1 | Reference(CapabilityStatement) | Required Capability Statement
fixture | | 0..* | BackboneElement | Fixture in the test script - by reference (uri)
autocreate | | 0..1 | boolean | Whether or not to implicitly create the fixture during setup
autodelete | | 0..1 | boolean | Whether or not to implicitly delete the fixture during teardown
resource | | 0..1 | Reference(Any) | Reference of the resource
profile | | 0..* | Reference(Any) | Reference of the validation profile
variable | I | 0..* | BackboneElement | Placeholder for evaluated elements + Variable can only contain one of expression, headerField or path.
name | | 1..1 | string | Descriptive name for this variable
defaultValue | | 0..1 | string | Default, hard-coded, or user-defined value for this variable
description | | 0..1 | string | Natural language description of the variable
expression | | 0..1 | string | The fluentpath expression against the fixture body
headerField | | 0..1 | string | HTTP header field name for source
hint | | 0..1 | string | Hint help text for default value to enter
path | | 0..1 | string | XPath or JSONPath against the fixture body
sourceId | | 0..1 | id | Fixture Id of source expression or headerField within this variable

rule | | 0..* | BackboneElement | Assert rule used within the test script
resource | | 1..1 | Reference(Any) | Assert rule resource reference
param | | 0..* | BackboneElement | Rule parameter template
name | | 1..1 | string | Parameter name matching external assert rule parameter
value | | 0..1 | string | Parameter value defined either explicitly or dynamically
ruleset | | 0..* | BackboneElement | Assert ruleset used within the test script
resource | | 1..1 | Reference(Any) | Assert ruleset resource reference
rule | | 1..* | BackboneElement | The referenced rule within the ruleset
ruleId | | 1..1 | id | Id of referenced rule within the ruleset
param | | 0..* | BackboneElement | Ruleset rule parameter template
name | | 1..1 | string | Parameter name matching external assert ruleset rule parameter
value | | 0..1 | string | Parameter value defined either explicitly or dynamically
setup | | 0..1 | BackboneElement | A series of required setup operations before tests are executed
action | I | 1..* | BackboneElement | A setup operation or assert to perform + Setup action SHALL contain either an operation or assert but not both.
operation | I | 0..1 | BackboneElement | The setup operation to perform + Setup operation SHALL contain either sourceId or targetId or params or url.
type | | 0..1 | Coding | The operation code type that will be executed TestScriptOperationCode (Extensible)
resource | | 0..1 | code | Resource type FHIRDefinedType (Required)
label | | 0..1 | string | Tracking/logging operation label
description | | 0..1 | string | Tracking/reporting operation description
accept | | 0..1 | code | xml | json | ttl | none ContentType (Required)
contentType | | 0..1 | code | xml | json | ttl | none ContentType (Required)
destination | | 0..1 | integer | Server responding to the request
encodeRequestUrl | | 0..1 | boolean | Whether or not to send the request url in encoded format
origin | | 0..1 | integer | Server initiating the request
params | | 0..1 | string | Explicitly defined path parameters
requestHeader | | 0..* | BackboneElement | Each operation can have one or more header elements
field | | 1..1 | string | HTTP header field name
value | | 1..1 | string | HTTP headerfield value
requestId | | 0..1 | id | Fixture Id of mapped request
responseId | | 0..1 | id | Fixture Id of mapped response
sourceId | | 0..1 | id | Fixture Id of body for PUT and POST requests
targetId | | 0..1 | id | Id of fixture used for extracting the [id], [type], and [vid] for GET requests
url | | 0..1 | string | Request URL
assert | I | 0..1 | BackboneElement | The assertion to perform + Only a single assertion SHALL be present within setup action assert element. + Setup action assert SHALL contain either compareToSourceId and compareToSourceExpression, compareToSourceId and compareToSourcePath or neither. + Setup action assert response and responseCode SHALL be empty when direction equals request
label | | 0..1 | string | Tracking/logging assertion label
description | | 0..1 | string | Tracking/reporting assertion description
direction | | 0..1 | code | response | request AssertionDirectionType (Required)
compareToSourceId | | 0..1 | string | Id of the source fixture to be evaluated
compareToSourceExpression | | 0..1 | string | The fluentpath expression to evaluate against the source fixture
compareToSourcePath | | 0..1 | string | XPath or JSONPath expression to evaluate against the source fixture
contentType | | 0..1 | code | xml | json | ttl | none ContentType (Required)
expression | | 0..1 | string | The fluentpath expression to be evaluated
headerField | | 0..1 | string | HTTP header field name
minimumId | | 0..1 | string | Fixture Id of minimum content resource
navigationLinks | | 0..1 | boolean | Perform validation on navigation links?
operator | | 0..1 | code | equals | notEquals | in | notIn | greaterThan | lessThan | empty | notEmpty | contains | notContains | eval AssertionOperatorType (Required)
path | | 0..1 | string | XPath or JSONPath expression
requestMethod | | 0..1 | code | delete | get | options | patch | post | put TestScriptRequestMethodCode (Required)
requestURL | | 0..1 | string | Request URL comparison value
resource | | 0..1 | code | Resource type FHIRDefinedType (Required)
response | | 0..1 | code | okay | created | noContent | notModified | bad | forbidden | notFound | methodNotAllowed | conflict | gone | preconditionFailed | unprocessable AssertionResponseTypes (Required)
responseCode | | 0..1 | string | HTTP response code to test
rule | | 0..1 | BackboneElement | The reference to a TestScript.rule
ruleId | | 1..1 | id | Id of the TestScript.rule
param | | 0..* | BackboneElement | Rule parameter template
name | | 1..1 | string | Parameter name matching external assert rule parameter
value | | 1..1 | string | Parameter value defined either explicitly or dynamically
ruleset | | 0..1 | BackboneElement | The reference to a TestScript.ruleset
rulesetId | | 1..1 | id | Id of the TestScript.ruleset
rule | | 0..* | BackboneElement | The referenced rule within the ruleset
ruleId | | 1..1 | id | Id of referenced rule within the ruleset
param | | 0..* | BackboneElement | Rule parameter template
name | | 1..1 | string | Parameter name matching external assert ruleset rule parameter
value | | 1..1 | string | Parameter value defined either explicitly or dynamically
sourceId | | 0..1 | id | Fixture Id of source expression or headerField
validateProfileId | | 0..1 | id | Profile Id of validation profile reference
value | | 0..1 | string | The value to compare to
warningOnly | | 0..1 | boolean | Will this assert produce a warning only on error?
test | | 0..* | BackboneElement | A test in this script

name | | 0..1 | string | Tracking/logging name of this test

description | | 0..1 | string | Tracking/reporting short description of the test

action | I | 1..* | BackboneElement | A test operation or assert to perform + Test action SHALL contain either an operation or assert but not both.

operation | I | 0..1 | see operation | The setup operation to perform + Test operation SHALL contain either sourceId or targetId or params or url.

assert | I | 0..1 | see assert | The setup assertion to perform + Only a single assertion SHALL be present within test action assert element. + Test action assert SHALL contain either compareToSourceId and compareToSourceExpression, compareToSourceId and compareToSourcePath or neither. + Test action assert response and response and responseCode SHALL be empty when direction equals request

teardown | | 0..1 | BackboneElement | A series of required clean up steps

action | | 1..* | BackboneElement | One or more teardown operations to perform

operation | I | 1..1 | see operation | The teardown operation to perform + Teardown operation SHALL contain either sourceId or targetId or params or url.

Documentation for this format

UML Diagram (Legend)

XML Template

```
<TestScript xmlns="http://hl7.org/fhir">
 <!-- from Resource: id, meta, implicitRules, and language -->
 <!-- from DomainResource: text, contained, extension, and modifierExtension -->
 <url value="[uri]"/><!-- 1..1 Logical URI to reference this test script (globally unique) -->
 <identifier><!-- 0..1 Identifier Additional identifier for the test script --></identifier>
 <version value="[string]"/><!-- 0..1 Business version of the test script -->
 <name value="[string]"/><!-- 1..1 Name for this test script (computer friendly) -->
 <title value="[string]"/><!-- 0..1 Name for this test script (human friendly) -->
 <status value="[code]"/><!-- 1..1 draft | active | retired | unknown -->
 <experimental value="[boolean]"/><!-- 0..1 For testing purposes, not real usage -->
 <date value="[dateTime]"/><!-- 0..1 Date this was last changed -->
 <publisher value="[string]"/><!-- 0..1 Name of the publisher (organization or individual) -->
 <contact><!-- 0..* ContactDetail Contact details for the publisher --></contact>
 <description value="[markdown]"/><!-- 0..1 Natural language description of the test script -->
 <useContext><!-- 0..* UsageContext Context the content is intended to support --></useContext>
 <jurisdiction><!-- 0..* CodeableConcept Intended jurisdiction for test script (if applicable) --></jurisdiction>
 <purpose value="[markdown]"/><!-- 0..1 Why this test script is defined -->
 <copyright value="[markdown]"/><!-- 0..1 Use and/or publishing restrictions -->
 <origin>  <!-- 0..* An abstract server representing a client or sender in a message exchange -->
  <index value="[integer]"/><!-- 1..1 The index of the abstract origin server starting at 1 -->
  <profile><!-- 1..1 Coding FHIR-Client | FHIR-SDC-FormFiller --></profile>
 </origin>
 <destination>  <!-- 0..* An abstract server representing a destination or receiver in a message exchange -->
  <index value="[integer]"/><!-- 1..1 The index of the abstract destination server starting at 1 -->
  <profile><!-- 1..1 Coding FHIR-Server | FHIR-SDC-FormManager | FHIR-SDC-FormReceiver | FHIR-SDC-FormProcessor
    --></profile>
 </destination>
 <metadata>  <!-- 0..1 Required capability that is assumed to function correctly on the FHIR server being tested -->
  <link>  <!-- 0..* Links to the FHIR specification -->
   <url value="[uri]"/><!-- 1..1 URL to the specification -->
   <description value="[string]"/><!-- 0..1 Short description -->
  </link>
  <capability>  <!-- 1..* Capabilities  that are assumed to function correctly on the FHIR server being tested -->
   <required value="[boolean]"/><!-- 0..1 Are the capabilities required? -->
   <validated value="[boolean]"/><!-- 0..1 Are the capabilities validated? -->
   <description value="[string]"/><!-- 0..1 The expected capabilities of the server -->
   <origin value="[integer]"/><!-- 0..* Which origin server these requirements apply to -->
   <destination value="[integer]"/><!-- 0..1 Which server these requirements apply to -->
   <link value="[uri]"/><!-- 0..* Links to the FHIR specification -->
   <capabilities><!-- 1..1 Reference(CapabilityStatement) Required Capability Statement --></capabilities>
  </capability>
 </metadata>
 <fixture>  <!-- 0..* Fixture in the test script - by reference (uri) -->
  <autocreate value="[boolean]"/><!-- 0..1 Whether or not to implicitly create the fixture during setup -->
  <autodelete value="[boolean]"/><!-- 0..1 Whether or not to implicitly delete the fixture during teardown -->
  <resource><!-- 0..1 Reference(Any) Reference of the resource --></resource>
 </fixture>
 <profile><!-- 0..* Reference(Any) Reference of the validation profile --></profile>
 <variable>  <!-- 0..* Placeholder for evaluated elements -->
  <name value="[string]"/><!-- 1..1 Descriptive name for this variable -->
  <defaultValue value="[string]"/><!-- 0..1 Default, hard-coded, or user-defined value for this variable -->
  <description value="[string]"/><!-- 0..1 Natural language description of the variable -->
  <expression value="[string]"/><!-- 0..1 The fluentpath expression against the fixture body -->
  <headerField value="[string]"/><!-- 0..1 HTTP header field name for source -->
  <hint value="[string]"/><!-- 0..1 Hint help text for default value to enter -->
  <path value="[string]"/><!-- 0..1 XPath or JSONPath against the fixture body -->
  <sourceId value="[id]"/><!-- 0..1 Fixture Id of source expression or headerField within this variable -->
 </variable>
 <rule>  <!-- 0..* Assert rule used within the test script -->
  <resource><!-- 1..1 Reference(Any) Assert rule resource reference --></resource>
  <param>  <!-- 0..* Rule parameter template -->
   <name value="[string]"/><!-- 1..1 Parameter name matching external assert rule parameter -->
   <value value="[string]"/><!-- 0..1 Parameter value defined either explicitly or dynamically -->
  </param>
 </rule>
 <ruleset>  <!-- 0..* Assert ruleset used within the test script -->
  <resource><!-- 1..1 Reference(Any) Assert ruleset resource reference --></resource>
  <rule>  <!-- 1..* The referenced rule within the ruleset -->
   <ruleId value="[id]"/><!-- 1..1 Id of referenced rule within the ruleset -->
   <param>  <!-- 0..* Ruleset rule parameter template -->
    <name value="[string]"/><!-- 1..1 Parameter name matching external assert ruleset rule parameter -->
```

```
    <value value="[string]"/><!-- 0..1 Parameter value defined either explicitly or dynamically -->
   </param>
  </rule>
 </ruleset>
 <setup>  <!-- 0..1 A series of required setup operations before tests are executed -->
  <action>  <!-- 1..* A setup operation or assert to perform -->
   <operation>  <!-- 0..1 The setup operation to perform -->
    <type><!-- 0..1 Coding The operation code type that will be executed --></type>
    <resource value="[code]"/><!-- 0..1 Resource type -->
    <label value="[string]"/><!-- 0..1 Tracking/logging operation label -->
    <description value="[string]"/><!-- 0..1 Tracking/reporting operation description -->
    <accept value="[code]"/><!-- 0..1 xml | json | ttl | none -->
    <contentType value="[code]"/><!-- 0..1 xml | json | ttl | none -->
    <destination value="[integer]"/><!-- 0..1 Server responding to the request -->
    <encodeRequestUrl value="[boolean]"/><!-- 0..1 Whether or not to send the request url in encoded format -->
    <origin value="[integer]"/><!-- 0..1 Server initiating the request -->
    <params value="[string]"/><!-- 0..1 Explicitly defined path parameters -->
    <requestHeader>  <!-- 0..* Each operation can have one or more header elements -->
     <field value="[string]"/><!-- 1..1 HTTP header field name -->
     <value value="[string]"/><!-- 1..1 HTTP headerfield value -->
    </requestHeader>
    <requestId value="[id]"/><!-- 0..1 Fixture Id of mapped request -->
    <responseId value="[id]"/><!-- 0..1 Fixture Id of mapped response -->
    <sourceId value="[id]"/><!-- 0..1 Fixture Id of body for PUT and POST requests -->
    <targetId value="[id]"/><!-- 0..1 Id of fixture used for extracting the [id],  [type], and [vid] for GET requests
    -->
    <url value="[string]"/><!-- 0..1 Request URL -->
   </operation>
   <assert>  <!-- 0..1 The assertion to perform -->
    <label value="[string]"/><!-- 0..1 Tracking/logging assertion label -->
    <description value="[string]"/><!-- 0..1 Tracking/reporting assertion description -->
    <direction value="[code]"/><!-- 0..1 response | request -->
    <compareToSourceId value="[string]"/><!-- 0..1 Id of the source fixture to be evaluated -->
    <compareToSourceExpression value="[string]"/><!-- 0..1 The fluentpath expression to evaluate against the source
    fixture -->
    <compareToSourcePath value="[string]"/><!-- 0..1 XPath or JSONPath expression to evaluate against the source fixture
     -->
    <contentType value="[code]"/><!-- 0..1 xml | json | ttl | none -->
    <expression value="[string]"/><!-- 0..1 The fluentpath expression to be evaluated -->
    <headerField value="[string]"/><!-- 0..1 HTTP header field name -->
    <minimumId value="[string]"/><!-- 0..1 Fixture Id of minimum content resource -->
    <navigationLinks value="[boolean]"/><!-- 0..1 Perform validation on navigation links? -->
    <operator value="[code]"/><!-- 0..1 equals | notEquals | in | notIn | greaterThan | lessThan | empty | notEmpty |
    contains | notContains | eval -->
    <path value="[string]"/><!-- 0..1 XPath or JSONPath expression -->
    <requestMethod value="[code]"/><!-- 0..1 delete | get | options | patch | post | put -->
    <requestURL value="[string]"/><!-- 0..1 Request URL comparison value -->
    <resource value="[code]"/><!-- 0..1 Resource type -->
    <response value="[code]"/><!-- 0..1 okay | created | noContent | notModified | bad | forbidden | notFound |
    methodNotAllowed | conflict | gone | preconditionFailed | unprocessable -->
    <responseCode value="[string]"/><!-- 0..1 HTTP response code to test -->
    <rule>  <!-- 0..1 The reference to a TestScript.rule -->
     <ruleId value="[id]"/><!-- 1..1 Id of the TestScript.rule -->
     <param>  <!-- 0..* Rule parameter template -->
      <name value="[string]"/><!-- 1..1 Parameter name matching external assert rule parameter -->
      <value value="[string]"/><!-- 1..1 Parameter value defined either explicitly or dynamically -->
     </param>
    </rule>
    <ruleset>  <!-- 0..1 The reference to a TestScript.ruleset -->
     <rulesetId value="[id]"/><!-- 1..1 Id of the TestScript.ruleset -->
     <rule>  <!-- 0..* The referenced rule within the ruleset -->
      <ruleId value="[id]"/><!-- 1..1 Id of referenced rule within the ruleset -->
      <param>  <!-- 0..* Rule parameter template -->
       <name value="[string]"/><!-- 1..1 Parameter name matching external assert ruleset rule parameter -->
       <value value="[string]"/><!-- 1..1 Parameter value defined either explicitly or dynamically -->
      </param>
     </rule>
    </ruleset>
    <sourceId value="[id]"/><!-- 0..1 Fixture Id of source expression or headerField -->
    <validateProfileId value="[id]"/><!-- 0..1 Profile Id of validation profile reference -->
    <value value="[string]"/><!-- 0..1 The value to compare to -->
    <warningOnly value="[boolean]"/><!-- 0..1 Will this assert produce a warning only on error? -->
   </assert>
  </action>
 </setup>
 <test>  <!-- 0..* A test in this script -->
  <name value="[string]"/><!-- 0..1 Tracking/logging name of this test -->
  <description value="[string]"/><!-- 0..1 Tracking/reporting short description of the test -->
  <action>  <!-- 1..* A test operation or assert to perform -->
   <operation><!-- 0..1 Content as for TestScript.setup.action.operation The setup operation to perform --></operation>
   <assert><!-- 0..1 Content as for TestScript.setup.action.assert The setup assertion to perform --></assert>
  </action>
 </test>
 <teardown>  <!-- 0..1 A series of required clean up steps -->
  <action>  <!-- 1..* One or more teardown operations to perform -->
   <operation><!-- 1..1 Content as for TestScript.setup.action.operation The teardown operation to perform
    --></operation>
  </action>
 </teardown>
```

```
</TestScript>
```

JSON Template

```
{
  "resourceType" : "TestScript",
  // from Resource: id, meta, implicitRules, and language
  // from DomainResource: text, contained, extension, and modifierExtension
  "url" : "<uri>", // R!  Logical URI to reference this test script (globally unique)
  "identifier" : { Identifier }, // Additional identifier for the test script
  "version" : "<string>", // Business version of the test script
  "name" : "<string>", // R!  Name for this test script (computer friendly)
  "title" : "<string>", // Name for this test script (human friendly)
  "status" : "<code>", // R!  draft | active | retired | unknown
  "experimental" : <boolean>, // For testing purposes, not real usage
  "date" : "<dateTime>", // Date this was last changed
  "publisher" : "<string>", // Name of the publisher (organization or individual)
  "contact" : [{ ContactDetail }], // Contact details for the publisher
  "description" : "<markdown>", // Natural language description of the test script
  "useContext" : [{ UsageContext }], // Context the content is intended to support
  "jurisdiction" : [{ CodeableConcept }], // Intended jurisdiction for test script (if applicable)
  "purpose" : "<markdown>", // Why this test script is defined
  "copyright" : "<markdown>", // Use and/or publishing restrictions
  "origin" : [{ // An abstract server representing a client or sender in a message exchange
    "index" : <integer>, // R!  The index of the abstract origin server starting at 1
    "profile" : { Coding } // R!  FHIR-Client | FHIR-SDC-FormFiller
  }],
  "destination" : [{ // An abstract server representing a destination or receiver in a message exchange
    "index" : <integer>, // R!  The index of the abstract destination server starting at 1
    "profile" : { Coding } // R!  FHIR-Server | FHIR-SDC-FormManager | FHIR-SDC-FormReceiver | FHIR-SDC-FormProcessor
  }],
  "metadata" : { // Required capability that is assumed to function correctly on the FHIR server being tested
    "link" : [{ // Links to the FHIR specification
      "url" : "<uri>", // R!  URL to the specification
      "description" : "<string>" // Short description
    }],
    "capability" : [{ // R!  Capabilities  that are assumed to function correctly on the FHIR server being tested
      "required" : <boolean>, // Are the capabilities required?
      "validated" : <boolean>, // Are the capabilities validated?
      "description" : "<string>", // The expected capabilities of the server
      "origin" : [<integer>], // Which origin server these requirements apply to
      "destination" : <integer>, // Which server these requirements apply to
      "link" : ["<uri>"], // Links to the FHIR specification
      "capabilities" : { Reference(CapabilityStatement) } // R!  Required Capability Statement
    }]
  },
  "fixture" : [{ // Fixture in the test script - by reference (uri)
    "autocreate" : <boolean>, // Whether or not to implicitly create the fixture during setup
    "autodelete" : <boolean>, // Whether or not to implicitly delete the fixture during teardown
    "resource" : { Reference(Any) } // Reference of the resource
  }],
  "profile" : [{ Reference(Any) }], // Reference of the validation profile
  "variable" : [{ // Placeholder for evaluated elements
    "name" : "<string>", // R!  Descriptive name for this variable
    "defaultValue" : "<string>", // Default, hard-coded, or user-defined value for this variable
    "description" : "<string>", // Natural language description of the variable
    "expression" : "<string>", // The fluentpath expression against the fixture body
    "headerField" : "<string>", // HTTP header field name for source
    "hint" : "<string>", // Hint help text for default value to enter
    "path" : "<string>", // XPath or JSONPath against the fixture body
    "sourceId" : "<id>" // Fixture Id of source expression or headerField within this variable
  }],
  "rule" : [{ // Assert rule used within the test script
    "resource" : { Reference(Any) }, // R!  Assert rule resource reference
    "param" : [{ // Rule parameter template
      "name" : "<string>", // R!  Parameter name matching external assert rule parameter
      "value" : "<string>" // Parameter value defined either explicitly or dynamically
    }]
  }],
  "ruleset" : [{ // Assert ruleset used within the test script
    "resource" : { Reference(Any) }, // R!  Assert ruleset resource reference
    "rule" : [{ // R!  The referenced rule within the ruleset
      "ruleId" : "<id>", // R!  Id of referenced rule within the ruleset
      "param" : [{ // Ruleset rule parameter template
        "name" : "<string>", // R!  Parameter name matching external assert ruleset rule parameter
        "value" : "<string>" // Parameter value defined either explicitly or dynamically
      }]
    }]
  }],
  "setup" : { // A series of required setup operations before tests are executed
    "action" : [{ // R!  A setup operation or assert to perform
      "operation" : { // The setup operation to perform
        "type" : { Coding }, // The operation code type that will be executed
        "resource" : "<code>", // Resource type
        "label" : "<string>", // Tracking/logging operation label
        "description" : "<string>", // Tracking/reporting operation description
        "accept" : "<code>", // xml | json | ttl | none
        "contentType" : "<code>", // xml | json | ttl | none
        "destination" : <integer>, // Server responding to the request
        "encodeRequestUrl" : <boolean>, // Whether or not to send the request url in encoded format
```

```
      "origin" : <integer>, // Server initiating the request
      "params" : "<string>", // Explicitly defined path parameters
      "requestHeader" : [{ // Each operation can have one or more header elements
        "field" : "<string>", // R!  HTTP header field name
        "value" : "<string>" // R!  HTTP headerfield value
      }],
      "requestId" : "<id>", // Fixture Id of mapped request
      "responseId" : "<id>", // Fixture Id of mapped response
      "sourceId" : "<id>", // Fixture Id of body for PUT and POST requests
      "targetId" : "<id>", // Id of fixture used for extracting the [id],  [type], and [vid] for GET requests
      "url" : "<string>" // Request URL
    },
    "assert" : { // The assertion to perform
      "label" : "<string>", // Tracking/logging assertion label
      "description" : "<string>", // Tracking/reporting assertion description
      "direction" : "<code>", // response | request
      "compareToSourceId" : "<string>", // Id of the source fixture to be evaluated
      "compareToSourceExpression" : "<string>", // The fluentpath expression to evaluate against the source fixture
      "compareToSourcePath" : "<string>", // XPath or JSONPath expression to evaluate against the source fixture
      "contentType" : "<code>", // xml | json | ttl | none
      "expression" : "<string>", // The fluentpath expression to be evaluated
      "headerField" : "<string>", // HTTP header field name
      "minimumId" : "<string>", // Fixture Id of minimum content resource
      "navigationLinks" : <boolean>, // Perform validation on navigation links?
      "operator" : "<code>", // equals | notEquals | in | notIn | greaterThan | lessThan | empty | notEmpty | contains
    | notContains | eval
      "path" : "<string>", // XPath or JSONPath expression
      "requestMethod" : "<code>", // delete | get | options | patch | post | put
      "requestURL" : "<string>", // Request URL comparison value
      "resource" : "<code>", // Resource type
      "response" : "<code>", // okay | created | noContent | notModified | bad | forbidden | notFound |
  methodNotAllowed | conflict | gone | preconditionFailed | unprocessable
      "responseCode" : "<string>", // HTTP response code to test
      "rule" : { // The reference to a TestScript.rule
        "ruleId" : "<id>", // R!  Id of the TestScript.rule
        "param" : [{ // Rule parameter template
          "name" : "<string>", // R!  Parameter name matching external assert rule parameter
          "value" : "<string>" // R!  Parameter value defined either explicitly or dynamically
        }]
      },
      "ruleset" : { // The reference to a TestScript.ruleset
        "rulesetId" : "<id>", // R!  Id of the TestScript.ruleset
        "rule" : [{ // The referenced rule within the ruleset
          "ruleId" : "<id>", // R!  Id of referenced rule within the ruleset
          "param" : [{ // Rule parameter template
            "name" : "<string>", // R!  Parameter name matching external assert ruleset rule parameter
            "value" : "<string>" // R!  Parameter value defined either explicitly or dynamically
          }]
        }]
      },
      "sourceId" : "<id>", // Fixture Id of source expression or headerField
      "validateProfileId" : "<id>", // Profile Id of validation profile reference
      "value" : "<string>", // The value to compare to
      "warningOnly" : <boolean> // Will this assert produce a warning only on error?
    }
  }]
},
"test" : [{ // A test in this script
  "name" : "<string>", // Tracking/logging name of this test
  "description" : "<string>", // Tracking/reporting short description of the test
  "action" : [{ // R!  A test operation or assert to perform
    "operation" : { Content as for TestScript.setup.action.operation }, // The setup operation to perform
    "assert" : { Content as for TestScript.setup.action.assert } // The setup assertion to perform
  }]
}],
"teardown" : { // A series of required clean up steps
  "action" : [{ // R!  One or more teardown operations to perform
    "operation" : { Content as for TestScript.setup.action.operation } // R!  The teardown operation to perform
  }]
}
}
```

Turtle Template

```
@prefix fhir: <http://hl7.org/fhir/> .


[ a fhir:TestScript;
  fhir:nodeRole fhir:treeRoot; # if this is the parser root

  # from Resource: .id, .meta, .implicitRules, and .language
  # from DomainResource: .text, .contained, .extension, and .modifierExtension
  fhir:TestScript.url [ uri ]; # 1..1 Logical URI to reference this test script (globally unique)
  fhir:TestScript.identifier [ Identifier ]; # 0..1 Additional identifier for the test script
  fhir:TestScript.version [ string ]; # 0..1 Business version of the test script
  fhir:TestScript.name [ string ]; # 1..1 Name for this test script (computer friendly)
  fhir:TestScript.title [ string ]; # 0..1 Name for this test script (human friendly)
  fhir:TestScript.status [ code ]; # 1..1 draft | active | retired | unknown
  fhir:TestScript.experimental [ boolean ]; # 0..1 For testing purposes, not real usage
```

```
fhir:TestScript.date [ dateTime ]; # 0..1 Date this was last changed
fhir:TestScript.publisher [ string ]; # 0..1 Name of the publisher (organization or individual)
fhir:TestScript.contact [ ContactDetail ], ... ; # 0..* Contact details for the publisher
fhir:TestScript.description [ markdown ]; # 0..1 Natural language description of the test script
fhir:TestScript.useContext [ UsageContext ], ... ; # 0..* Context the content is intended to support
fhir:TestScript.jurisdiction [ CodeableConcept ], ... ; # 0..* Intended jurisdiction for test script (if applicable)
fhir:TestScript.purpose [ markdown ]; # 0..1 Why this test script is defined
fhir:TestScript.copyright [ markdown ]; # 0..1 Use and/or publishing restrictions
fhir:TestScript.origin [ # 0..* An abstract server representing a client or sender in a message exchange
  fhir:TestScript.origin.index [ integer ]; # 1..1 The index of the abstract origin server starting at 1
  fhir:TestScript.origin.profile [ Coding ]; # 1..1 FHIR-Client | FHIR-SDC-FormFiller
], ...;
fhir:TestScript.destination [ # 0..* An abstract server representing a destination or receiver in a message exchange
  fhir:TestScript.destination.index [ integer ]; # 1..1 The index of the abstract destination server starting at 1
  fhir:TestScript.destination.profile [ Coding ]; # 1..1 FHIR-Server | FHIR-SDC-FormManager | FHIR-SDC-FormReceiver |
  FHIR-SDC-FormProcessor
], ...;
fhir:TestScript.metadata [ # 0..1 Required capability that is assumed to function correctly on the FHIR server being
  tested
  fhir:TestScript.metadata.link [ # 0..* Links to the FHIR specification
    fhir:TestScript.metadata.link.url [ uri ]; # 1..1 URL to the specification
    fhir:TestScript.metadata.link.description [ string ]; # 0..1 Short description
  ], ...;
  fhir:TestScript.metadata.capability [ # 1..* Capabilities  that are assumed to function correctly on the FHIR server
   being tested
    fhir:TestScript.metadata.capability.required [ boolean ]; # 0..1 Are the capabilities required?
    fhir:TestScript.metadata.capability.validated [ boolean ]; # 0..1 Are the capabilities validated?
    fhir:TestScript.metadata.capability.description [ string ]; # 0..1 The expected capabilities of the server
    fhir:TestScript.metadata.capability.origin [ integer ], ... ; # 0..* Which origin server these requirements apply
  to
    fhir:TestScript.metadata.capability.destination [ integer ]; # 0..1 Which server these requirements apply to
    fhir:TestScript.metadata.capability.link [ uri ], ... ; # 0..* Links to the FHIR specification
    fhir:TestScript.metadata.capability.capabilities [ Reference(CapabilityStatement) ]; # 1..1 Required Capability
  Statement
  ], ...;
];
fhir:TestScript.fixture [ # 0..* Fixture in the test script - by reference (uri)
  fhir:TestScript.fixture.autocreate [ boolean ]; # 0..1 Whether or not to implicitly create the fixture during setup
  fhir:TestScript.fixture.autodelete [ boolean ]; # 0..1 Whether or not to implicitly delete the fixture during
  teardown
  fhir:TestScript.fixture.resource [ Reference(Any) ]; # 0..1 Reference of the resource
], ...;
fhir:TestScript.profile [ Reference(Any) ], ... ; # 0..* Reference of the validation profile
fhir:TestScript.variable [ # 0..* Placeholder for evaluated elements
  fhir:TestScript.variable.name [ string ]; # 1..1 Descriptive name for this variable
  fhir:TestScript.variable.defaultValue [ string ]; # 0..1 Default, hard-coded, or user-defined value for this
  variable
  fhir:TestScript.variable.description [ string ]; # 0..1 Natural language description of the variable
  fhir:TestScript.variable.expression [ string ]; # 0..1 The fluentpath expression against the fixture body
  fhir:TestScript.variable.headerField [ string ]; # 0..1 HTTP header field name for source
  fhir:TestScript.variable.hint [ string ]; # 0..1 Hint help text for default value to enter
  fhir:TestScript.variable.path [ string ]; # 0..1 XPath or JSONPath against the fixture body
  fhir:TestScript.variable.sourceId [ id ]; # 0..1 Fixture Id of source expression or headerField within this variable
], ...;
fhir:TestScript.rule [ # 0..* Assert rule used within the test script
  fhir:TestScript.rule.resource [ Reference(Any) ]; # 1..1 Assert rule resource reference
  fhir:TestScript.rule.param [ # 0..* Rule parameter template
    fhir:TestScript.rule.param.name [ string ]; # 1..1 Parameter name matching external assert rule parameter
    fhir:TestScript.rule.param.value [ string ]; # 0..1 Parameter value defined either explicitly or dynamically
  ], ...;
], ...;
fhir:TestScript.ruleset [ # 0..* Assert ruleset used within the test script
  fhir:TestScript.ruleset.resource [ Reference(Any) ]; # 1..1 Assert ruleset resource reference
  fhir:TestScript.ruleset.rule [ # 1..* The referenced rule within the ruleset
    fhir:TestScript.ruleset.rule.ruleId [ id ]; # 1..1 Id of referenced rule within the ruleset
    fhir:TestScript.ruleset.rule.param [ # 0..* Ruleset rule parameter template
      fhir:TestScript.ruleset.rule.param.name [ string ]; # 1..1 Parameter name matching external assert ruleset rule
  parameter
      fhir:TestScript.ruleset.rule.param.value [ string ]; # 0..1 Parameter value defined either explicitly or
  dynamically
    ], ...;
  ], ...;
], ...;
fhir:TestScript.setup [ # 0..1 A series of required setup operations before tests are executed
  fhir:TestScript.setup.action [ # 1..* A setup operation or assert to perform
    fhir:TestScript.setup.action.operation [ # 0..1 The setup operation to perform
      fhir:TestScript.setup.action.operation.type [ Coding ]; # 0..1 The operation code type that will be executed
      fhir:TestScript.setup.action.operation.resource [ code ]; # 0..1 Resource type
      fhir:TestScript.setup.action.operation.label [ string ]; # 0..1 Tracking/logging operation label
      fhir:TestScript.setup.action.operation.description [ string ]; # 0..1 Tracking/reporting operation description
      fhir:TestScript.setup.action.operation.accept [ code ]; # 0..1 xml | json | ttl | none
      fhir:TestScript.setup.action.operation.contentType [ code ]; # 0..1 xml | json | ttl | none
      fhir:TestScript.setup.action.operation.destination [ integer ]; # 0..1 Server responding to the request
      fhir:TestScript.setup.action.operation.encodeRequestUrl [ boolean ]; # 0..1 Whether or not to send the request
  url in encoded format
      fhir:TestScript.setup.action.operation.origin [ integer ]; # 0..1 Server initiating the request
      fhir:TestScript.setup.action.operation.params [ string ]; # 0..1 Explicitly defined path parameters
      fhir:TestScript.setup.action.operation.requestHeader [ # 0..* Each operation can have one or more header
  elements
```

```
        fhir:TestScript.setup.action.operation.requestHeader.field [ string ]; # 1..1 HTTP header field name
        fhir:TestScript.setup.action.operation.requestHeader.value [ string ]; # 1..1 HTTP headerfield value
      ], ...;
      fhir:TestScript.setup.action.operation.requestId [ id ]; # 0..1 Fixture Id of mapped request
      fhir:TestScript.setup.action.operation.responseId [ id ]; # 0..1 Fixture Id of mapped response
      fhir:TestScript.setup.action.operation.sourceId [ id ]; # 0..1 Fixture Id of body for PUT and POST requests
      fhir:TestScript.setup.action.operation.targetId [ id ]; # 0..1 Id of fixture used for extracting the [id],
  [type], and [vid] for GET requests
      fhir:TestScript.setup.action.operation.url [ string ]; # 0..1 Request URL
    ];
    fhir:TestScript.setup.action.assert [ # 0..1 The assertion to perform
      fhir:TestScript.setup.action.assert.label [ string ]; # 0..1 Tracking/logging assertion label
      fhir:TestScript.setup.action.assert.description [ string ]; # 0..1 Tracking/reporting assertion description
      fhir:TestScript.setup.action.assert.direction [ code ]; # 0..1 response | request
      fhir:TestScript.setup.action.assert.compareToSourceId [ string ]; # 0..1 Id of the source fixture to be
  evaluated
      fhir:TestScript.setup.action.assert.compareToSourceExpression [ string ]; # 0..1 The fluentpath expression to
  evaluate against the source fixture
      fhir:TestScript.setup.action.assert.compareToSourcePath [ string ]; # 0..1 XPath or JSONPath expression to
  evaluate against the source fixture
      fhir:TestScript.setup.action.assert.contentType [ code ]; # 0..1 xml | json | ttl | none
      fhir:TestScript.setup.action.assert.expression [ string ]; # 0..1 The fluentpath expression to be evaluated
      fhir:TestScript.setup.action.assert.headerField [ string ]; # 0..1 HTTP header field name
      fhir:TestScript.setup.action.assert.minimumId [ string ]; # 0..1 Fixture Id of minimum content resource
      fhir:TestScript.setup.action.assert.navigationLinks [ boolean ]; # 0..1 Perform validation on navigation links?
      fhir:TestScript.setup.action.assert.operator [ code ]; # 0..1 equals | notEquals | in | notIn | greaterThan |
  lessThan | empty | notEmpty | contains | notContains | eval
      fhir:TestScript.setup.action.assert.path [ string ]; # 0..1 XPath or JSONPath expression
      fhir:TestScript.setup.action.assert.requestMethod [ code ]; # 0..1 delete | get | options | patch | post | put
      fhir:TestScript.setup.action.assert.requestURL [ string ]; # 0..1 Request URL comparison value
      fhir:TestScript.setup.action.assert.resource [ code ]; # 0..1 Resource type
      fhir:TestScript.setup.action.assert.response [ code ]; # 0..1 okay | created | noContent | notModified | bad |
  forbidden | notFound | methodNotAllowed | conflict | gone | preconditionFailed | unprocessable
      fhir:TestScript.setup.action.assert.responseCode [ string ]; # 0..1 HTTP response code to test
      fhir:TestScript.setup.action.assert.rule [ # 0..1 The reference to a TestScript.rule
        fhir:TestScript.setup.action.assert.rule.ruleId [ id ]; # 1..1 Id of the TestScript.rule
        fhir:TestScript.setup.action.assert.rule.param [ # 0..* Rule parameter template
          fhir:TestScript.setup.action.assert.rule.param.name [ string ]; # 1..1 Parameter name matching external
  assert rule parameter
          fhir:TestScript.setup.action.assert.rule.param.value [ string ]; # 1..1 Parameter value defined either
  explicitly or dynamically
        ], ...;
      ];
      fhir:TestScript.setup.action.assert.ruleset [ # 0..1 The reference to a TestScript.ruleset
        fhir:TestScript.setup.action.assert.ruleset.rulesetId [ id ]; # 1..1 Id of the TestScript.ruleset
        fhir:TestScript.setup.action.assert.ruleset.rule [ # 0..* The referenced rule within the ruleset
          fhir:TestScript.setup.action.assert.ruleset.rule.ruleId [ id ]; # 1..1 Id of referenced rule within the
  ruleset
          fhir:TestScript.setup.action.assert.ruleset.rule.param [ # 0..* Rule parameter template
            fhir:TestScript.setup.action.assert.ruleset.rule.param.name [ string ]; # 1..1 Parameter name matching
  external assert ruleset rule parameter
            fhir:TestScript.setup.action.assert.ruleset.rule.param.value [ string ]; # 1..1 Parameter value defined
  either explicitly or dynamically
          ], ...;
        ], ...;
      ];
      fhir:TestScript.setup.action.assert.sourceId [ id ]; # 0..1 Fixture Id of source expression or headerField
      fhir:TestScript.setup.action.assert.validateProfileId [ id ]; # 0..1 Profile Id of validation profile reference
      fhir:TestScript.setup.action.assert.value [ string ]; # 0..1 The value to compare to
      fhir:TestScript.setup.action.assert.warningOnly [ boolean ]; # 0..1 Will this assert produce a warning only on
  error?
    ];
  ], ...;
];
fhir:TestScript.test [ # 0..* A test in this script
  fhir:TestScript.test.name [ string ]; # 0..1 Tracking/logging name of this test
  fhir:TestScript.test.description [ string ]; # 0..1 Tracking/reporting short description of the test
  fhir:TestScript.test.action [ # 1..* A test operation or assert to perform
    fhir:TestScript.test.action.operation [ See TestScript.setup.action.operation ]; # 0..1 The setup operation to
  perform
    fhir:TestScript.test.action.assert [ See TestScript.setup.action.assert ]; # 0..1 The setup assertion to perform
  ], ...;
], ...;
fhir:TestScript.teardown [ # 0..1 A series of required clean up steps
  fhir:TestScript.teardown.action [ # 1..* One or more teardown operations to perform
    fhir:TestScript.teardown.action.operation [ See TestScript.setup.action.operation ]; # 1..1 The teardown operation
   to perform
  ], ...;
];
]
```

Changes since DSTU2

TestScript

TestScript.title | Added Element

TestScript.status | Change value set from http://hl7.org/fhir/ValueSet/conformance-resource-status to http://hl7.org/fhir/ValueSet/publication-status

TestScript.experimental | Now marked as Modifier

TestScript.contact | Type changed from BackboneElement to ContactDetail

TestScript.description | Type changed from string to markdown
TestScript.useContext | Type changed from CodeableConcept to UsageContext Remove Binding http://hl7.org/fhir/ValueSet/use-context (extensible)
TestScript.jurisdiction | Added Element
TestScript.purpose | Added Element
TestScript.copyright | Type changed from string to markdown
TestScript.origin | Added Element
TestScript.origin.index | Added Element
TestScript.origin.profile | Added Element
TestScript.destination | Added Element
TestScript.destination.index | Added Element
TestScript.destination.profile | Added Element
TestScript.metadata.capability.origin | Added Element
TestScript.metadata.capability.capabilities | Added Element
TestScript.fixture.autocreate | Default Value "false" added
TestScript.fixture.autodelete | Default Value "false" added
TestScript.variable.defaultValue | Added Element
TestScript.variable.description | Added Element
TestScript.variable.expression | Added Element
TestScript.variable.hint | Added Element
TestScript.rule | Added Element
TestScript.rule.resource | Added Element
TestScript.rule.param | Added Element
TestScript.rule.param.name | Added Element
TestScript.rule.param.value | Added Element
TestScript.ruleset | Added Element
TestScript.ruleset.resource | Added Element
TestScript.ruleset.rule | Added Element
TestScript.ruleset.rule.ruleId | Added Element
TestScript.ruleset.rule.param | Added Element
TestScript.ruleset.rule.param.name | Added Element
TestScript.ruleset.rule.param.value | Added Element
TestScript.setup.action.operation.destination | Default Value "0" removed
TestScript.setup.action.operation.origin | Added Element
TestScript.setup.action.operation.requestId | Added Element
TestScript.setup.action.assert.compareToSourceExpression | Added Element
TestScript.setup.action.assert.expression | Added Element
TestScript.setup.action.assert.requestMethod | Added Element
TestScript.setup.action.assert.requestURL | Added Element
TestScript.setup.action.assert.rule | Added Element
TestScript.setup.action.assert.rule.ruleId | Added Element
TestScript.setup.action.assert.rule.param | Added Element
TestScript.setup.action.assert.rule.param.name | Added Element
TestScript.setup.action.assert.rule.param.value | Added Element
TestScript.setup.action.assert.ruleset | Added Element
TestScript.setup.action.assert.ruleset.rulesetId | Added Element
TestScript.setup.action.assert.ruleset.rule | Added Element
TestScript.setup.action.assert.ruleset.rule.ruleId | Added Element
TestScript.setup.action.assert.ruleset.rule.param | Added Element
TestScript.setup.action.assert.ruleset.rule.param.name | Added Element
TestScript.setup.action.assert.ruleset.rule.param.value | Added Element
TestScript.teardown.action.operation | Min Cardinality changed from 0 to 1
TestScript.contact.name | deleted
TestScript.contact.telecom | deleted
TestScript.requirements | deleted
TestScript.metadata.capability.conformance | deleted
TestScript.multiserver | deleted
TestScript.setup.metadata | deleted
TestScript.test.metadata | deleted

See the Full Difference for further information

This analysis is available as XML or JSON.

See R2 <--> R3 Conversion Maps (status = 6 tests that all execute ok. 6 fail round-trip testing and 6 r3 resources are invalid (11 errors).).

Alternate definitions: Master Definition (XML, JSON), XML Schema/Schematron (for) + JSON Schema, ShEx (for Turtle)

7.3.4.1 Terminology Bindings

Path | Definition | Type | Reference
TestScript.status | The lifecycle status of a Value Set or Concept Map. | Required | PublicationStatus
TestScript.jurisdiction | Countries and regions within which this artifact is targeted for use | Extensible | Jurisdiction ValueSet
TestScript.origin.profile | The type of origin profile the test system supports. | Extensible | TestScriptProfileOriginType
TestScript.destination.profile | The type of destination profile the test system supports. | Extensible | TestScriptProfileDestinationType

TestScript.setup.action.operation.type | The allowable operation code types. | Extensible | TestScriptOperationCode
TestScript.setup.action.operation.resource TestScript.setup.action.assert.resource | Either a resource or a data type. | Required | Defined Types
TestScript.setup.action.operation.accept TestScript.setup.action.operation.contentType TestScript.setup.action.assert.contentType | The content or mime type. | Required | ContentType
TestScript.setup.action.assert.direction | The type of direction to use for assertion. | Required | AssertionDirectionType
TestScript.setup.action.assert.operator | The type of operator to use for assertion. | Required | AssertionOperatorType
TestScript.setup.action.assert.requestMethod | The allowable request method or HTTP operation codes. | Required | TestScriptRequestMethodCode
TestScript.setup.action.assert.response | The type of response code to use for assertion. | Required | AssertionResponseTypes

7.3.4.2 Constraints

* inv-1: On TestScript.setup.action: Setup action SHALL contain either an operation or assert but not both. (expression on TestScript.setup.action: operation.exists() xor assert.exists())

* inv-10: On TestScript.setup.action.assert: Setup action assert SHALL contain either compareToSourceId and compareToSourceExpression, compareToSourceId and compareToSourcePath or neither. (expression on TestScript.setup.action.assert: compareToSourceId.empty() xor (compareToSourceExpression.exists() or compareToSourcePath.exists()))

* inv-11: On TestScript.test.action.assert: Test action assert SHALL contain either compareToSourceId and compareToSourceExpression, compareToSourceId and compareToSourcePath or neither. (expression on TestScript.test.action.assert: compareToSourceId.empty() xor (compareToSourceExpression.exists() or compareToSourcePath.exists()))

* inv-12: On TestScript.setup.action.assert: Setup action assert response and responseCode SHALL be empty when direction equals request (expression on TestScript.setup.action.assert: (response.empty() and responseCode.empty() and direction = 'request') or direction.empty() or direction = 'response')

* inv-13: On TestScript.test.action.assert: Test action assert response and response and responseCode SHALL be empty when direction equals request (expression on TestScript.test.action.assert: (response.empty() and responseCode.empty() and direction = 'request') or direction.empty() or direction = 'response')

* inv-2: On TestScript.test.action: Test action SHALL contain either an operation or assert but not both. (expression on TestScript.test.action: operation.exists() xor assert.exists())

* inv-3: On TestScript.variable: Variable can only contain one of expression, headerField or path. (expression on TestScript.variable: expression.empty() or headerField.empty() or path.empty())

* inv-4: On TestScript.metadata: TestScript metadata capability SHALL contain required or validated or both. (expression on TestScript.metadata: capability.required.exists() or capability.validated.exists())

* inv-5: On TestScript.setup.action.assert: Only a single assertion SHALL be present within setup action assert element. (expression on TestScript.setup.action.assert: contentType.count() + expression.count() + headerField.count() + minimumId.count() + navigationLinks.count() + path.count() + requestMethod.count() + resource.count() + responseCode.count() + response.count() + rule.count() + ruleset.count() + validateProfileId.count() <=1)

* inv-6: On TestScript.test.action.assert: Only a single assertion SHALL be present within test action assert element. (expression on TestScript.test.action.assert: contentType.count() + expression.count() + headerField.count() + minimumId.count() + navigationLinks.count() + path.count() + requestMethod.count() + resource.count() + responseCode.count() + response.count() + rule.count() + ruleset.count() + validateProfileId.count() <=1)

* inv-7: On TestScript.setup.action.operation: Setup operation SHALL contain either sourceId or targetId or params or url. (expression on TestScript.setup.action.operation: sourceId.exists() or (targetId.count() + url.count() + params.count() = 1) or (type.code in ('capabilities' |'search' | 'transaction' | 'history')))

* inv-8: On TestScript.test.action.operation: Test operation SHALL contain either sourceId or targetId or params or url. (expression on TestScript.test.action.operation: sourceId.exists() or (targetId.count() + url.count() + params.count() = 1) or (type.code in ('capabilities' | 'search' | 'transaction' | 'history')))

* inv-9: On TestScript.teardown.action.operation: Teardown operation SHALL contain either sourceId or targetId or params or url. (expression on TestScript.teardown.action.operation: sourceId.exists() or (targetId.count() + url.count() + params.count() = 1) or (type.code in ('capabilities' | 'search' | 'transaction' | 'history')))

7.3.5 Notes

Please refer to the Testing FHIR - How Tos for examples on working with this resource.

7.3.6 Search Parameters

Search parameters for this resource. The common parameters also apply. See Searching for more information about searching in REST, messaging, and services.

Name | Type | Description | Expression | In Common
date | date | The test script publication date | TestScript.date
description | string | The description of the test script | TestScript.description
identifier | token | External identifier for the test script | TestScript.identifier
jurisdiction | token | Intended jurisdiction for the test script | TestScript.jurisdiction
name | string | Computationally friendly name of the test script | TestScript.name
publisher | string | Name of the publisher of the test script | TestScript.publisher

status | token | The current status of the test script | TestScript.status
testscript-capability | string | TestScript required and validated capability | TestScript.metadata.capability.description
title | string | The human-friendly name of the test script | TestScript.title
url | uri | The uri that identifies the test script | TestScript.url
version | token | The business version of the test script | TestScript.version

7.3.7 Resource TestScript - Examples

URL: https://hl7.org/fhir/STU3/testscript-examples.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.3.7 Resource TestScript - Examples

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative | Compartments: Not linked to any defined compartments

Example List:

General TestScript Example | testscript-example | XML | JSON | Turtle
History TestScript Example | testscript-example-history | XML | JSON | Turtle
Multisystem TestScript Example | testscript-example-multisystem | XML | JSON | Turtle
Read TestScript Example | testscript-example-readtest | XML | JSON | Turtle
Rule TestScript Example | testscript-example-rule | XML | JSON | Turtle
Search TestScript Example | testscript-example-search | XML | JSON | Turtle
Update TestScript Example | testscript-example-update | XML | JSON | Turtle

Usage note: every effort has been made to ensure that the examples are correct and useful, but they are not a normative part of the specification.

7.3.8 Resource TestScript - Detailed Descriptions

URL: https://hl7.org/fhir/STU3/testscript-definitions.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.3.8 Resource TestScript - Detailed Descriptions

FHIR Infrastructure Work Group | Maturity Level: 2 | Trial Use | Compartments: Not linked to any defined compartments

Detailed Descriptions for the elements in the TestScript resource.

TestScript
Definition | A structured set of tests against a FHIR server implementation to determine compliance against the FHIR specification.
Control | 1..1
TestScript.url
Definition | An absolute URI that is used to identify this test script when it is referenced in a specification, model, design or an instance. This SHALL be a URL, SHOULD be globally unique, and SHOULD be an address at which this test script is (or will be) published. The URL SHOULD include the major version of the test script. For more information see Technical and Business Versions.
Control | 1..1
Type | uri
Requirements | Allows the test script to be referenced by a single globally unique identifier.
Alternate Names | url; authoritative-url; destination; identity
Summary | true
Comments | Can be a urn:uuid: or a urn:oid:, but real http: addresses are preferred. Multiple instances may share the same url if they have a distinct version.
TestScript.identifier
Definition | A formal identifier that is used to identify this test script when it is represented in other formats, or referenced in a specification, model, design or an instance.
Note | This is a business identifer, not a resource identifier (see discussion)
Control | 0..1
Type | Identifier
Requirements | Allows externally provided and/or usable business identifiers to be easily associated with the module.
Summary | true
Comments | Typically, this is used for identifiers that can go in an HL7 V3 II (instance identifier) data type, e.g., to identify this test script outside of FHIR, where it is not possible to use the logical URI.
TestScript.version
Definition | The identifier that is used to identify this version of the test script when it is referenced in a specification, model, design or instance. This is an arbitrary value managed by the test script author and is not expected to be globally unique. For example, it might be a timestamp (e.g. yyyymmdd) if a managed version is not available. There is also no expectation that versions can be placed in a lexicographical sequence.
Note | This is a business versionId, not a resource version id (see discussion)
Control | 0..1
Type | string
Summary | true
Comments | There may be different test script instances that have the same identifier but different versions. The version can be appended to the url in a reference to allow a refrence to a particular business version of the test script with the format [url]|[version].
TestScript.name

Definition | A natural language name identifying the test script. This name should be usable as an identifier for the module by machine processing applications such as code generation.
Control | 1..1
Type | string
Requirements | Support human navigation and code generation.
Summary | true
Comments | The name is not expected to be globally unique. The name should be a simple alpha-numeric type name to ensure that it is computable friendly.
TestScript.title
Definition | A short, descriptive, user-friendly title for the test script.
Control | 0..1
Type | string
Summary | true
Comments | This name does not need to be machine-processing friendly and may contain punctuation, white-space, etc.
TestScript.status
Definition | The status of this test script. Enables tracking the life-cycle of the content.
Control | 1..1
Terminology Binding | PublicationStatus (Required)
Type | code
Is Modifier | true
Summary | true
Comments | Allows filtering of test scripts that are appropriate for use vs. not. This is labeled as "Is Modifier" because applications should not use a retired test script without due consideration.
TestScript.experimental
Definition | A boolean value to indicate that this test script is authored for testing purposes (or education/evaluation/marketing), and is not intended to be used for genuine usage.
Control | 0..1
Type | boolean
Is Modifier | true
Requirements | Enables experimental content to be developed following the same lifecycle that would be used for a production-level test script.
Summary | true
Comments | Allows filtering of test script that are appropriate for use vs. not. This is labeled as "Is Modifier" because applications should not use an experimental test script in production.
TestScript.date
Definition | The date (and optionally time) when the test script was published. The date must change if and when the business version changes and it must change if the status code changes. In addition, it should change when the substantive content of the test script changes.
Control | 0..1
Type | dateTime
Alternate Names | Revision Date
Summary | true
Comments | Note that this is not the same as the resource last-modified-date, since the resource may be a secondary representation of the test script. Additional specific dates may be added as extensions or be found by consulting Provenances associated with past versions of the resource.
TestScript.publisher
Definition | The name of the individual or organization that published the test script.
Control | 0..1
Type | string
Requirements | Helps establish the "authority/credibility" of the test script. May also allow for contact.
Summary | true
Comments | Usually an organization, but may be an individual. The publisher (or steward) of the test script is the organization or individual primarily responsible for the maintenance and upkeep of the test script. This is not necessarily the same individual or organization that developed and initially authored the content. The publisher is the primary point of contact for questions or issues with the test script. This item SHOULD be populated unless the information is available from context.
TestScript.contact
Definition | Contact details to assist a user in finding and communicating with the publisher.
Control | 0..*
Type | ContactDetail
Summary | true
Comments | May be a web site, an email address, a telephone number, etc.
TestScript.description
Definition | A free text natural language description of the test script from a consumer's perspective.
Control | 0..1
Type | markdown
Comments | This description can be used to capture details such as why the test script was built, comments about misuse, instructions for clinical use and interpretation, literature references, examples from the paper world, etc. It is not a rendering of the test script as conveyed in the 'text' field of the resource itself. This item SHOULD be populated unless the information is available from context (e.g. the language of the profile is presumed to be the predominant language in the place the profile was created).
TestScript.useContext
Definition | The content was developed with a focus and intent of supporting the contexts that are listed. These terms may be used to assist with indexing and searching for appropriate test script instances.
Control | 0..*
Type | UsageContext
Requirements | Assist in searching for appropriate content.
Summary | true

Comments | When multiple useContexts are specified, there is no expectation whether all or any of the contexts apply.
TestScript.jurisdiction
Definition | A legal or geographic region in which the test script is intended to be used.
Control | 0..*
Terminology Binding | Jurisdiction ValueSet (Extensible)
Type | CodeableConcept
Summary | true
Comments | It may be possible for the test script to be used in jurisdictions other than those for which it was originally designed or intended.
TestScript.purpose
Definition | Explaination of why this test script is needed and why it has been designed as it has.
Control | 0..1
Type | markdown
Comments | This element does not describe the usage of the test script Instead it provides traceability of "why" the resource is either needed or "why" it is defined as it is. This may be used to point to source materials or specifications that drove the structure of this test script.
TestScript.copyright
Definition | A copyright statement relating to the test script and/or its contents. Copyright statements are generally legal restrictions on the use and publishing of the test script.
Control | 0..1
Type | markdown
Requirements | Consumers must be able to determine any legal restrictions on the use of the test script and/or its content.
Alternate Names | License; Restrictions
TestScript.origin
Definition | An abstract server used in operations within this test script in the origin element.
Control | 0..*
Comments | The purpose of this element is to define the profile of an origin element used elsewhere in the script. Test engines could then use the origin-profile mapping to offer a filtered list of test systems that can serve as the sender for the interaction.
TestScript.origin.index
Definition | Abstract name given to an origin server in this test script. The name is provided as a number starting at 1.
Control | 1..1
Type | integer
Comments | A given origin index (e.g. 1) can appear only once in the list (e.g. Origin 1 cannot be specified twice ... once as FormFiller and and again as FormProcessor within the same script as that could get confusing during test configuration). Different origin indices could play the same actor in the same test script (e.g. You could have two different test systems acting as Form-Filler). The origin indices provided elsewhere in the test script must be one of these origin indices.
TestScript.origin.profile
Definition | The type of origin profile the test system supports.
Control | 1..1
Terminology Binding | TestScriptProfileOriginType (Extensible)
Type | Coding
Meaning if Missing | FHIR-Client
Comments | Must be a "sender"/"client" profile.
TestScript.destination
Definition | An abstract server used in operations within this test script in the destination element.
Control | 0..*
Comments | The purpose of this element is to define the profile of a destination element used elsewhere in the script. Test engines could then use the destination-profile mapping to offer a filtered list of test systems that can serve as the receiver for the interaction.
TestScript.destination.index
Definition | Abstract name given to a destination server in this test script. The name is provided as a number starting at 1.
Control | 1..1
Type | integer
Comments | A given destination index (e.g. 1) can appear only once in the list (e.g. Destination 1 cannot be specified twice ... once as Form-Manager and again as Form-Processor within the same script as that could get confusing during test configuration). Different destination indices could play the same actor in the same test script (e.g. You could have two different test systems acting as Form-Manager). The destination indices provided elsewhere in the test script must be one of these destination indices.
TestScript.destination.profile
Definition | The type of destination profile the test system supports.
Control | 1..1
Terminology Binding | TestScriptProfileDestinationType (Extensible)
Type | Coding
Meaning if Missing | FHIR-Server
Comments | Must be a "receiver"/"server" profile.
TestScript.metadata
Definition | The required capability must exist and are assumed to function correctly on the FHIR server being tested.
Control | 0..1
Invariants | Defined on this element inv-4: TestScript metadata capability SHALL contain required or validated or both. (expression : capability.required.exists() or capability.validated.exists(), xpath: f:capability/f:required or f:capability/f:validated or (f:capability/f:required and f:capability/f:validated))
TestScript.metadata.link
Definition | A link to the FHIR specification that this test is covering.
Control | 0..*
TestScript.metadata.link.url
Definition | URL to a particular requirement or feature within the FHIR specification.
Control | 1..1

Type | uri
TestScript.metadata.link.description
Definition | Short description of the link.
Control | 0..1
Type | string
TestScript.metadata.capability
Definition | Capabilities that must exist and are assumed to function correctly on the FHIR server being tested.
Control | 1..*
Comments | When the metadata capabilities section is defined at TestScript.metadata or at TestScript.setup.metadata, and the server's conformance statement does not contain the elements defined in the minimal conformance statement, then all the tests in the TestScript are skipped. When the metadata capabilities section is defined at TestScript.test.metadata and the server's conformance statement does not contain the elements defined in the minimal conformance statement, then only that test is skipped. The "metadata.capabilities.required" and "metadata.capabilities.validated" elements only indicate whether the capabilities are the primary focus of the test script or not. They do not impact the skipping logic. Capabilities whose "metadata.capabilities.validated" flag is true are the primary focus of the test script.
TestScript.metadata.capability.required
Definition | Whether or not the test execution will require the given capabilities of the server in order for this test script to execute.
Control | 0..1
Type | boolean
Default Value | false
TestScript.metadata.capability.validated
Definition | Whether or not the test execution will validate the given capabilities of the server in order for this test script to execute.
Control | 0..1
Type | boolean
Default Value | false
TestScript.metadata.capability.description
Definition | Description of the capabilities that this test script is requiring the server to support.
Control | 0..1
Type | string
TestScript.metadata.capability.origin
Definition | Which origin server these requirements apply to.
Control | 0..*
Type | integer
TestScript.metadata.capability.destination
Definition | Which server these requirements apply to.
Control | 0..1
Type | integer
TestScript.metadata.capability.link
Definition | Links to the FHIR specification that describes this interaction and the resources involved in more detail.
Control | 0..*
Type | uri
TestScript.metadata.capability.capabilities
Definition | Minimum capabilities required of server for test script to execute successfully. If server does not meet at a minimum the referenced capability statement, then all tests in this script are skipped.
Control | 1..1
Type | Reference(CapabilityStatement)
Comments | The conformance statement of the server has to contain at a minimum the contents of the reference pointed to by this element.
TestScript.fixture
Definition | Fixture in the test script - by reference (uri). All fixtures are required for the test script to execute.
Control | 0..*
TestScript.fixture.autocreate
Definition | Whether or not to implicitly create the fixture during setup. If true, the fixture is automatically created on each server being tested during setup, therefore no create operation is required for this fixture in the TestScript.setup section.
Control | 0..1
Type | boolean
Default Value | false
TestScript.fixture.autodelete
Definition | Whether or not to implicitly delete the fixture during teardown. If true, the fixture is automatically deleted on each server being tested during teardown, therefore no delete operation is required for this fixture in the TestScript.teardown section.
Control | 0..1
Type | boolean
Default Value | false
TestScript.fixture.resource
Definition | Reference to the resource (containing the contents of the resource needed for operations).
Control | 0..1
Type | Reference(Any)
Comments | See http://hl7.org/fhir/STU3/resourcelist.html for complete list of resource types.
TestScript.profile
Definition | Reference to the profile to be used for validation.
Control | 0..*
Type | Reference(Any)
Comments | See http://hl7.org/fhir/STU3/resourcelist.html for complete list of resource types.
TestScript.variable
Definition | Variable is set based either on element value in response body or on header field value in the response headers.

Control | 0..*
Comments | Variables would be set based either on XPath/JSONPath expressions against fixtures (static and response), or headerField evaluations against response headers. If variable evaluates to nodelist or anything other than a primitive value, then test engine would report an error. Variables would be used to perform clean replacements in "operation.params", "operation.requestHeader.value", and "operation.url" element values during operation calls and in "assert.value" during assertion evaluations. This limits the places that test engines would need to look for placeholders "${}". Variables are scoped to the whole script. They are NOT evaluated at declaration. They are evaluated by test engine when used for substitutions in "operation.params", "operation.requestHeader.value", and "operation.url" element values during operation calls and in "assert.value" during assertion evaluations. See example testscript-search.xml.
Invariants | Defined on this element inv-3: Variable can only contain one of expression, headerField or path. (expression : expression.empty() or headerField.empty() or path.empty(), xpath: not(f:expression and f:headerField and f:path))
TestScript.variable.name
Definition | Descriptive name for this variable.
Control | 1..1
Type | string
Comments | Placeholders would contain the variable name wrapped in ${} in "operation.params", "operation.requestHeader.value", and "operation.url" elements. These placeholders would need to be replaced by the variable value before the operation is executed.
TestScript.variable.defaultValue
Definition | A default, hard-coded, or user-defined value for this variable.
Control | 0..1
Type | string
Comments | The purpose of this element is to allow for a pre-defined value that can be used as a default or as an override value. Test engines can optionally use this as a placeholder for user-defined execution time values.
TestScript.variable.description
Definition | A free text natural language description of the variable and its purpose.
Control | 0..1
Type | string
TestScript.variable.expression
Definition | The fluentpath expression to evaluate against the fixture body. When variables are defined, only one of either expression, headerField or path must be specified.
Control | 0..1
Type | string
Comments | If headerField is defined, then the variable will be evaluated against the headers that sourceId is pointing to. If expression or path is defined, then the variable will be evaluated against the fixture body that sourceId is pointing to. It is an error to define any combination of expression, headerField and path.
TestScript.variable.headerField
Definition | Will be used to grab the HTTP header field value from the headers that sourceId is pointing to.
Control | 0..1
Type | string
Comments | If headerField is defined, then the variable will be evaluated against the headers that sourceId is pointing to. If path is defined, then the variable will be evaluated against the fixture body that sourceId is pointing to. It is an error to define both headerField and path.
TestScript.variable.hint
Definition | Displayable text string with hint help information to the user when entering a default value.
Control | 0..1
Type | string
TestScript.variable.path
Definition | XPath or JSONPath to evaluate against the fixture body. When variables are defined, only one of either expression, headerField or path must be specified.
Control | 0..1
Type | string
Comments | If headerField is defined, then the variable will be evaluated against the headers that sourceId is pointing to. If expression or path is defined, then the variable will be evaluated against the fixture body that sourceId is pointing to. It is an error to define any combination of expression, headerField and path.
TestScript.variable.sourceId
Definition | Fixture to evaluate the XPath/JSONPath expression or the headerField against within this variable.
Control | 0..1
Type | id
Comments | This can be a statically defined fixture (at the top of the TestScript) or a dynamically set fixture created by responseId of the action.operation element.
TestScript.rule
Definition | Assert rule to be used in one or more asserts within the test script.
Control | 0..*
Comments | Each rule should be treated by test engines as one assertion regardless of how many assertions are contained within the external rule template. The impact of negative rule evaluation on test script execution is the same as an assertion failure which is described elsewhere in the TestScript resource.
TestScript.rule.resource
Definition | Reference to the resource (containing the contents of the rule needed for assertions).
Control | 1..1
Type | Reference(Any)
TestScript.rule.param
Definition | Each rule template can take one or more parameters for rule evaluation.
Control | 0..*
Comments | The parameter value can be dynamic at runtime.
TestScript.rule.param.name
Definition | Descriptive name for this parameter that matches the external assert rule parameter name.

Control | 1..1
Type | string
Comments | The external rule template would be looking for the parameter by this name.
TestScript.rule.param.value
Definition | The explicit or dynamic value for the parameter that will be passed on to the external rule template.
Control | 0..1
Type | string
Comments | This value can be overwritten by the assert.rule.param.value i.e. TestScript.rule.param.value will be used if assert.rule.param.value is not specified. The param value can be a string-representation of a number, string, or boolean that is expected. Test engines do have to look for placeholders (${}) and replace the variable placeholders with the variable values at runtime before supplying this value to the external rule template.
TestScript.ruleset
Definition | Contains one or more rules. Offers a way to group rules so assertions could reference the group of rules and have them all applied.
Control | 0..*
Comments | Each rule within a ruleset should be treated by test engines as one assertion regardless of how many assertions are contained within the external rule template. The impact of negative rule evaluation on test script execution is the same as an assertion failure which is described elsewhere in the TestScript resource.
TestScript.ruleset.resource
Definition | Reference to the resource (containing the contents of the ruleset needed for assertions).
Control | 1..1
Type | Reference(Any)
TestScript.ruleset.rule
Definition | The referenced rule within the external ruleset template.
Control | 1..*
Comments | This qualifies each param name so that a parameter with the same name can be used differently by the different rules with the ruleset.
TestScript.ruleset.rule.ruleId
Definition | Id of the referenced rule within the external ruleset template.
Control | 1..1
Type | id
TestScript.ruleset.rule.param
Definition | Each rule template can take one or more parameters for rule evaluation.
Control | 0..*
Comments | The parameter value can be dynamic at runtime.
TestScript.ruleset.rule.param.name
Definition | Descriptive name for this parameter that matches the external assert ruleset rule parameter name.
Control | 1..1
Type | string
Comments | The external rule template would be looking for the parameter by this name.
TestScript.ruleset.rule.param.value
Definition | The value for the parameter that will be passed on to the external ruleset rule template.
Control | 0..1
Type | string
Comments | This value can be overwritten by the assert.ruleset.rule.param.value i.e. TestScript.ruleset.rule.param.value will be used if assert.ruleset.rule.param.value is not specified. The param value can be a string-representation of a number, string, or boolean that is expected. Test engines do have to look for placeholders (${}) and replace the variable placeholders with the variable values at runtime before supplying this value to the external rule template.
TestScript.setup
Definition | A series of required setup operations before tests are executed.
Control | 0..1
TestScript.setup.action
Definition | Action would contain either an operation or an assertion.
Control | 1..*
Comments | An action should contain either an operation or an assertion but not both. It can contain any number of variables.
Invariants | Defined on this element inv-1: Setup action SHALL contain either an operation or assert but not both. (expression : operation.exists() xor assert.exists(), xpath: (f:operation or f:assert) and not(f:operation and f:assert))
TestScript.setup.action.operation
Definition | The operation to perform.
Control | 0..1
Invariants | Defined on this element inv-7: Setup operation SHALL contain either sourceId or targetId or params or url. (expression : sourceId.exists() or (targetId.count() + url.count() + params.count() = 1) or (type.code in ('capabilities' |'search' | 'transaction' | 'history')), xpath: f:sourceId or ((f:targetId or f:url or f:params) and (count(f:targetId) + count(f:url) + count(f:params) =1)) or (f:type/f:code/@value='capabilities' or f:type/f:code/@value='search' or f:type/f:code/@value='transaction' or f:type/f:code/@value='history'))
TestScript.setup.action.operation.type
Definition | Server interaction or operation type.
Control | 0..1
Terminology Binding | TestScriptOperationCode (Extensible)
Type | Coding
Comments | See http://hl7.org/fhir/STU3/http.html for list of server interactions.
TestScript.setup.action.operation.resource
Definition | The type of the resource. See http://hl7.org/fhir/STU3/resourcelist.html.
Control | 0..1
Terminology Binding | Any defined Resource or Data Type name
Type | code

Comments | If "url" element is specified, then "targetId", "params", and "resource" elements will be ignored as "url" element will have everything needed for constructing the request url. If "params" element is specified, then "targetId" element is ignored. For FHIR operations that require a resource (e.g. "read" and "vread" operations), the "resource" element must be specified when "params" element is specified. If "url" and "params" elements are absent, then the request url will be constructed from "targetId" fixture if present. For "read" operation, the resource and id values will be extracted from "targetId" fixture and used to construct the url. For "vread" and "history" operations, the versionId value will also be used.
TestScript.setup.action.operation.label
Definition | The label would be used for tracking/logging purposes by test engines.
Control | 0..1
Type | string
Comments | This has no impact on the verification itself.
TestScript.setup.action.operation.description
Definition | The description would be used by test engines for tracking and reporting purposes.
Control | 0..1
Type | string
Comments | This has no impact on the verification itself.
TestScript.setup.action.operation.accept
Definition | The content-type or mime-type to use for RESTful operation in the 'Accept' header.
Control | 0..1
Terminology Binding | ContentType (Required)
Type | code
Meaning if Missing | xml
Comments | If this is specified, then test engine shall set the 'Accept' header to the corresponding value. If 'xml' is specified, then 'Accept' header of 'application/fhir+xml' will be set. If 'json' is specified, then 'application/fhir+json' will be used. If you'd like to explicitly set the 'Accept' to some other value then use the 'requestHeader' element.
TestScript.setup.action.operation.contentType
Definition | The content-type or mime-type to use for RESTful operation in the 'Content-Type' header.
Control | 0..1
Terminology Binding | ContentType (Required)
Type | code
Meaning if Missing | xml
Comments | If this is specified, then test engine shall set the 'Content-Type' header to the corresponding value. If 'xml' is specified, then 'Content-Type' header of 'application/fhir+xml' will be set. If 'json' is specified, then 'application/fhir+json' will be used. If you'd like to explicitly set the 'Content-Type' to some other value then use the 'requestHeader' element.
TestScript.setup.action.operation.destination
Definition | The server where the request message is destined for. Must be one of the server numbers listed in TestScript.destination section.
Control | 0..1
Type | integer
Comments | If multiple TestScript.destination elements are defined and operation.destination is undefined, test engine will report an error as it cannot determine what destination to use for the exchange.
TestScript.setup.action.operation.encodeRequestUrl
Definition | Whether or not to implicitly send the request url in encoded format. The default is true to match the standard RESTful client behavior. Set to false when communicating with a server that does not support encoded url paths.
Control | 0..1
Type | boolean
Default Value | true
TestScript.setup.action.operation.origin
Definition | The server where the request message originates from. Must be one of the server numbers listed in TestScript.origin section.
Control | 0..1
Type | integer
Comments | If absent, test engine will send the message. When present, test engine will not send the request message but will wait for the request message to be sent from this origin server.
TestScript.setup.action.operation.params
Definition | Path plus parameters after [type]. Used to set parts of the request URL explicitly.
Control | 0..1
Type | string
Comments | If "url" element is specified, then "targetId", "params", and "resource" elements will be ignored as "url" element will have everything needed for constructing the request url. If "params" element is specified, then "targetId" element is ignored. For FHIR operations that require a resource (e.g. "read" and "vread" operations), the "resource" element must be specified when "params" element is specified. If "url" and "params" elements are absent, then the request url will be constructed from "targetId" fixture if present. For "read" operation, the resource and id values will be extracted from "targetId" fixture and used to construct the url. For "vread" and "history" operations, the versionId value will also be used. Test engines would append whatever is specified for "params" to the URL after the resource type without tampering with the string (beyond encoding the URL for HTTP). The "params" element does not correspond exactly to "search parameters". Nor is it the "path". It corresponds to the part of the URL that comes after the [type] (when "resource" element is specified); e.g. It corresponds to "/[id]/history/[vid] {?format=[mime-type]}" in the following operation: GET [base]/[type]/[id]/history/[vid] {?format=[mime-type]} Test engines do have to look for placeholders (${}) and replace the variable placeholders with the variable values at runtime before sending the request.
TestScript.setup.action.operation.requestHeader
Definition | Header elements would be used to set HTTP headers.
Control | 0..*
Comments | This gives control to test-script writers to set headers explicitly based on test requirements. It will allow for testing using: - "If-Modified-Since" and "If-None-Match" headers. See http://hl7.org/fhir/STU3/http.html#2.1.0.5.1 - "If-Match" header. See http://hl7.org/fhir/STU3/http.html#2.1.0.11 - Conditional Create using "If-None-Exist". See http://hl7.org/fhir/STU3/http.html#2.1.0.13.1 - Invalid "Content-Type" header for negative testing. - etc.
TestScript.setup.action.operation.requestHeader.field
Definition | The HTTP header field e.g. "Accept".

Control | 1..1
Type | string
Comments | If header element is specified, then field is required.
TestScript.setup.action.operation.requestHeader.value
Definition | The value of the header e.g. "application/fhir+xml".
Control | 1..1
Type | string
Comments | If header element is specified, then value is required. No conversions will be done by the test engine e.g. "xml" to "application/fhir+xml". The values will be set in HTTP headers "as-is". Test engines do have to look for placeholders (${}) and replace the variable placeholders with the variable values at runtime before sending the request.
TestScript.setup.action.operation.requestId
Definition | The fixture id (maybe new) to map to the request.
Control | 0..1
Type | id
Comments | If a requestId is supplied, then the resulting request (both headers and body) is mapped to the fixture ID (which may be entirely new and previously undeclared) designated by "requestId". If requestId is not specified, it is the test engine's responsibility to store the request and use it as the requestId in subsequent assertions when assertion path and/or headerField is specified, direction is equal to request, and the requestId in not specified.
TestScript.setup.action.operation.responseId
Definition | The fixture id (maybe new) to map to the response.
Control | 0..1
Type | id
Comments | If a responseId is supplied, and the server responds, then the resulting response (both headers and body) is mapped to the fixture ID (which may be entirely new and previously undeclared) designated by "responseId". If responseId is not specified, it is the test engine's responsibility to store the response and use it as the responseId in subsequent assertions when assertion path and/or headerField is specified and the responseId is not specified.
TestScript.setup.action.operation.sourceId
Definition | The id of the fixture used as the body of a PUT or POST request.
Control | 0..1
Type | id
TestScript.setup.action.operation.targetId
Definition | Id of fixture used for extracting the [id], [type], and [vid] for GET requests.
Control | 0..1
Type | id
Comments | If "url" element is specified, then "targetId", "params", and "resource" elements will be ignored as "url" element will have everything needed for constructing the request url. If "params" element is specified, then "targetId" element is ignored. For FHIR operations that require a resource (e.g. "read" and "vread" operations), the "resource" element must be specified when "params" element is specified. If "url" and "params" elements are absent, then the request url will be constructed from "targetId" fixture if present. For "read" operation, the resource and id values will be extracted from "targetId" fixture and used to construct the url. For "vread" and "history" operations, the versionId value will also be used.
TestScript.setup.action.operation.url
Definition | Complete request URL.
Control | 0..1
Type | string
Comments | Used to set the request URL explicitly. If "url" element is defined, then "targetId", "resource", and "params" elements will be ignored. Test engines would use whatever is specified in "url" without tampering with the string (beyond encoding the URL for HTTP). Test engines do have to look for placeholders (${}) and replace the variable placeholders with the variable values at runtime before sending the request.
TestScript.setup.action.assert
Definition | Evaluates the results of previous operations to determine if the server under test behaves appropriately.
Control | 0..1
Comments | In order to evaluate an assertion, the request, response, and results of the most recently executed operation must always be maintained by the test engine.
Invariants | Defined on this element inv-10: Setup action assert SHALL contain either compareToSourceId and compareToSourceExpression, compareToSourceId and compareToSourcePath or neither. (expression : compareToSourceId.empty() xor (compareToSourceExpression.exists() or compareToSourcePath.exists()), xpath: (f:compareToSourceId and f:compareToSourceExpression) or (f:compareToSourceId and f:compareToSourcePath) or not(f:compareToSourceId or f:compareToSourceExpression or f:compareToSourcePath)) inv-12: Setup action assert response and responseCode SHALL be empty when direction equals request (expression : (response.empty() and responseCode.empty() and direction = 'request') or direction.empty() or direction = 'response', xpath: ((count(f:response) + count(f:responseCode)) = 0 and (f:direction/@value='request')) or (count(f:direction) = 0) or (f:direction/@value='response')) inv-5: Only a single assertion SHALL be present within setup action assert element. (expression : contentType.count() + expression.count() + headerField.count() + minimumId.count() + navigationLinks.count() + path.count() + requestMethod.count() + resource.count() + responseCode.count() + response.count() + rule.count() + ruleset.count() + validateProfileId.count() <=1, xpath: count(f:contentType) + count(f:expression) + count(f:headerField) + count(f:minimumId) + count(f:navigationLinks) + count(f:path) + count(f:requestMethod) + count(f:resource) + count(f:responseCode) + count(f:response) + count(f:rule) + count(f:ruleset) + count(f:validateProfileId) <=1)
TestScript.setup.action.assert.label
Definition | The label would be used for tracking/logging purposes by test engines.
Control | 0..1
Type | string
Comments | This has no impact on the verification itself.
TestScript.setup.action.assert.description
Definition | The description would be used by test engines for tracking and reporting purposes.
Control | 0..1
Type | string
Comments | This has no impact on the verification itself.
TestScript.setup.action.assert.direction
Definition | The direction to use for the assertion.

Control | 0..1
Terminology Binding | AssertionDirectionType (Required)
Type | code
Comments | If the direction is specified as "response" (the default), then the processing of this assert is against the received response message. If the direction is specified as "request", then the processing of this assert is against the sent request message.
TestScript.setup.action.assert.compareToSourceId
Definition | Id of the source fixture used as the contents to be evaluated by either the "source/expression" or "sourceId/path" definition.
Control | 0..1
Type | string
TestScript.setup.action.assert.compareToSourceExpression
Definition | The fluentpath expression to evaluate against the source fixture. When compareToSourceId is defined, either compareToSourceExpression or compareToSourcePath must be defined, but not both.
Control | 0..1
Type | string
Comments | Thefhirpath expression to be evaluated against the expected fixture to compare to. Ignored if "assert.value" is used. The evaluation will be done before the assertion is evaluated.
TestScript.setup.action.assert.compareToSourcePath
Definition | XPath or JSONPath expression to evaluate against the source fixture. When compareToSourceId is defined, either compareToSourceExpression or compareToSourcePath must be defined, but not both.
Control | 0..1
Type | string
Comments | The XPath or JSONPath expression to be evaluated against the expected fixture to compare to. Ignored if "assert.value" is used. The evaluation will be done before the assertion is evaluated.
TestScript.setup.action.assert.contentType
Definition | The content-type or mime-type to use for RESTful operation in the 'Content-Type' header.
Control | 0..1
Terminology Binding | ContentType (Required)
Type | code
Meaning if Missing | xml
Comments | If this is specified, then test engine shall confirm that the content-type of the last operation's headers is set to this value. If "assert.sourceId" element is specified, then the evaluation will be done against the headers mapped to that sourceId (and not the last operation's headers). If 'xml' is specified, then 'Content-Type' header of 'application/fhir+xml' will be confirmed. If 'json' is specified, then 'application/fhir+json' will be used. If you'd like to have more control over the string, then use 'assert.headerField' instead.
TestScript.setup.action.assert.expression
Definition | The fluentpath expression to be evaluated against the request or response message contents - HTTP headers and payload.
Control | 0..1
Type | string
Comments | If both "expression" and a "fixtureId" are specified, then the expression will be evaluated against the request or response body mapped to the fixtureId. If "expression" is specified and a "fixtureId" is not, then the expression will be evaluated against the response body of the last operation. Test engines are to store the request and response body and headers of the last operation at all times for subsequent assertions.
TestScript.setup.action.assert.headerField
Definition | The HTTP header field name e.g. 'Location'.
Control | 0..1
Type | string
Comments | If "headerField" is specified then "value" must be specified. If "sourceId" is not specified, then "headerField" will be evaluated against the last operation's response headers. Test engines are to keep track of the last operation's response body and response headers.
TestScript.setup.action.assert.minimumId
Definition | The ID of a fixture. Asserts that the response contains at a minimum the fixture specified by minimumId.
Control | 0..1
Type | string
Comments | Asserts that the response contains all the element/content in another fixture pointed to by minimumId. This can be a statically defined fixture or one that is dynamically set via responseId.
TestScript.setup.action.assert.navigationLinks
Definition | Whether or not the test execution performs validation on the bundle navigation links.
Control | 0..1
Type | boolean
Comments | Asserts that the Bundle contains first, last, and next links.
TestScript.setup.action.assert.operator
Definition | The operator type defines the conditional behavior of the assert. If not defined, the default is equals.
Control | 0..1
Terminology Binding | AssertionOperatorType (Required)
Type | code
Comments | Operators are useful especially for negative testing. If operator is not specified, then the "equals" operator is assumed; e.g. <code> <assert> <operator value="in" /> <responseCode value="200,201,204" /> </assert> <assert> <operator value="notEquals" /> <response value="okay"/> </assert> <assert> <operator value="greaterThan" /> <responseHeader> <field value="Content-Length" /> <value value="0" /> <responseHeader/> </assert> </code>.
TestScript.setup.action.assert.path
Definition | The XPath or JSONPath expression to be evaluated against the fixture representing the response received from server.
Control | 0..1
Type | string
Comments | If both "path" and a "fixtureId" are specified, then the path will be evaluated against the request or response body mapped to the fixtureId. If "path" is specified and a "fixtureId" is not, then the path will be evaluated against the response body of the last operation. Test engines are to store the request and response body and headers of the last operation at all times for subsequent assertions.
TestScript.setup.action.assert.requestMethod

Definition | The request method or HTTP operation code to compare against that used by the client system under test.
Control | 0..1
Terminology Binding | TestScriptRequestMethodCode (Required)
Type | code
Comments | If "requestMethod" is specified then it will be used in place of "value". The "requestMethod" will evaluate against the last operation's request HTTP operation.
TestScript.setup.action.assert.requestURL
Definition | The value to use in a comparison against the request URL path string.
Control | 0..1
Type | string
Comments | If "requestURL" is specified then it will be used in place of "value". The "requestURL" will evaluate against the last operation's full request URL path string.
TestScript.setup.action.assert.resource
Definition | The type of the resource. See http://hl7.org/fhir/STU3/resourcelist.html.
Control | 0..1
Terminology Binding | Any defined Resource or Data Type name
Type | code
Comments | This will be expected resource type in response body e.g. in read, vread, search, etc. See http://hl7.org/fhir/STU3/resourcelist.html for complete list of resource types; e.g. <assert > <resourceType value="Patient" </assert>.
TestScript.setup.action.assert.response
Definition | okay | created | noContent | notModified | bad | forbidden | notFound | methodNotAllowed | conflict | gone | preconditionFailed | unprocessable.
Control | 0..1
Terminology Binding | AssertionResponseTypes (Required)
Type | code
Comments | This is a shorter way of achieving similar verifications via "assert.responseCode". If you need more control, then use "assert.responseCode" e.g. <assert> <contentType value="json" /> <response value="okay"/> </assert>.
TestScript.setup.action.assert.responseCode
Definition | The value of the HTTP response code to be tested.
Control | 0..1
Type | string
Comments | To be used with "operator" attribute value. Asserts that the response code equals this value if "operator" is not specified. If the operator is "in" or "notIn" then the responseCode would be a comma-separated list of values e.g. "200,201". Otherwise, it's expected to be a numeric value. If "fixture" is not specified, then the "responseBodyId" value of the last operation is assumed.
TestScript.setup.action.assert.rule
Definition | The TestScript.rule this assert will evaluate.
Control | 0..1
Comments | Each rule should get evaluated by test engines as one assertion regardless of how many assertions are contained within the external rule template. The impact of negative rule evaluation on test script execution is the same as an assertion failure which is descibed elsewhere in the TestScript resource.
TestScript.setup.action.assert.rule.ruleId
Definition | The TestScript.rule id value this assert will evaluate.
Control | 1..1
Type | id
TestScript.setup.action.assert.rule.param
Definition | Each rule template can take one or more parameters for rule evaluation.
Control | 0..*
Comments | The parameter value can be dynamic at runtime.
TestScript.setup.action.assert.rule.param.name
Definition | Descriptive name for this parameter that matches the external assert rule parameter name.
Control | 1..1
Type | string
Comments | The external rule template would be looking for the parameter by this name.
TestScript.setup.action.assert.rule.param.value
Definition | The value for the parameter that will be passed on to the external rule template.
Control | 1..1
Type | string
Comments | This value overwrites the value (if any) specified in TestScript.rule.param.value. The param value can be a string-representation of a number, string, or boolean that is expected. Test engines do have to look for placeholders (${}) and replace the variable placeholders with the variable values at runtime before supplying this value to the external rule template.
TestScript.setup.action.assert.ruleset
Definition | The TestScript.ruleset this assert will evaluate.
Control | 0..1
Comments | Each rule within a ruleset should get evaluated by test engines as a separate assertion. The impact of negative rule evaluation on test script execution is the same as an assertion failure which is descibed elsewhere in the TestScript resource. If the first rule within the ruleset results in a failed assertion, then test engines do not have to evaluate the rest of the rules within the ruleset.
TestScript.setup.action.assert.ruleset.rulesetId
Definition | The TestScript.ruleset id value this assert will evaluate.
Control | 1..1
Type | id
TestScript.setup.action.assert.ruleset.rule
Definition | The referenced rule within the external ruleset template.
Control | 0..*

Comments | This qualifies each param name so that a parameter with the same name can be used differently by the different rules with the ruleset.
TestScript.setup.action.assert.ruleset.rule.ruleId
Definition | Id of the referenced rule within the external ruleset template.
Control | 1..1
Type | id
TestScript.setup.action.assert.ruleset.rule.param
Definition | Each rule template can take one or more parameters for rule evaluation.
Control | 0..*
Comments | The parameter value can be dynamic at runtime.
TestScript.setup.action.assert.ruleset.rule.param.name
Definition | Descriptive name for this parameter that matches the external assert ruleset rule parameter name.
Control | 1..1
Type | string
Comments | The external rule template would be looking for the parameter by this name.
TestScript.setup.action.assert.ruleset.rule.param.value
Definition | The value for the parameter that will be passed on to the external ruleset rule template.
Control | 1..1
Type | string
Comments | This value overwrites the value (if any) specified in TestScript.ruleset.rule.param.value. The param value can be a string-representation of a number, string, or boolean that is expected. Test engines do have to look for placeholders (${}) and replace the variable placeholders with the variable values at runtime before supplying this value to the external rule template.
TestScript.setup.action.assert.sourceId
Definition | Fixture to evaluate the XPath/JSONPath expression or the headerField against.
Control | 0..1
Type | id
Comments | This can be a statically defined fixture (at the top of the testscript) or a dynamically set fixture created by responseId of the action.operation element.
TestScript.setup.action.assert.validateProfileId
Definition | The ID of the Profile to validate against.
Control | 0..1
Type | id
Comments | The ID of a Profile fixture. Asserts that the response is valid according to the Profile specified by validateProfileId.
TestScript.setup.action.assert.value
Definition | The value to compare to.
Control | 0..1
Type | string
Comments | The string-representation of a number, string, or boolean that is expected. Test engines do have to look for placeholders (${}) and replace the variable placeholders with the variable values at runtime before comparing this value to the actual value.
TestScript.setup.action.assert.warningOnly
Definition | Whether or not the test execution will produce a warning only on error for this assert.
Control | 0..1
Type | boolean
Default Value | false
Comments | If this element is specified and it is true, then assertion failures can be logged by test engine but should not stop the test script execution from proceeding. There are likely cases where the spec is not clear on what should happen. If the spec says something is optional (maybe a response header for example), but a server doesn’t do it, we could choose to issue a warning.
TestScript.test
Definition | A test in this script.
Control | 0..*
TestScript.test.name
Definition | The name of this test used for tracking/logging purposes by test engines.
Control | 0..1
Type | string
TestScript.test.description
Definition | A short description of the test used by test engines for tracking and reporting purposes.
Control | 0..1
Type | string
TestScript.test.action
Definition | Action would contain either an operation or an assertion.
Control | 1..*
Comments | An action should contain either an operation or an assertion but not both. It can contain any number of variables.
Invariants | Defined on this element inv-2: Test action SHALL contain either an operation or assert but not both. (expression : operation.exists() xor assert.exists(), xpath: (f:operation or f:assert) and not(f:operation and f:assert))
TestScript.test.action.operation
Definition | An operation would involve a REST request to a server.
Control | 0..1
Type | See TestScript.setup.action.operation
Invariants | Defined on this element inv-8: Test operation SHALL contain either sourceId or targetId or params or url. (expression : sourceId.exists() or (targetId.count() + url.count() + params.count() = 1) or (type.code in ('capabilities' | 'search' | 'transaction' | 'history')), xpath: f:sourceId or (f:targetId or f:url or f:params) and (count(f:targetId) + count(f:url) + count(f:params) =1) or (f:type/f:code/@value='capabilities' or f:type/f:code/@value='search' or f:type/f:code/@value='transaction' or f:type/f:code/@value='history'))
TestScript.test.action.assert
Definition | Evaluates the results of previous operations to determine if the server under test behaves appropriately.

Control | 0..1
Type | See TestScript.setup.action.assert
Comments | In order to evaluate an assertion, the request, response, and results of the most recently executed operation must always be maintained by the test engine.
Invariants | Defined on this element inv-11: Test action assert SHALL contain either compareToSourceId and compareToSourceExpression, compareToSourceId and compareToSourcePath or neither. (expression : compareToSourceId.empty() xor (compareToSourceExpression.exists() or compareToSourcePath.exists()), xpath: (f:compareToSourceId and f:compareToSourceExpression) or (f:compareToSourceId and f:compareToSourcePath) or not(f:compareToSourceId or f:compareToSourceExpression or f:compareToSourcePath)) inv-13: Test action assert response and response and responseCode SHALL be empty when direction equals request (expression : (response.empty() and responseCode.empty() and direction = 'request') or direction.empty() or direction = 'response', xpath: ((count(f:response) + count(f:responseCode)) = 0 and (f:direction/@value='request')) or (count(f:direction) = 0) or (f:direction/@value='response')) inv-6: Only a single assertion SHALL be present within test action assert element. (expression : contentType.count() + expression.count() + headerField.count() + minimumId.count() + navigationLinks.count() + path.count() + requestMethod.count() + resource.count() + responseCode.count() + response.count() + rule.count() + ruleset.count() + validateProfileId.count() <=1, xpath: count(f:contentType) + count(f:expression) + count(f:headerField) + count(f:minimumId) + count(f:navigationLinks) + count(f:path) + count(f:requestMethod) + count(f:resource) + count(f:responseCode) + count(f:response) + count(f:rule) + count(f:ruleset) + count(f:validateProfileId) <=1)
TestScript.teardown
Definition | A series of operations required to clean up after the all the tests are executed (successfully or otherwise).
Control | 0..1
TestScript.teardown.action
Definition | The teardown action will only contain an operation.
Control | 1..*
Comments | An action should contain either an operation or an assertion but not both. It can contain any number of variables.
TestScript.teardown.action.operation
Definition | An operation would involve a REST request to a server.
Control | 1..1
Type | See TestScript.setup.action.operation
Invariants | Defined on this element inv-9: Teardown operation SHALL contain either sourceId or targetId or params or url. (expression : sourceId.exists() or (targetId.count() + url.count() + params.count() = 1) or (type.code in ('capabilities' | 'search' | 'transaction' | 'history')), xpath: f:sourceId or (f:targetId or f:url or (f:params and f:resource)) and (count(f:targetId) + count(f:url) + count(f:params) =1) or (f:type/f:code/@value='capabilities' or f:type/f:code/@value='search' or f:type/f:code/@value='transaction' or f:type/f:code/@value='history'))

7.3.9 Resource TestScript - Mappings

URL: https://hl7.org/fhir/STU3/testscript-mappings.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.3.9 Resource TestScript - Mappings

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative | Compartments: Not linked to any defined compartments

Mappings:

RIM Mapping

Mappings for the testscript resource.

7.3.9.1 Mappings for RIM Mapping (http://hl7.org/v3)

TestScript | n/a

7.3.10 Resource TestScript - Design Notes

URL: https://hl7.org/fhir/STU3/testscript-explanations.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.3.10 Resource TestScript - Design Notes

to do

7.3.11 Resource TestScript - Extensions & Profiles

URL: https://hl7.org/fhir/STU3/testscript-profiles.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.3.11 Resource TestScript - Extensions & Profiles

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

This table lists profiles and extensions for the TestScript resource. For background information, see Profiling Resources and Extensibility. Additional profiles and resources may be found in published Implementation Guides , or in the Conformance resource registry .

7.3.11.1 Profiles

No Profiles defined for this resource

7.3.11.2 Extensions

No Extensions defined for this resource

Extensions that reference this resource:

No Extensions refer to this resource

7.3.11.3 Search Extensions

No Search Extensions defined for this resource

7.3.12 Resource TestScript - Detailed Descriptions

URL: https://hl7.org/fhir/STU3/testscript-version-maps.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.3.12 Resource TestScript - Detailed Descriptions

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative | Compartments: Not linked to any defined compartments

R2 : R3 Converson maps for TestScript.

Functional status for this map: 6 tests that all execute ok. 6 fail round-trip testing and 6 r3 resources are invalid (11 errors). (see documentation)

7.3.12.1 R2 to R3

```
map "http://hl7.org/fhir/StructureMap/TestScript2to3" = "R2 to R3 Conversions for TestScript"

uses "http://hl7.org/fhir/DSTU2/StructureDefinition/TestScript" alias TestScriptR2 as source
uses "http://hl7.org/fhir/StructureDefinition/TestScript" alias TestScript as target

imports "http://hl7.org/fhir/StructureMap/primitives2to3"
imports "http://hl7.org/fhir/StructureMap/DomainResource2to3"
imports "http://hl7.org/fhir/StructureMap/ContactPoint2to3"
imports "http://hl7.org/fhir/StructureMap/Identifier2to3"
imports "http://hl7.org/fhir/StructureMap/Reference2to3"

group for type+types  TestScript extends DomainResource
  input src : TestScriptR2 as source
  input tgt : TestScript as target

  "TestScript-url" : for src.url make tgt.url
  "TestScript-version" : for src.version make tgt.version
  "TestScript-name" : for src.name make tgt.name
  "TestScript-status" : for src.status make tgt.status
  "TestScript-identifier" : for src.identifier make tgt.identifier
  "TestScript-experimental" : for src.experimental make tgt.experimental
  "TestScript-publisher" : for src.publisher make tgt.publisher
  "TestScript-contact" : for src.contact as vs0 make tgt.contact as vt0 then {
    "ContactDetail-name" : for vs0.name make vt0.name
    "ContactDetail-telecom" : for vs0.telecom make vt0.telecom
  }

  "TestScript-date" : for src.date make tgt.date
  "TestScript-metadata" : for src.metadata as vs0 make tgt.metadata as vt0
  "TestScript-fixture" : for src.fixture as vs0 make tgt.fixture as vt0 then {
    "TestScript.fixture-autocreate" : for vs0.autocreate make vt0.autocreate
    "TestScript.fixture-autodelete" : for vs0.autodelete make vt0.autodelete
    "TestScript.fixture-resource" : for vs0.resource make vt0.resource
  }

  "TestScript-profile" : for src.profile make tgt.profile
  "TestScript-variable" : for src.variable as vs0 make tgt.variable as vt0 then {
    "TestScript.variable-name" : for vs0.name make vt0.name
    "TestScript.variable-headerField" : for vs0.headerField make vt0.headerField
    "TestScript.variable-path" : for vs0.path make vt0.path
    "TestScript.variable-sourceId" : for vs0.sourceId make vt0.sourceId
  }

  "TestScript-setup" : for src.setup as vs0 make tgt.setup as vt0
  "TestScript-test" : for src.test as vs0 make tgt.test as vt0 then {
    "TestScript.test-name" : for vs0.name make vt0.name
    "TestScript.test-description" : for vs0.description make vt0.description
  }

  "TestScript-teardown" : for src.teardown as vs0 make tgt.teardown as vt0

endgroup
```

7.3.12.2 R3 to R2

```
map "http://hl7.org/fhir/StructureMap/TestScript3to2" = "R3 to R2 Conversion for TestScript"

uses "http://hl7.org/fhir/StructureDefinition/TestScript" alias TestScript as source
```

```
uses "http://hl7.org/fhir/DSTU2/StructureDefinition/TestScript" alias TestScriptR2 as target

imports "http://hl7.org/fhir/StructureMap/primitives3to2"
imports "http://hl7.org/fhir/StructureMap/DomainResource3to2"
imports "http://hl7.org/fhir/StructureMap/ContactPoint3to2"
imports "http://hl7.org/fhir/StructureMap/Identifier3to2"
imports "http://hl7.org/fhir/StructureMap/Reference3to2"
imports "http://hl7.org/fhir/StructureMap/CodeableConcept3to2"

group for type+types  TestScript extends DomainResource
  input src : TestScript as source
  input tgt : TestScriptR2 as target

  "TestScript-url" : for src.url make tgt.url
  "TestScript-version" : for src.version make tgt.version
  "TestScript-name" : for src.name make tgt.name
  "TestScript-status" : for src.status make tgt.status
  "TestScript-identifier" : for src.identifier make tgt.identifier
  "TestScript-experimental" : for src.experimental make tgt.experimental
  "TestScript-publisher" : for src.publisher make tgt.publisher
  "TestScript-contact" : for src.contact as vs0 make tgt.contact as vt0 then {
    "TestScript.contact-name" : for vs0.name make vt0.name
    "TestScript.contact-telecom" : for vs0.telecom make vt0.telecom
  }

  "TestScript-date" : for src.date make tgt.date
  "TestScript-useContext" : for src.jurisdiction make tgt.useContext
  "TestScript-metadata" : for src.metadata as vs0 make tgt.metadata as vt0
  "TestScript-fixture" : for src.fixture as vs0 make tgt.fixture as vt0 then {
    "TestScript.fixture-autocreate" : for vs0.autocreate make vt0.autocreate
    "TestScript.fixture-autodelete" : for vs0.autodelete make vt0.autodelete
    "TestScript.fixture-resource" : for vs0.resource make vt0.resource
  }

  "TestScript-profile" : for src.profile make tgt.profile
  "TestScript-variable" : for src.variable as vs0 make tgt.variable as vt0 then {
    "TestScript.variable-name" : for vs0.name make vt0.name
    "TestScript.variable-headerField" : for vs0.headerField make vt0.headerField
    "TestScript.variable-path" : for vs0.path make vt0.path
    "TestScript.variable-sourceId" : for vs0.sourceId make vt0.sourceId
  }

  "TestScript-setup" : for src.setup as vs0 make tgt.setup as vt0
  "TestScript-test" : for src.test as vs0 make tgt.test as vt0 then {
    "TestScript.test-name" : for vs0.name make vt0.name
    "TestScript.test-description" : for vs0.description make vt0.description
  }

  "TestScript-teardown" : for src.teardown as vs0 make tgt.teardown as vt0

endgroup
```

7.3.12.3 R3 Validation Errors

example | ERROR @ TestScript.test: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test.action': minimum required = 1, but only found 0 (src = InstanceValidator)

multiserver | ERROR @ TestScript.test[1]: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test[1].action': minimum required = 1, but only found 0 (src = InstanceValidator) ERROR @ TestScript.test[2]: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test[2].action': minimum required = 1, but only found 0 (src = InstanceValidator) ERROR @ TestScript.test[3]: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test[3].action': minimum required = 1, but only found 0 (src = InstanceValidator)

history | ERROR @ TestScript.test: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test.action': minimum required = 1, but only found 0 (src = InstanceValidator)

ReadTestScript | ERROR @ TestScript.test[1]: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test[1].action': minimum required = 1, but only found 0 (src = InstanceValidator) ERROR @ TestScript.test[2]: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test[2].action': minimum required = 1, but only found 0 (src = InstanceValidator) ERROR @ TestScript.test[3]: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test[3].action': minimum required = 1, but only found 0 (src = InstanceValidator) ERROR @ TestScript.test[4]: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test[4].action': minimum required = 1, but only found 0 (src = InstanceValidator)

search | ERROR @ TestScript.test: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test.action': minimum required = 1, but only found 0 (src = InstanceValidator)

update | ERROR @ TestScript.test: Profile http://hl7.org/fhir/StructureDefinition/TestScript, Element 'TestScript.test.action': minimum required = 1, but only found 0 (src = InstanceValidator)

7.4 Resource TestReport - Content

URL: https://hl7.org/fhir/STU3/testreport.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

FHIR Infrastructure Work Group | Maturity Level: 0 | Draft | Compartments: Not linked to any defined compartments

A summary of information based on the results of executing a TestScript.

7.4.1 Scope and Usage

The TestScript resource is used to define tests that can be executed on one or more FHIR servers. The TestReport resource defines how systems should encode the summarized results of executing a TestScript.

The TestReport structure mirrors the TestScript concepts of having sections for setup, tests, and teardown. If present, these ordered lists should mirror the actions (either operations or assertions) of the referenced TestScript with a result code: pass, skip, fail, warning, or error.

7.4.2 Background and Context

7.4.2.1 Background

Implementers should be familiar with the testing concepts and descriptions found in Testing FHIR before working with this resource.

7.4.3 Resource Content

* Structure

* UML

* XML

* JSON

* Turtle

* R2 Diff

* All

Structure

Name | Flags | Card. | Type | Description & Constraints
TestReport | | | DomainResource | Describes the results of a TestScript execution Elements defined in Ancestors: id, meta, implicitRules, language, text, contained, extension, modifierExtension
identifier | ? | 0..1 | Identifier | External identifier
name | ? | 0..1 | string | Informal name of the executed TestScript
status | ?!? | 1..1 | code | completed | in-progress | waiting | stopped | entered-in-error TestReportStatus (Required)
testScript | ? | 1..1 | Reference(TestScript) | Reference to the version-specific TestScript that was executed to produce this TestReport
result | ? | 1..1 | code | pass | fail | pending TestReportResult (Required)
score | ? | 0..1 | decimal | The final score (percentage of tests passed) resulting from the execution of the TestScript
tester | ? | 0..1 | string | Name of the tester producing this report (Organization or individual)
issued | ? | 0..1 | dateTime | When the TestScript was executed and this TestReport was generated
participant | | 0..* | BackboneElement | A participant in the test execution, either the execution engine, a client, or a server
type | | 1..1 | code | test-engine | client | server TestReportParticipantType (Required)
uri | | 1..1 | uri | The uri of the participant. An absolute URL is preferred
display | | 0..1 | string | The display name of the participant
setup | | 0..1 | BackboneElement | The results of the series of required setup operations before the tests were executed
action | I | 1..* | BackboneElement | A setup operation or assert that was executed + Setup action SHALL contain either an operation or assert but not both.
operation | | 0..1 | BackboneElement | The operation to perform
result | | 1..1 | code | pass | skip | fail | warning | error TestReportActionResult (Required)
message | | 0..1 | markdown | A message associated with the result
detail | | 0..1 | uri | A link to further details on the result
assert | | 0..1 | BackboneElement | The assertion to perform
result | | 1..1 | code | pass | skip | fail | warning | error TestReportActionResult (Required)
message | | 0..1 | markdown | A message associated with the result
detail | | 0..1 | string | A link to further details on the result
test | | 0..* | BackboneElement | A test executed from the test script
name | | 0..1 | string | Tracking/logging name of this test
description | | 0..1 | string | Tracking/reporting short description of the test
action | I | 1..* | BackboneElement | A test operation or assert that was performed + Test action SHALL contain either an operation or assert but not both.
operation | | 0..1 | see operation | The operation performed
assert | | 0..1 | see assert | The assertion performed
teardown | | 0..1 | BackboneElement | The results of running the series of required clean up steps
action | | 1..* | BackboneElement | One or more teardown operations performed
operation | | 1..1 | see operation | The teardown operation performed
Documentation for this format

UML Diagram (Legend)

XML Template

```
<TestReport xmlns="http://hl7.org/fhir">
 <!-- from Resource: id, meta, implicitRules, and language -->
 <!-- from DomainResource: text, contained, extension, and modifierExtension -->
 <identifier><!-- 0..1 Identifier External identifier --></identifier>
 <name value="[string]"/><!-- 0..1 Informal name of the executed TestScript -->
 <status value="[code]"/><!-- 1..1 completed | in-progress | waiting | stopped | entered-in-error -->
 <testScript><!-- 1..1 Reference(TestScript) Reference to the  version-specific TestScript that was executed to produce
    this TestReport --></testScript>
```

```
 <result value="[code]"/><!-- 1..1 pass | fail | pending -->
 <score value="[decimal]"/><!-- 0..1 The final score (percentage of tests passed) resulting from the execution of the
    TestScript -->
 <tester value="[string]"/><!-- 0..1 Name of the tester producing this report (Organization or individual) -->
 <issued value="[dateTime]"/><!-- 0..1 When the TestScript was executed and this TestReport was generated -->
 <participant>  <!-- 0..* A participant in the test execution, either the execution engine, a client, or a server -->
  <type value="[code]"/><!-- 1..1 test-engine | client | server -->
  <uri value="[uri]"/><!-- 1..1 The uri of the participant. An absolute URL is preferred -->
  <display value="[string]"/><!-- 0..1 The display name of the participant -->
 </participant>
 <setup>  <!-- 0..1 The results of the series of required setup operations before the tests were executed -->
  <action>  <!-- 1..* A setup operation or assert that was executed -->
   <operation>  <!-- 0..1 The operation to perform -->
    <result value="[code]"/><!-- 1..1 pass | skip | fail | warning | error -->
    <message value="[markdown]"/><!-- 0..1 A message associated with the result -->
    <detail value="[uri]"/><!-- 0..1 A link to further details on the result -->
   </operation>
   <assert>  <!-- 0..1 The assertion to perform -->
    <result value="[code]"/><!-- 1..1 pass | skip | fail | warning | error -->
    <message value="[markdown]"/><!-- 0..1 A message associated with the result -->
    <detail value="[string]"/><!-- 0..1 A link to further details on the result -->
   </assert>
  </action>
 </setup>
 <test>  <!-- 0..* A test executed from the test script -->
  <name value="[string]"/><!-- 0..1 Tracking/logging name of this test -->
  <description value="[string]"/><!-- 0..1 Tracking/reporting short description of the test -->
  <action>  <!-- 1..* A test operation or assert that was performed -->
   <operation><!-- 0..1 Content as for TestReport.setup.action.operation The operation performed --></operation>
   <assert><!-- 0..1 Content as for TestReport.setup.action.assert The assertion performed --></assert>
  </action>
 </test>
 <teardown>  <!-- 0..1 The results of running the series of required clean up steps -->
  <action>  <!-- 1..* One or more teardown operations performed -->
   <operation><!-- 1..1 Content as for TestReport.setup.action.operation The teardown operation performed
    --></operation>
  </action>
 </teardown>
</TestReport>
```

JSON Template

```
{
  "resourceType" : "TestReport",
  // from Resource: id, meta, implicitRules, and language
  // from DomainResource: text, contained, extension, and modifierExtension
  "identifier" : { Identifier }, // External identifier
  "name" : "<string>", // Informal name of the executed TestScript
  "status" : "<code>", // R!  completed | in-progress | waiting | stopped | entered-in-error
  "testScript" : { Reference(TestScript) }, // R!  Reference to the  version-specific TestScript that was executed to
    produce this TestReport
  "result" : "<code>", // R!  pass | fail | pending
  "score" : <decimal>, // The final score (percentage of tests passed) resulting from the execution of the TestScript
  "tester" : "<string>", // Name of the tester producing this report (Organization or individual)
  "issued" : "<dateTime>", // When the TestScript was executed and this TestReport was generated
  "participant" : [{ // A participant in the test execution, either the execution engine, a client, or a server
    "type" : "<code>", // R!  test-engine | client | server
    "uri" : "<uri>", // R!  The uri of the participant. An absolute URL is preferred
    "display" : "<string>" // The display name of the participant
  }],
  "setup" : { // The results of the series of required setup operations before the tests were executed
    "action" : [{ // R!  A setup operation or assert that was executed
      "operation" : { // The operation to perform
        "result" : "<code>", // R!  pass | skip | fail | warning | error
        "message" : "<markdown>", // A message associated with the result
        "detail" : "<uri>" // A link to further details on the result
      },
      "assert" : { // The assertion to perform
        "result" : "<code>", // R!  pass | skip | fail | warning | error
        "message" : "<markdown>", // A message associated with the result
        "detail" : "<string>" // A link to further details on the result
      }
    }]
  },
  "test" : [{ // A test executed from the test script
    "name" : "<string>", // Tracking/logging name of this test
    "description" : "<string>", // Tracking/reporting short description of the test
    "action" : [{ // R!  A test operation or assert that was performed
      "operation" : { Content as for TestReport.setup.action.operation }, // The operation performed
      "assert" : { Content as for TestReport.setup.action.assert } // The assertion performed
    }]
  }],
  "teardown" : { // The results of running the series of required clean up steps
    "action" : [{ // R!  One or more teardown operations performed
      "operation" : { Content as for TestReport.setup.action.operation } // R!  The teardown operation performed
    }]
  }
}
```

Turtle Template

```
@prefix fhir: <http://hl7.org/fhir/> .


[ a fhir:TestReport;
  fhir:nodeRole fhir:treeRoot; # if this is the parser root

  # from Resource: .id, .meta, .implicitRules, and .language
  # from DomainResource: .text, .contained, .extension, and .modifierExtension
  fhir:TestReport.identifier [ Identifier ]; # 0..1 External identifier
  fhir:TestReport.name [ string ]; # 0..1 Informal name of the executed TestScript
  fhir:TestReport.status [ code ]; # 1..1 completed | in-progress | waiting | stopped | entered-in-error
  fhir:TestReport.testScript [ Reference(TestScript) ]; # 1..1 Reference to the  version-specific TestScript that was
    executed to produce this TestReport
  fhir:TestReport.result [ code ]; # 1..1 pass | fail | pending
  fhir:TestReport.score [ decimal ]; # 0..1 The final score (percentage of tests passed) resulting from the execution of
     the TestScript
  fhir:TestReport.tester [ string ]; # 0..1 Name of the tester producing this report (Organization or individual)
  fhir:TestReport.issued [ dateTime ]; # 0..1 When the TestScript was executed and this TestReport was generated
  fhir:TestReport.participant [ # 0..* A participant in the test execution, either the execution engine, a client, or a
    server
    fhir:TestReport.participant.type [ code ]; # 1..1 test-engine | client | server
    fhir:TestReport.participant.uri [ uri ]; # 1..1 The uri of the participant. An absolute URL is preferred
    fhir:TestReport.participant.display [ string ]; # 0..1 The display name of the participant
  ], ...;
  fhir:TestReport.setup [ # 0..1 The results of the series of required setup operations before the tests were executed
    fhir:TestReport.setup.action [ # 1..* A setup operation or assert that was executed
      fhir:TestReport.setup.action.operation [ # 0..1 The operation to perform
        fhir:TestReport.setup.action.operation.result [ code ]; # 1..1 pass | skip | fail | warning | error
        fhir:TestReport.setup.action.operation.message [ markdown ]; # 0..1 A message associated with the result
        fhir:TestReport.setup.action.operation.detail [ uri ]; # 0..1 A link to further details on the result
      ];
      fhir:TestReport.setup.action.assert [ # 0..1 The assertion to perform
        fhir:TestReport.setup.action.assert.result [ code ]; # 1..1 pass | skip | fail | warning | error
        fhir:TestReport.setup.action.assert.message [ markdown ]; # 0..1 A message associated with the result
        fhir:TestReport.setup.action.assert.detail [ string ]; # 0..1 A link to further details on the result
      ];
    ], ...;
  ];
  fhir:TestReport.test [ # 0..* A test executed from the test script
    fhir:TestReport.test.name [ string ]; # 0..1 Tracking/logging name of this test
    fhir:TestReport.test.description [ string ]; # 0..1 Tracking/reporting short description of the test
    fhir:TestReport.test.action [ # 1..* A test operation or assert that was performed
      fhir:TestReport.test.action.operation [ See TestReport.setup.action.operation ]; # 0..1 The operation performed
      fhir:TestReport.test.action.assert [ See TestReport.setup.action.assert ]; # 0..1 The assertion performed
    ], ...;
  ], ...;
  fhir:TestReport.teardown [ # 0..1 The results of running the series of required clean up steps
    fhir:TestReport.teardown.action [ # 1..* One or more teardown operations performed
      fhir:TestReport.teardown.action.operation [ See TestReport.setup.action.operation ]; # 1..1 The teardown operation
     performed
    ], ...;
  ];
]
```

Changes since DSTU2

This resource did not exist in Release 2

This analysis is available as XML or JSON.

Structure

Name | Flags | Card. | Type | Description & Constraints

TestReport | | | DomainResource | Describes the results of a TestScript execution Elements defined in Ancestors: id, meta, implicitRules, language, text, contained, extension, modifierExtension

identifier | ? | 0..1 | Identifier | External identifier

name | ? | 0..1 | string | Informal name of the executed TestScript

status | ?!? | 1..1 | code | completed | in-progress | waiting | stopped | entered-in-error TestReportStatus (Required)

testScript | ? | 1..1 | Reference(TestScript) | Reference to the version-specific TestScript that was executed to produce this TestReport

result | ? | 1..1 | code | pass | fail | pending TestReportResult (Required)

score | ? | 0..1 | decimal | The final score (percentage of tests passed) resulting from the execution of the TestScript

tester | ? | 0..1 | string | Name of the tester producing this report (Organization or individual)

issued | ? | 0..1 | dateTime | When the TestScript was executed and this TestReport was generated

participant | | 0..* | BackboneElement | A participant in the test execution, either the execution engine, a client, or a server

type | | 1..1 | code | test-engine | client | server TestReportParticipantType (Required)

uri | | 1..1 | uri | The uri of the participant. An absolute URL is preferred

display | | 0..1 | string | The display name of the participant

setup | | 0..1 | BackboneElement | The results of the series of required setup operations before the tests were executed

action | I | 1..* | BackboneElement | A setup operation or assert that was executed + Setup action SHALL contain either an operation or assert but not both.

operation | | 0..1 | BackboneElement | The operation to perform

result | | 1..1 | code | pass | skip | fail | warning | error TestReportActionResult (Required)

message | | 0..1 | markdown | A message associated with the result

detail | | 0..1 | uri | A link to further details on the result

assert | | 0..1 | BackboneElement | The assertion to perform

result | | 1..1 | code | pass | skip | fail | warning | error TestReportActionResult (Required)

message | | 0..1 | markdown | A message associated with the result
detail | | 0..1 | string | A link to further details on the result
test | | 0..* | BackboneElement | A test executed from the test script
name | | 0..1 | string | Tracking/logging name of this test
description | | 0..1 | string | Tracking/reporting short description of the test
action | I | 1..* | BackboneElement | A test operation or assert that was performed + Test action SHALL contain either an operation or assert but not both.
operation | | 0..1 | see operation | The operation performed
assert | | 0..1 | see assert | The assertion performed
teardown | | 0..1 | BackboneElement | The results of running the series of required clean up steps
action | | 1..* | BackboneElement | One or more teardown operations performed
operation | | 1..1 | see operation | The teardown operation performed
Documentation for this format

UML Diagram (Legend)

XML Template

```
<TestReport xmlns="http://hl7.org/fhir">
 <!-- from Resource: id, meta, implicitRules, and language -->
 <!-- from DomainResource: text, contained, extension, and modifierExtension -->
 <identifier><!-- 0..1 Identifier External identifier --></identifier>
 <name value="[string]"/><!-- 0..1 Informal name of the executed TestScript -->
 <status value="[code]"/><!-- 1..1 completed | in-progress | waiting | stopped | entered-in-error -->
 <testScript><!-- 1..1 Reference(TestScript) Reference to the  version-specific TestScript that was executed to produce
    this TestReport --></testScript>
 <result value="[code]"/><!-- 1..1 pass | fail | pending -->
 <score value="[decimal]"/><!-- 0..1 The final score (percentage of tests passed) resulting from the execution of the
    TestScript -->
 <tester value="[string]"/><!-- 0..1 Name of the tester producing this report (Organization or individual) -->
 <issued value="[dateTime]"/><!-- 0..1 When the TestScript was executed and this TestReport was generated -->
 <participant>  <!-- 0..* A participant in the test execution, either the execution engine, a client, or a server -->
  <type value="[code]"/><!-- 1..1 test-engine | client | server -->
  <uri value="[uri]"/><!-- 1..1 The uri of the participant. An absolute URL is preferred -->
  <display value="[string]"/><!-- 0..1 The display name of the participant -->
 </participant>
 <setup>  <!-- 0..1 The results of the series of required setup operations before the tests were executed -->
  <action>  <!-- 1..* A setup operation or assert that was executed -->
   <operation>  <!-- 0..1 The operation to perform -->
    <result value="[code]"/><!-- 1..1 pass | skip | fail | warning | error -->
    <message value="[markdown]"/><!-- 0..1 A message associated with the result -->
    <detail value="[uri]"/><!-- 0..1 A link to further details on the result -->
   </operation>
   <assert>  <!-- 0..1 The assertion to perform -->
    <result value="[code]"/><!-- 1..1 pass | skip | fail | warning | error -->
    <message value="[markdown]"/><!-- 0..1 A message associated with the result -->
    <detail value="[string]"/><!-- 0..1 A link to further details on the result -->
   </assert>
  </action>
 </setup>
 <test>  <!-- 0..* A test executed from the test script -->
  <name value="[string]"/><!-- 0..1 Tracking/logging name of this test -->
  <description value="[string]"/><!-- 0..1 Tracking/reporting short description of the test -->
  <action>  <!-- 1..* A test operation or assert that was performed -->
   <operation><!-- 0..1 Content as for TestReport.setup.action.operation The operation performed --></operation>
   <assert><!-- 0..1 Content as for TestReport.setup.action.assert The assertion performed --></assert>
  </action>
 </test>
 <teardown>  <!-- 0..1 The results of running the series of required clean up steps -->
  <action>  <!-- 1..* One or more teardown operations performed -->
   <operation><!-- 1..1 Content as for TestReport.setup.action.operation The teardown operation performed
    --></operation>
  </action>
 </teardown>
</TestReport>
```

JSON Template

```
{
  "resourceType" : "TestReport",
  // from Resource: id, meta, implicitRules, and language
  // from DomainResource: text, contained, extension, and modifierExtension
  "identifier" : { Identifier }, // External identifier
  "name" : "<string>", // Informal name of the executed TestScript
  "status" : "<code>", // R!  completed | in-progress | waiting | stopped | entered-in-error
  "testScript" : { Reference(TestScript) }, // R!  Reference to the  version-specific TestScript that was executed to
    produce this TestReport
  "result" : "<code>", // R!  pass | fail | pending
  "score" : <decimal>, // The final score (percentage of tests passed) resulting from the execution of the TestScript
  "tester" : "<string>", // Name of the tester producing this report (Organization or individual)
  "issued" : "<dateTime>", // When the TestScript was executed and this TestReport was generated
  "participant" : [{ // A participant in the test execution, either the execution engine, a client, or a server
    "type" : "<code>", // R!  test-engine | client | server
    "uri" : "<uri>", // R!  The uri of the participant. An absolute URL is preferred
    "display" : "<string>" // The display name of the participant
  }],
  "setup" : { // The results of the series of required setup operations before the tests were executed
    "action" : [{ // R!  A setup operation or assert that was executed
```

```
      "operation" : { // The operation to perform
        "result" : "<code>", // R!  pass | skip | fail | warning | error
        "message" : "<markdown>", // A message associated with the result
        "detail" : "<uri>" // A link to further details on the result
      },
      "assert" : { // The assertion to perform
        "result" : "<code>", // R!  pass | skip | fail | warning | error
        "message" : "<markdown>", // A message associated with the result
        "detail" : "<string>" // A link to further details on the result
      }
    }]
  },
  "test" : [{ // A test executed from the test script
    "name" : "<string>", // Tracking/logging name of this test
    "description" : "<string>", // Tracking/reporting short description of the test
    "action" : [{ // R!  A test operation or assert that was performed
      "operation" : { Content as for TestReport.setup.action.operation }, // The operation performed
      "assert" : { Content as for TestReport.setup.action.assert } // The assertion performed
    }]
  }],
  "teardown" : { // The results of running the series of required clean up steps
    "action" : [{ // R!  One or more teardown operations performed
      "operation" : { Content as for TestReport.setup.action.operation } // R!  The teardown operation performed
    }]
  }
}
```

Turtle Template

```
@prefix fhir: <http://hl7.org/fhir/> .


[ a fhir:TestReport;
  fhir:nodeRole fhir:treeRoot; # if this is the parser root

  # from Resource: .id, .meta, .implicitRules, and .language
  # from DomainResource: .text, .contained, .extension, and .modifierExtension
  fhir:TestReport.identifier [ Identifier ]; # 0..1 External identifier
  fhir:TestReport.name [ string ]; # 0..1 Informal name of the executed TestScript
  fhir:TestReport.status [ code ]; # 1..1 completed | in-progress | waiting | stopped | entered-in-error
  fhir:TestReport.testScript [ Reference(TestScript) ]; # 1..1 Reference to the  version-specific TestScript that was
    executed to produce this TestReport
  fhir:TestReport.result [ code ]; # 1..1 pass | fail | pending
  fhir:TestReport.score [ decimal ]; # 0..1 The final score (percentage of tests passed) resulting from the execution of
     the TestScript
  fhir:TestReport.tester [ string ]; # 0..1 Name of the tester producing this report (Organization or individual)
  fhir:TestReport.issued [ dateTime ]; # 0..1 When the TestScript was executed and this TestReport was generated
  fhir:TestReport.participant [ # 0..* A participant in the test execution, either the execution engine, a client, or a
    server
    fhir:TestReport.participant.type [ code ]; # 1..1 test-engine | client | server
    fhir:TestReport.participant.uri [ uri ]; # 1..1 The uri of the participant. An absolute URL is preferred
    fhir:TestReport.participant.display [ string ]; # 0..1 The display name of the participant
  ], ...;
  fhir:TestReport.setup [ # 0..1 The results of the series of required setup operations before the tests were executed
    fhir:TestReport.setup.action [ # 1..* A setup operation or assert that was executed
      fhir:TestReport.setup.action.operation [ # 0..1 The operation to perform
        fhir:TestReport.setup.action.operation.result [ code ]; # 1..1 pass | skip | fail | warning | error
        fhir:TestReport.setup.action.operation.message [ markdown ]; # 0..1 A message associated with the result
        fhir:TestReport.setup.action.operation.detail [ uri ]; # 0..1 A link to further details on the result
      ];
      fhir:TestReport.setup.action.assert [ # 0..1 The assertion to perform
        fhir:TestReport.setup.action.assert.result [ code ]; # 1..1 pass | skip | fail | warning | error
        fhir:TestReport.setup.action.assert.message [ markdown ]; # 0..1 A message associated with the result
        fhir:TestReport.setup.action.assert.detail [ string ]; # 0..1 A link to further details on the result
      ];
    ], ...;
  ];
  fhir:TestReport.test [ # 0..* A test executed from the test script
    fhir:TestReport.test.name [ string ]; # 0..1 Tracking/logging name of this test
    fhir:TestReport.test.description [ string ]; # 0..1 Tracking/reporting short description of the test
    fhir:TestReport.test.action [ # 1..* A test operation or assert that was performed
      fhir:TestReport.test.action.operation [ See TestReport.setup.action.operation ]; # 0..1 The operation performed
      fhir:TestReport.test.action.assert [ See TestReport.setup.action.assert ]; # 0..1 The assertion performed
    ], ...;
  ], ...;
  fhir:TestReport.teardown [ # 0..1 The results of running the series of required clean up steps
    fhir:TestReport.teardown.action [ # 1..* One or more teardown operations performed
      fhir:TestReport.teardown.action.operation [ See TestReport.setup.action.operation ]; # 1..1 The teardown operation
     performed
    ], ...;
  ];
]
```

Changes since DSTU2

This resource did not exist in Release 2

This analysis is available as XML or JSON.

Alternate definitions: Master Definition (XML, JSON), XML Schema/Schematron (for) + JSON Schema, ShEx (for Turtle)

7.4.3.1 Terminology Bindings

Path | Definition | Type | Reference
TestReport.status | The current status of the test report. | Required | TestReportStatus
TestReport.result | The reported execution result. | Required | TestReportResult
TestReport.participant.type | The type of participant. | Required | TestReportParticipantType
TestReport.setup.action.operation.result TestReport.setup.action.assert.result | The results of executing an action. | Required | TestReportActionResult

7.4.3.2 Constraints

* inv-1: On TestReport.setup.action: Setup action SHALL contain either an operation or assert but not both. (expression on TestReport.setup.action: operation.exists() xor assert.exists())

* inv-2: On TestReport.test.action: Test action SHALL contain either an operation or assert but not both. (expression on TestReport.test.action: operation.exists() xor assert.exists())

7.4.4 Notes

Please refer to the Testing FHIR - How Tos for examples on working with this resource.

7.4.5 Search Parameters

Search parameters for this resource. The common parameters also apply. See Searching for more information about searching in REST, messaging, and services.

Name | Type | Description | Expression | In Common
identifier | token | An external identifier for the test report | TestReport.identifier
issued | date | The test report generation date | TestReport.issued
participant | uri | The reference to a participant in the test execution | TestReport.participant.uri
result | token | The result disposition of the test execution | TestReport.result
tester | string | The name of the testing organization | TestReport.tester
testscript | reference | The test script executed to produce this report | TestReport.testScript (TestScript)

7.4.6 Resource TestReport - Examples

URL: https://hl7.org/fhir/STU3/testreport-examples.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.4.6 Resource TestReport - Examples

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative | Compartments: Not linked to any defined compartments

Example List:

General TestReport Example | testreport-example | XML | JSON | Turtle

Usage note: every effort has been made to ensure that the examples are correct and useful, but they are not a normative part of the specification.

7.4.7 Resource TestReport - Detailed Descriptions

URL: https://hl7.org/fhir/STU3/testreport-definitions.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.4.7 Resource TestReport - Detailed Descriptions

FHIR Infrastructure Work Group | Maturity Level: 0 | Draft | Compartments: Not linked to any defined compartments

Detailed Descriptions for the elements in the TestReport resource.

TestReport
Definition | A summary of information based on the results of executing a TestScript.
Control | 1..1
TestReport.identifier
Definition | Identifier for the TestScript assigned for external purposes outside the context of FHIR.
Note | This is a business identifer, not a resource identifier (see discussion)
Control | 0..1
Type | Identifier
Summary | true
TestReport.name
Definition | A free text natural language name identifying the executed TestScript.
Control | 0..1
Type | string
Summary | true
Comments | Not expected to be globally unique.
TestReport.status

Definition | The current state of this test report.
Control | 1..1
Terminology Binding | TestReportStatus (Required)
Type | code
Is Modifier | true
Summary | true
Comments | The status represents where the execution is currently within the test script execution life cycle. This element is labeled as a modifier because the status contains codes that mark the resource as not currently valid.
TestReport.testScript
Definition | Ideally this is an absolute URL that is used to identify the version-specific TestScript that was executed, matching the TestScript.url.
Control | 1..1
Type | Reference(TestScript)
Summary | true
TestReport.result
Definition | The overall result from the execution of the TestScript.
Control | 1..1
Terminology Binding | TestReportResult (Required)
Type | code
Summary | true
Comments | The pass and fail result represents a completed test script execution. The pending result represents a test script execution that has not yet started or is currently in progress.
TestReport.score
Definition | The final score (percentage of tests passed) resulting from the execution of the TestScript.
Control | 0..1
Type | decimal
Summary | true
TestReport.tester
Definition | Name of the tester producing this report (Organization or individual).
Control | 0..1
Type | string
Requirements | Helps establish the "authority/credibility" of the TestReport. May also allow for contact.
Summary | true
Comments | Usually an organization, but may be an individual. This item SHOULD be populated unless the information is available from context.
TestReport.issued
Definition | When the TestScript was executed and this TestReport was generated.
Control | 0..1
Type | dateTime
Summary | true
Comments | Additional specific dates may be added as extensions.
TestReport.participant
Definition | A participant in the test execution, either the execution engine, a client, or a server.
Control | 0..*
TestReport.participant.type
Definition | The type of participant.
Control | 1..1
Terminology Binding | TestReportParticipantType (Required)
Type | code
TestReport.participant.uri
Definition | The uri of the participant. An absolute URL is preferred.
Control | 1..1
Type | uri
TestReport.participant.display
Definition | The display name of the participant.
Control | 0..1
Type | string
TestReport.setup
Definition | The results of the series of required setup operations before the tests were executed.
Control | 0..1
TestReport.setup.action
Definition | Action would contain either an operation or an assertion.
Control | 1..*
Comments | An action should contain either an operation or an assertion but not both. It can contain any number of variables.
Invariants | Defined on this element inv-1: Setup action SHALL contain either an operation or assert but not both. (expression : operation.exists() xor assert.exists(), xpath: (f:operation or f:assert) and not(f:operation and f:assert))
TestReport.setup.action.operation
Definition | The operation performed.
Control | 0..1
TestReport.setup.action.operation.result
Definition | The result of this operation.
Control | 1..1
Terminology Binding | TestReportActionResult (Required)
Type | code

TestReport.setup.action.operation.message
Definition | An explanatory message associated with the result.
Control | 0..1
Type | markdown
TestReport.setup.action.operation.detail
Definition | A link to further details on the result.
Control | 0..1
Type | uri
TestReport.setup.action.assert
Definition | The results of the assertion performed on the previous operations.
Control | 0..1
TestReport.setup.action.assert.result
Definition | The result of this assertion.
Control | 1..1
Terminology Binding | TestReportActionResult (Required)
Type | code
TestReport.setup.action.assert.message
Definition | An explanatory message associated with the result.
Control | 0..1
Type | markdown
TestReport.setup.action.assert.detail
Definition | A link to further details on the result.
Control | 0..1
Type | string
TestReport.test
Definition | A test executed from the test script.
Control | 0..*
TestReport.test.name
Definition | The name of this test used for tracking/logging purposes by test engines.
Control | 0..1
Type | string
TestReport.test.description
Definition | A short description of the test used by test engines for tracking and reporting purposes.
Control | 0..1
Type | string
TestReport.test.action
Definition | Action would contain either an operation or an assertion.
Control | 1..*
Comments | An action should contain either an operation or an assertion but not both. It can contain any number of variables.
Invariants | Defined on this element inv-2: Test action SHALL contain either an operation or assert but not both. (expression : operation.exists() xor assert.exists(), xpath: (f:operation or f:assert) and not(f:operation and f:assert))
TestReport.test.action.operation
Definition | An operation would involve a REST request to a server.
Control | 0..1
Type | See TestReport.setup.action.operation
TestReport.test.action.assert
Definition | The results of the assertion performed on the previous operations.
Control | 0..1
Type | See TestReport.setup.action.assert
TestReport.teardown
Definition | The results of the series of operations required to clean up after the all the tests were executed (successfully or otherwise).
Control | 0..1
TestReport.teardown.action
Definition | The teardown action will only contain an operation.
Control | 1..*
Comments | An action should contain either an operation or an assertion but not both. It can contain any number of variables.
TestReport.teardown.action.operation
Definition | An operation would involve a REST request to a server.
Control | 1..1
Type | See TestReport.setup.action.operation

7.4.8 Resource TestReport - Mappings

URL: https://hl7.org/fhir/STU3/testreport-mappings.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.4.8 Resource TestReport - Mappings

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative | Compartments: Not linked to any defined compartments

Mappings:

RIM Mapping

Mappings for the testreport resource.

7.4.8.1 Mappings for RIM Mapping (http://hl7.org/v3)

TestReport | n/a

7.4.9 Resource TestReport - Design Notes

URL: https://hl7.org/fhir/STU3/testreport-explanations.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.4.9 Resource TestReport - Design Notes

to do

7.4.10 Resource TestReport - Extensions & Profiles

URL: https://hl7.org/fhir/STU3/testreport-profiles.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.4.10 Resource TestReport - Extensions & Profiles

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

This table lists profiles and extensions for the TestReport resource. For background information, see Profiling Resources and Extensibility. Additional profiles and resources may be found in published Implementation Guides , or in the Conformance resource registry .

7.4.10.1 Profiles

No Profiles defined for this resource

7.4.10.2 Extensions

No Extensions defined for this resource

Extensions that reference this resource:

No Extensions refer to this resource

7.4.10.3 Search Extensions

No Search Extensions defined for this resource

7.5 Validating Resources

URL: https://hl7.org/fhir/STU3/validation.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.5 Validating Resources

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

This page provides a quick overview of how the FHIR specification supports validation of resources.

Validating a resource means, checking that the following aspects of the resource are valid:

* Structure: Check that all the content in the resource is described by the specification, and nothing extra is present

* Cardinality: Check that the cardinality of all properties is correct (min & max)

* Value Domains: Check that the values of all properties conform to the rules for the specified types (including checking that enumerated codes are valid)

* Coding/CodeableConcept bindings: Check that codes/displays provided in the Coding/CodeableConcept types are valid

* Invariants: Check that the invariants (co-occurrence rules, etc.) have been followed correctly

* Profiles: Check that any rules in profiles have been followed (including those listed in the Resource.meta.profile, or in CapabilityStatement, or in an ImplementationGuide, or otherwise required by context)

* Questionnaires: Check that a QuestionnaireResponse is valid against its matching Questionnaire

* Business Rules: Business rules are made outside the specification, such as checking for duplicates, checking that references resolve, checking that a user is authorized to do what they want to do, etc.

There are multiple ways to validate resources. This table summarizes the options described in this specification, and which of the aspects above they can validate:

Method | XML | JSON | RDF | Structure | Cardinality | Values | Bindings | Invariants | Profiles | Questionnaires | Business Rules

XML Schema
XML Schema + Schematron | | | | | | | | | 1
JSON Schema | | | | | | | | | 2
ShEx | | | | | | | 3
Validator
Validation Operation4

Notes:

* Schematron generated for a profile can test cardinality and invariants, but not bindings, and slicing is not really supported well

* JSON schema generated for a profile can test cardinality, and slicing is partially supported

* ShEx can enforce some bindings for well understood terminologies, but this is an ongoing area of development

* It is at the discretion of the server how much validation to perform, but most servers use the validation jar, or code derived from it, and offer the same services. Some servers also offer a web interface

Note that all these validation methods are incomplete; they can only validate the computable aspects of conformance. There are always additional rules made in narrative that they are not able to check (e.g. a rule such as "All the clinically important content in the data SHALL be in the narrative", which might be made in an implementation guide, but could never be checked by a conformance tool).

In case of disagreement between these conformance methods, note that:

* The schema/schematron is the least capable - mainly because it is not connected to a terminology server

* The java validator is only as good as the underlying definitions, and in particular, depends on whether the underlying terminology server supports all the relevant terminologies

* In general, the server validation operations use or derive from the java validation code, so have the same caveats

* The final arbiter is human inspection of the content of the resources, and the relevant implementation guides and base specification

Also, note that static testing of resource content is not enough to prove conformance to the specification. For further information, see FHIR Conformance Testing .

7.5.1 Using the XML schema

The XML schema can be used to validate XML representations of the resources. When validating a resource, you can nominate one of the following schema:

* fhir-all.xsd: links in all the individual modular schemas

* fhir-single.xsd: a single large file, mainly provided for schema processors that can't support circular references

In addition, the validation schema includes schematron that can be initiated with transform "iso_svrl_for_xslt2.xsl" included in the XML Tools download. Note that XSLT2 is required to run the schematrons.

When running the schematron, use the file "fhir-invariants.sch". This includes all the schematrons. The individual schematron files for each resource are provided to allow implementers to build their own smaller combined file that covers the relevant resource types for them.

7.5.2 Using the JSON schema

The JSON schema can be used with JSON schema validation software. Links:

* JSON Schema

* JSON Validation Software

7.5.3 Using the FHIR Validator

The FHIR Validator is a Java jar that is provided as part of the specification, and that is used during the publication process to validate that all the published examples. To execute the FHIR validator, follow the following steps:

* Download the FHIR Validator

* (optional) Download One of the FHIR definitions (with or without text)

* Execute the validator, providing the path to the definitions, and a reference to the resource to validate

Here is an example windows batch file that demonstrates the process (using the common utilities wget and 7z :

```
@ECHO OFF

 ECHO get the validator and unzip it
 wget http://hl7.org/fhir/STU3/validator.zip
 7z.exe x validator.zip

 ECHO Get the validation source file (dictionary)
 wget http://hl7.org/fhir/STU3/definitions.xml.zip

 ECHO 1. First example shows how to validate against the base spec:
 ECHO   a. get an example to validate
 wget http://hl7.org/fhir/STU3/patient-example.xml -O pat-ex.xml

 ECHO   b. validate it
 java -jar org.hl7.fhir.validator.jar pat-ex.xml -defn definitions.xml.zip
```

```
ECHO 2. Second example shows how to validate against a profile in the spec:
ECHO   a. get an example to validate
wget http://hl7.org/fhir/STU3/observation-example-heart-rate.xml -O obs-ex.xml

ECHO   b. validate it
java -jar org.hl7.fhir.validator.jar obs-ex.xml -defn definitions.xml.zip -profile
   http://hl7.org/fhir/StructureDefinition/heartrate

ECHO 3. Third example shows how to validate against a profile in an implementation guide:
ECHO   a. get an example to validate
wget http://hl7.org/fhir/STU3/observation-example-heart-rate.xml -O obs-ex.xml

ECHO   b. validate it. note that you have to tell the validator where to get the implementation guide information
java -jar org.hl7.fhir.validator.jar obs-ex.xml -defn definitions.xml.zip -ig http://hl7.org/fhir/us/core -profile
   http://hl7.org/fhir/us/core/StructureDefinition/us-core-patient

ECHO Press Any Key to Close
pause
```

Note that it is not necessary to download the resource first; the http address can be supplied directly:

```
java -jar org.hl7.fhir.validator.jar http://hl7.org/fhir/STU3//patient-example.html -defn validation-min.xml.zip
   -profile http://hl7.org/fhir/StructureDefinition/Patient
```

The validator requires an underlying terminology server. By default, this is http://fhir3.healthintersections.com.au. Use the parameter -tx to specify another server. For a full list of parameters and options, just run the validator.jar without any parameters.

7.5.4 Asking a FHIR Server

The operation validate can be used to check whether a resource conforms to a profile. The simplest way to execute this operation is to post the resource to a server:

```
POST [base]/Observation/$validate?profile=http://hl7.org/fhir/StructureDefinition/heartrate
[other HTTP headers]

<Observation>... resource to check as the body
```

The server will return an OperationOutcome resource listing issues found in the resource.

There are several things to consider when using this operation:

* Not all servers support the $validate operation, though some of the public test servers do

* Servers support the $validate operation generally will only validate against profiles already registered with the server

* Servers may choose to support either XML, JSON, or both

7.5.5 Via a web interface

Some servers expose the $validate functionality though a web page. For known public implementations, see the FHIR wiki

7.6 FHIR Mapping Language

URL: https://hl7.org/fhir/STU3/mapping-language.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.6.0 FHIR Mapping Language

FHIR Infrastructure Work Group | Maturity Level: 0 (Draft) | Ballot Status: Trial Use

The FHIR Specification includes a mapping language. The mapping language has a concrete syntax, defined and described in this page, and an abstract syntax, which is found in the StructureMap resource (and there is an antlr grammar for the concrete syntax). See also the Tutorial.

The mapping language describes how one set of Directed Acyclic Graphs (an instance) is transformed to another set of directed acyclic graphs. It is not necessary for the instances to have formal declarations and/or be strongly typed - just that they have named children that themselves have properties. On the other hand, when the instances are strongly typed - specifically, when they have formal definitions that are represented as Structure Definitions, the mapping language can use additional type related features.

The mapping language addresses two very different kinds of transformations:

* Structural changes between the source and target structures

* Differences in content and formats in string (and related) primitives contained within the structures

A map has 6 parts:

* Metadata

* Embedded ConceptMaps to translate between different code systems

* References to the structures involved in the mapping

* Imports: additional Maps used by this map

* A series of groups, each with a list of input variables

* A series of transformation rules in each group

7.6.0.1 Executing Maps

Maps are executed by a mapping engine. This takes one or more inputs of instances (directed acyclic graphs) and a map, and produces a set of outputs as specified by the map. The exact details of the form that the instances take are a matter for the map engine / application API. This language assumes that the engine can query an element in the instance for its children, its primitive value, and (optionally) its type. The language also assumes that the engine has application support for the following operations:

* ValueSet validation operation

* Translation operation

* Lookup another tree of data

* Create an instance tree

* Return the correct string format to refer to a tree (input or output)

These functions constitute a Mapping Support API that makes maps portable between different systems

Generally, it is assumed the invocation of the engine follows some pattern like this:

* The host application creates the engine, and passes it a handle to the standard services

* The host application chooses the model map resource, and asks the engine to prepare it (load, check, cache-up etc)

* The host application asks the engine to execute and provides a set of resources that match defined inputs in the map

* Any created output will be created via the standard API

Some host applications may be able to determine how to combine maps and inputs on the fly based on their metadata, and require minimal configuration, while others may require manual arrangements in order to manage the map execution process.

7.6.0.2 General Syntax Notes

Mapping files are always plain text in unicode. Whitespace is any unicode whitespace, and the particular whitespace used is not significant, except that unicode end of line characters terminate a comment. Comments are started by the characters "//".

The abstract model includes documentation for each item. The canonical text representation is for each item to be on its own line, with documentation at the end of the line as a comment.

All names defined by the map language - group, rule and variable names - must be valid ids (1-64 characters, upper and lowercase letters, numbers, dashes, dots and underscores), and must start with a letter. The special boolean values 'true' and 'false' are not allowed as variable names.

7.6.0.3 Metadata

The first part of the mapping syntax establishes the name of the mapping:

```
map "[url]" = "[name]"
```

The letters "map" are the first non-whitespace non-comment characters in the source. This is followed by the canonical URL that identifies the map uniquely, and then a human readable name for the map.

todo: add additional metadata?

7.6.0.4 Structure Definition References

The next section of the map references the set of structure definitions that are used or produced by this map.

```
uses "[url]" (alias name) as [mode] // documentation
```

This optional section lists one or more structure definitions that the map makes use of, and indicates for each structure definition, how it is used. It may also provide an Alias - a name used for the type inside the mapping language - this may be necessary when transforming from source to target where both source and target use overlapping type names (not unusual).

Any kind of structure definition may be referenced, including data types, resources, constraints on those, and logical models.

There are 4 modes in which a structure definition may be used:

* source: One of more instances of this type are passed to the mapping engine when the mapping is executed, and serve as the source from which mapping is performed

* queried: The map may ask the (via the API) for some instances of this type. For further discussion, see below

* target: One or more instances of this type are passed in, and will be populated from the source material

* produced: The map may ask (via the API) for some instances of this type to be created. For further discussion, see below

The simplest case, which is common, is where a single structure is converted to another single structure. in this case, the map specifies one target, and one source. Such maps are easy to use automatically - the host application has content in one format, creates an empty instance of the target, and asks the mapping engine to convert.

However, many mappings are not so simple. For instance, converting from a single CDA document to FHIR typically creates a set of resources. In this case, there is a single target - a Bundle, but it is also useful to specify a set of other structure definitions for resources that may be created as part of the bundle. Alternatively converting from one source model to another might involve looking up other information in other instances of data.

It's also possible for a map not so specify any structure definition dependencies. A map that doesn't indicate any structure definitions can still be used, but the type features of the map language can't be used, and such maps typically require special development to integrate the execution of the map into an application.

7.6.0.5 Map Imports

This section references additional maps that are used by this map:

```
imports "[url]" // documentation
```

Typically, maps that are imported are type based, such as a CDA --> FHIR map that makes use of a CD --> CodeableConcept map. How imported maps are actually used is discussed below.

The [url] may contain a "*" as a wildcard character to include any matching maps that are available to the mapping engine.

7.6.0.6 Groups

Each Mapping source contains one or more groups of rules. Each group defines a set of related mapping rules that take the same input and output variables, that define exactly which instances are passed to the mapping, and provides names by which they may be passed when invoking the map:

```
group (for type) [group-name] (extends [other-group])
  input [name] : [type] as [mode] // documentation
```

Each group has a name, which is how the mapping is invoked. The first group is special, in that this is the group invoked if no name is provided (e.g. starting the mapping by a host application).

The inputs of a group are also referred to (below) as its input parameters, or just as parameters; or as input variables.

Each input to the group has a name. This is the name that applications use when passing the instance to the invocation engine, or that rules use when invoking the group. Inputs may have a type - and should have (see the discussion above), but are not required to. Input variables also have a mode, which may be one of source or target (see above). There must be at least two input variables (source and target) - else there's nothing to map. Maps may have additional input or output inputs, where that's necessary.

Groups may extend other groups, which means that the rules in the other group also apply (typically, this is used with specialising classes in an OO context). When a group extends another group, it SHALL have the same input parameters (by name, type and mode) though their order may differ, and it MAY have additional parameters.

The key word for is used to indicate that this group provides a set of mappings that are intended to be used as the default way to map from source to target. There are 2 variants:

* for types: Use this by default when a [source] of the specified type must be converted to a [target] of the specified type, and no specific dependent rules (see below) are specified.

* for type+types: in addition, to the above use, when a [source] of the specified type must be converted to a [target] and the type of the target is not fixed, use this group

In both these cases, the group SHALL have 2 parameters, a source, and a target, in that order, and both SHALL have specified types for the inputs.

7.6.0.7 Transform Rules

The main portion of a map consists of a set of transform rules that describe how source content is transformed into target content. The full format for a rule looks like this:

```
name_of_rule: for src_context.field as new_variable where condition make tgt_context.field as new_variable =
    create([type]) then [details].
```

Each rule has 4 main sections:

* Name: The identity of the rule, for logging/debugging

* Source Content: One or more elements from the source that contribute to the mapping

* Target Transform: One or more specifications of content to create in the target model

* Dependent Rules: Specifies which - if any - rules or groups to apply within the scope of the rule

Rules in a group may be applied in any order; there is no sense of sequentially applying one rule after another.

7.6.0.7.1 Name

Each rule is assigned a name. The name is used when specifying rule links, and in traces (a record generated by the conversion engine recording the transform process). Names must be unique within the context of the map.

7.6.0.7.2 Source Content

Each rule specifies one or more elements taken from the source that define variables that can be used when specifying target content, or re-used in subsequent transforms on the rule. Multiple source elements are separated by a comma, like this:

```
rule_name: for [source], [source], make ...
```

Each [source] contains the following items:

```
context.element { : type {min..max}} {default [value]} { list-option } as variable where [FHIRPath] check [FHIRPath]
```

* context: A variable that is the context in which this rule applies. Variables are not prefixed, and must be declared elsewhere in the map ('as variable' below, or in the input source variables)

* element: An optional name of a child element of the context. If this is not provided, the source is the context. If this is provided, the rule will apply once for each element on the context that matches this name. If the element name contains spaces (possible in some contexts), it can be quoted using "

* type: A type may be specified. If a type is specified, only elements that have the specified type are selected as part of the source

* min..max: Specified cardinality. If a cardinality is specified, and the input document does not conform to the cardinality, the mapping engine raises an exception instead of completing the transformation

* default: A default value, used if there is no source value found. If there's a default value on an item that can repeat, it will only be used once

* list-option: by default, the rule will apply once for each occurrence of the element in context. The list option can override this to specify only to apply the first, last, or not to those

* as variable: if an element is specified, a variable must be assigned. This variable name may be used in the target statement, and may be re-used in other dependent rules (see below). There is one special case described below (simple form) where the variable name is provided explicitly

* where [condition]: a fluent path (ref) expression that is evaluated to boolean on the context. If the expression returns false, the source element has no match

* check [condition]: a fluent path (ref) expression that is evaluated to boolean on the context. If the expression returns false, the mapping engine terminates execution with an error. Note: it's usual to have either a where or a check clause, but possible to have both. If both are provided, the where clause is evaluated first, and the check only applies if the where expression is true

If all the source elements have a match the rule applies for the permutation of the source elements (e.g. if there are 2 elements, each with 2 matches, the rule applies 4 times, one for each combination). Typically, if there is more than one source element, only one of the elements can repeat.

Once the source statement is evaluated, the engine performing the evaluation has a list of variables, each of which contains a single value for each named variable. These variables are now mapped into the target structures in the target transformation.

7.6.0.7.3 Target Transform

Each rule specifies zero or more elements to be created in the target structure. These targets can also be assigned to variables that can be used in subsequent transform rules. If no targets are specified, there are not created targets, just newly defined source variables. Multiple target elements are separated by a comma, like this:

```
... make [target], [target] then by...
```

Each [target] contains the following items:

```
make context.element = transform_code(parameters...) as variable {list_modes}
```

* context: The name of a variable that is the context in which the content will be created. Variables must be declared elsewhere in the map ('as variable' below, or in the input target variables)

* element: The name of a child element that is valid in the context. The created value will be placed into the named element

* transform+parameters: details how the content that is created is transformed from the source data. See below for possible transform codes. If no transform code is provided, then the element is autocreated. It is an error if auto-created elements are primitive types, or have more than one possible type

* as variable: a variable name may be assigned, which allows the created item to be re-used in the dependent rules

* list_modes: control how elements that repeat are managed when the transform rule is evaluated

Context and Element are optional as a pair. If no context/element is specified, then a variable must be defined, and the created value is only available in the variable.

Each time the rule is applied, the engine determines the value from the transforms, considers the list mode, if required, and creates that specified content in the target instance. Within a given transform url, the targets are processed in order, so that a transform rule may refer to a variable defined by a prior transform rule.

The following list specifies that transforms that can be specified. Each transform takes one or more parameters:

Name | parameters | Documentation
copy | source | simply copy the source to the target as is (only allowed when the types in source and target match- typically for primitive types). In the concrete syntax, this is simply represented as the source variable
create | type | use the standard API to create a new instance of data. Where structure definitions have been provided, the type parameter must be a string which is a known type of a root element. Where they haven't, the application must know the name somehow
truncate | source, length | source must be some stringy type that has some meaningful length property
escape | source, format1, format2 | Change the internal escaping of a string element. Note: this is not often needed, as mostly the escaping is done on the base format
cast | source, type? | cast source from one type to another. target type can be left as implicit if there is one and only one target type known
append | source... | source is element or string - just append them all together
translate | source, map_uri, output | use the translate operation. The source is some type of code or coded datatype, and the source and map_uri are passed to the translate operation. The output determines what value from the translate operation is used for the result of the operation (code, system, display, Coding, or CodeableConcept)
reference | source | return a string that references the provided tree properly
dateOp | ?? | Perform a date operation. Parameters to be documented
uuid | n/a | Generate a random UUID (in lowercase). No Parameters
pointer | resource | Return the appropriate string to put in a Reference that refers to the resource provided as a parameter
evaluate | resource | Execute the supplied FHIRPath expression and use the value returned by that. The 2nd parameter - FHIRPath expression - is evaluated in the context of the first parameter, and the result used as the value. In the concrete syntax, there is a short hand for this operation, by

supplying () around the parameter. In this case, there is no context for the FHIRPath expression, and it must start with a reference to one of the defined variables

cc | (text) or (system. Code[, display]) | Create a CodeableConcept from the parameters provided

c | system. Code[, display] | Create a Coding from the parameters provided

qty | (text) or (value, unit, [system, code]) | Create a quantity. Parameters = (text) or (value, unit, [system, code]) where text =s the natural represenation e.g. [comparator]value[space]unit

id | system, value[, type] | Create an identifier. where type is a code from the identifier type value set

cp | (value) or (system, value) | Create a contact details. If no system is provided, the system should be inferred from the content of the value

TODO: explain how optional parameters work with transforms (append only?), document list mode

7.6.0.7.4 Dependent Rules

Once the source elements are evaluated, and any specifed targets created, the engine has a set of variables that represent source and target contexts in which further mapping may occur. The set of variables includes those provided to the group that contains the rule, and those created by the application of the rule. For some created elements that are primitive types, that's the end of the road - there's nothing more to do with them. But if either or both the source and target types are complex, there are usually additional mapping rules that need to apply to the newly created variables.

Transform rules specify what additional rules are evaluated when the rule is complete, by containing other rules:,

```
.. then {
  .. other rules...
}
```

When a rule contains other rules, the variables from the containing rules are all available to the contained rules. Alternatively, a rule can nominating another group of rules from the same or an imported mapping. Each rule or group is listed by name, and then a set of parameters are provided.

```
.. then rule(param, param)
```

The parameters provided must match the parameters required by the dependenct rule, in order. In addition, the mode of the variable must match - inputs that are targets must be target variables. Note, though, that target variables can be treated as source for a group.

Groups are resolved by name by looking through all the groups in all the available maps referened by the uses (see above) statements. The name must be unique within the scope of these maps.

7.6.0.8 Simple Form + Implicit Type Dependencies

If no dependent rules are specified, and if the is only one source and target, and they both speecify a variable, the rule can be written in an abbreviated form:

```
"name" : for src.element make tgt.element
```

This is implicitly the same as

```
"name" : for src.element as vvs make create([type]) tgt.element as vvt then [typeGroup](vvs, vvt)
```

Where type and typeGroup are determined by the context of src.element and tgt.element:

* typeGroup is determined by looking through the available rule groups labeled as "for types" or "for type+types" and checking the types of their inputs looking for a match to the type of src.element, and [type]

* type is determined by the type of tgt.element. If tgt.element can have more than one type, then the type is inferred by looking through the available groups labeled "for type+types" for a match to the type of src.element

7.6.0.9 Formal Grammar

todo

7.6.0.10 Mapping Support API

todo

7.6.1 Tutorial

URL: https://hl7.org/fhir/STU3/mapping-tutorial.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.6.1 FHIR Mapping Language - Tutorial

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

This tutorial introduces the FHIR mapping language.

7.6.1.1 Step #1: Simplest possible transform

To start with, we're going to consider a very simple case: mapping between two structures that have the same definition, a single element with the same name and the same primitive type:

Source Structure | Target Structure

TLeft a : string [0..1] | TRight a : string [0..1]

The left instance is transformed to the right instance by copying a to a

Note that for clarity in this tutorial, all the types are prefixed with T.

The first task to do is to set up the mapping context on a default group. All mappings are divided up into a set of groups. For now, we just set up a group named "tutorial" - the same as the name of the mapping. For this tutorial, we also declaring the source and target models, and specify that an application invokes this with a copy of the left (source) instance, and also an empty copy of the right (target) instance:

```
map "http://hl7.org/fhir/StructureMap/tutorial" = tutorial

uses "http://hl7.org/fhir/StructureDefinition/tutorial-left" as source
uses "http://hl7.org/fhir/StructureDefinition/tutorial-right" as target

group tutorial
  input "source" : TLeft as source
  input "target" : TRight as target

// rules go here

endgroup
```

Note that the way the input variables are set up is a choice: we choose to provide the underlying type definitions on which both source and target models are based, and we choose to specify that the invoking application most provide both the source and the target instance trees. Other options are possible; these are discussed further below. The rest of the tutorial examples use the same setup for the group.

Having set up the context, we now need to define the relationships between the source and target structures:

```
"rule_a" : for source.a as a make target.a = a
```

This simple statement says that:

* for every source (there'll only be one)

* for any element 'a' in the source

* if there isn't any element 'a', then don't do anything

* if there is one, call it variable 'a'

* the value of property 'a' of the target will be a copy of variable a - that is, source.a

"rule_a" is a purely arbitrary name associated with the rule that appears in logs, error messages, trace files, etc. It has no other meaning in the mapping statements.

Note that there is no types explicitly in this mapping statement, but if the underlying system has types, then the types will have to be correct. If the underlying source and target trees are strongly typed, and the mapping groups have explicit types, then a short hand form is possible:

```
"rule_a" : for source.a make target.a
```

How this works is described described below.

7.6.1.2 Step #2: Fields with different names

Now consider the case where the elements have different names:

Source Structure | Target Structure
TLeft a1 : string [0..1] | TRight a2 : string [0..1]
The left instance is transformed to the right instance by copying a1 to a2

This relationship is a simple variation of the last:

```
"rule_a1" : for source.a1 as b make target.a2 = b
```

Note that the choice of variable name is purely arbitrary. It does not need to be the same as the element name.

7.6.1.3 Step #3: Length restriction

Still sticking with very simple mappings, let's consider the case where there is a length restriction on the target model that is shorter than the one on the source model - in this case, 20 characters.

Source Structure | Target Structure
TLeft a2 : string [0..1] | TRight a2 : string [0..1] {maxlength = 20}
The left instance is transformed to the right instance by copying a2 to a2, but target.a2 can only be 20 characters long

There are 3 different ways to express this mapping, depending on what should happen when the length of source.a is > 20 characters:

```
"rule_a20a" : for source.a2 as a make target.a2 = truncate(a, 20) // just cut it off at 20 characters
"rule_a20b" : for source.a2 as a where a2.length <= 20 make target.a2 = a // ignore it
"rule_a20c" : for source.a2 as a check a2.length <= 20 make target.a2 = a // error if it's longer than 20 characters
```

Note that it is implicit here that the transformation engine is not required to expected to validate the output against that underlying structure definitions that may apply to it. An application may - and usually should - validate the outputs after the transforms, but the transform engine itself does not automatically validate the output (e.g. it does not assume that it's the final step in the process).

7.6.1.4 Step #4: Type Conversion in Primitives

Now for the case where there is a simple type conversion between the primitive types on the left and right, in this case from a string to an integer.

Source Structure | Target Structure

TLeft a21 : string [0..1] | TRight a21 : integer [0..1]
The left instance is transformed to the right instance by copying a21 to a21, but a21 is converted to an integer

There are 3 different ways to express this mapping, depending on what should happen when a is not an integer:

```
"rule_a21a" : for source.a21 as a make target.a21 = cast(a, "integer") // error if it's not an integer
"rule_a21b" : for source.a21 as a where a.isInteger make target.a21 = cast(a, "integer") // ignore it
"rule_a21c" : for source.a21 as a where not at1.isInteger make target.a21 = 0 // just assign it 0
```

More than one of these mapping rules may be present to handle all possible cases - .e.g rule_a21b combined with rule_a21c.

Note that the mapping language does not itself define which primitive types exist. Typically, primitive types are defined by the underlying type system for the source and target trees, and the implementation layer makes these types available to the mapping language using the FHIRPath primitive types. The mapping language uses the FHIRPath syntax for primitive constants.

7.6.1.5 Step #5: Managing lists, part 1

Back to the simple case where source.a22 is copied to target.a22, but in this case, a22 can repeat (in both source and target):

Source Structure | Target Structure
TLeft a22 : string [0..*] | TRight a22 : string [0..*]
The left instance is transformed to the right instance by copying a22 to a22, once for each copy of a22

The transform rule simply asserts that a22 maps to a22. The engine will apply the rule once for each instance of a22:

```
"rule_a22" : for source.a22 as a make target.a22 = a
```

This will create one a22 in TRight for each a22 in TLeft.

7.6.1.6 Step #6: Managing lists, part 2

A more difficult case is where the source allows multiple repeats, but the target doesn't:

Source Structure | Target Structure
TLeft a23 : string [0..*] | TRight a23 : integer [0..1]
The left instance is transformed to the right instance by copying a23 to a23, but there can only be one copy of a23

Again, there are multiple different ways to write this, depending on out desired outcome if there is more than one copy of a23:

```
rule_a23a : for source.a23 as a make target.a23 = a  // leave it to the transform engine
rule_a23a : for source.a23 only_one as a make target.a23 = a  // transform engine throws an error if there is more than
    one
rule_a23b : for source.a23 first as a make target.a23 = a  // Only use the first one
rule_a23b : for source.a23 last as a make target.a23 = a  // Only use the last one
```

Leaving the outcome to the transform engine is not recommended; it may not always know whether a property is confined to a single value, and exactly what happens is unpredictable. However there are some circumstances where the appropriate action is to defer resolution, so this is allowed.

7.6.1.7 Step #7: Simple Nesting

Most transformations involve nested content. Let's start with a simple case, where element aa contains ab:

Source Structure | Target Structure
TLeft aa : [0..*] ab : string [1..1] | TRight aa : [0..*] ab : string [1..1]
The left instance is transformed to the right instance by copying aa to aa, and within aa, ab to ab

Note that there is no specified type for the element aa. Some structure definitions (FHIR resources) do leave these elements as anonymously typed, while others explicitly type them. However since the mapping does not refer to the type, it's literal type is not important.

```
rule_aa : for source.aa as s_aa make target.aa as t_aa then { // make aa exist
  rule_ab : for s_aa.ab as ab make t_aa.ab = ab // copy ab inside aa
}
```

This situation is handled by a pair of rules: the first rule establishes that relationship between source.aa and target.aa, and assigns 2 variable names to them. Then, the rule contains an additional set of rules (though only one in this example) to map with the context of s_aa and t_aa.

An alternate approach is to move the dependent rules to their own group:

```
rule_aa : for source.aa as s_aa make target.aa as t_aa then ab_content{s_aa, t_aa) // make aa exist

group ab_content
  input src as source
  input tgt as target

  rule_ab : for src.ab as ab make tgt.ab = ab // copy ab inside aa
}
```

Note that variables are divided into source and target; source variables are read-only, and cannot have their properties changed. Variable names may be reused in different contexts - they are only valid within the group or rule that defines them, and any dependent rules or groups.

7.6.1.8 Step #8: Translation

A common translation pattern is to perform a translation e.g. from one set of codes to another

Source Structure | Target Structure
TLeft d : code [0..1] | TRight d : code [0..1]

The left instance is transformed to the right instance by translating source.d from one set of codes to another

The key to this transformation is the ConceptMap resource, which actually specifies the mapping from one set of codes to the other:

```
rule_d : for source.d as d make target.d = translate(d, 'uri-of-concept-map', 'code')
```

This asks the mapping engine to use the $translate operation on the terminology server to translate the code using a specified concept map, and then to put the code value of the return translation in target.d.

7.6.1.9 Step #9: Co-dependency in translation

Another common translation is where the target mapping for one element depends on the value of another element.

Source Structure | Target Structure
TLeft i : string [0..1] m : integer [1..1] | TRight j : [0..1] k : [0..1]
How the left instance is transformed to the right instance depends on the value of m: if m < 2, then i maps to j, else it maps to k

This is managed using FHIRPath conditions on the mapping statements:

```
rule_i1 : for source.i as i where m < 2 make target.j = i
rule_i2 : for source.i as i where m >= 2 make target.k = i
```

7.6.1.10 Step #10: Using Types

Many/most trees are fully and strongly typed. In these cases, the mapping language can make use of the typing system to simply the mapping statements.

Source Structure | Target Structure
TLeft aa : TLeftInner [0..*] TLeftInner ab : string [1..1] | TRight aa : : TRightInner [0..*] TRightInner ab : string [1..1]
The left instance is transformed to the right instance by copying aa to aa, and within aa, ab to ab

This is the same case as Step 7 above, but the mapping statements take advantage of the types:

```
rule_aa : for source.aa make target.aa

group for types ab_content
  input src : TLeftInner as source
  input tgt : TRightInner as target

  rule_ab : for src.ab make tgt.ab
}
```

There are 2 different things happening in this short form:

* group for types - the "for types" indicates that this group is the default group to apply any time an element of type TLeftInner is mapped to a TRightInner

* Both the rules take advantage of the fact that the types of both source and target are known, and compatible, and instruct the mapping execution engine to make the target appropriately

In the case of the first rule (rule_aa), the engine finds a need to map aa to aa, and determines that it must map from TLeftInner to a TRightInner. Since a group is defined for this purpose, it creates a TRightInner in target.aa, and then applies the discovered rule as a dependency rule. Inside that rule that instructs the mapping engine to make tgt.ab from src.ab. It knows that both are primitive types, and compatible, and can apply this correctly. This short form is only applicable when there is only one source and target, when the types of both are known, and when no other dependency rules are nominated.

If the target element is polymorphic (can have more than one type), then the correct type of the target can only be inferred from the source type:

```
group for type+types ab_content
  input src : TLeftInner as source
  input tgt : TRightInner as target

  rule_ab : for src.ab make tgt.ab
}
```

Not only is this group the default for (TLeftInner:TRightInner), if the engine has a TLeftInner with an unknown target type, it should create a TRightInner, and proceed as above.

It is an error if the engine locates more than one group of rules claiming to be the correct group for a type pair of a single source type.

7.6.1.11 Step #11: Reworking Structure #1

It's now time to start maving away from relatively simple cases to some of the harder ones to manage mappings for. The first mixes list management, and converting from a specific structure to a general structure:

Source Structure | Target Structure
TLeft e : string [0..*] f : string [1..1] | TRight e : [0..*] f : string [1..1] g : code [1..1]
The left instance is transformed to the right instance by adding one instance of target.e for each source.e, where the value goes into target.e.f, and the value of target.e.g is 'g1'. source.f is also transformed into the same structure, but the value of target.e.g is 'g2'. As an added complication, the value for source.f must come first

This leads to some more complex mapping statements:

```
ef_a1: for source.e as s_e make target.e as t_e then {
  ef_a2: for s_e make t_e.f = s_e, t_e.g = "g1"
}

ef_b1: for source.f as s_f make target.e as t_e { first } then {
```

```
  ef_b2: for s_f make t_e.f = s_f, t_e.g = "g2"
}
```

7.6.1.12 Step #12: Reworking Structure #2

The second example for reworking structure moves cardinality around the hierarchy. in this case, the source has an optional structure that contains a repeating structure, while the target puts the cardinality at the next level up:

Source Structure | Target Structure

TLeft az1 :[0..1] az2 : string [1..1] az3 : string [0..*] | TRight az1 :[0..*] az2 : string [1..1] az3 : string [0..1]

The left instance is transformed to the right instance creating on target.az1 for every source.az1.az3, and then populating each az1 with the matching value of az3, and copying the value of az2 to each instance

The key to setting this mapping up is to create a variable context for source.az1, and then carry it down, performing the actual mappings at the next level down:

```
// setting up a variable for the parent
aza : for src.az1 as s_az1 then {

  // one target.az1 for each az3
  azb : for s_az1.az3 as s_az3 make target.az1 as t_az1 then {
    // value for az2. Note that this refers to a previous context in the source
    az2 : for s_az1.az2 as az2 make t_az1.az2 = az2

    // value for az3
    az3 : for s_az3 make tgt_az1.az3 = src_az3
  }
}
```

7.6.1.13 Step #13: Multiple structures #1

Simple mappings, such as we've dealt with so far, where the source and target structure both have the same scope, and there is only one of each, are all well and good, but there are many mappings where this is not the case. There is a set of complications when dealing with multiple instances:

* If there are multiple source inputs, how does the application know what they are? Sometimes, they are just independent inputs, but more often, the inputs are dependent on references in the source input, and therefore which source inputs are required depends on the mapping rules

* If there are multiple output instances, how are they identified as they are created, and how do the target models reference each other? Mostly, the answer is that it depends on the context; the actual identification details are not part of the mapping

* It may even be the case that the kind of output structure to produce depends on the mapping rules, so the application can't create the target structure before invoking the map

For our first example, we're going to look at creating multiple output structures from a single input structure.

Source Structure | Target Structure

TLeft f1 : String [0..*]; | TRight ptr : Resource(TRight2) [0..*] TRight2 f1 : String [1..1];

The left instance is transformed to the right instance creating a copy of TRight2 for each f1 in the source, and then putting the value of source.f1 in TRight2.f1

The key to setting this mapping up is to create a variable context for source.az1, and then carry it down, performing the actual mappings at the next level down:

```
f1 : for source.f1 as s_f1 make create("TRight2") as rr, target.ptr = reference(rr) then {
  f1a: for s_f1 make rr.f2 = srcff
}
```

This mapping statement makes use a special known value "null" for the target context to indicate that the created element/object of type "TRight2" doesn't get added to any existing target context. Instead, it will only be available as a context in which to perform further mappings - as rule f1a does.

The mapping engine passes the create request through to the host application, which is using the mapping. It must create a valid instance of TRight, and identify it as appropriate for the technical context in which the mapping is being used. The reference transform is also passed back to the host application for it to determine how to represent the reference - but this is usually some kind of URL.

7.6.1.14 Step #14: Multiple structures #2

For our second example, we're going to look at the reverse: where multiple input structures create a single input structure.

Source Structure | Target Structure

TLeft ptr : Resource(TLeft2) [0..*] TLeft2 f2 : String [0..*]; | TRight f2 : String [1..*];

The left instance is transformed to the right instance finding each ptr reference, getting it's value for f1, and adding this in target.f2

The first task of the map is to ask the application host to find the structure identified by source.ptr, and create a variable for it

```
f2: [todo]
```

7.6.1.15 Step #15: Grouping

```
f2: [todo]
```

7.7 Resource StructureMap - Content

URL: https://hl7.org/fhir/STU3/structuremap.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

FHIR Infrastructure Work Group | Maturity Level: 2 | Trial Use | Compartments: Not linked to any defined compartments

A Map of relationships between 2 structures that can be used to transform data.

7.7.1 Scope and Usage

The StructureMap resource defines a detailed set of rules that describe how one Structure is related to another, and provides sufficient detail to allow for automated conversion of instances.

The intention of the structure map resource is to allow a specialist in formats and interoperability to specify the full relationships between two structures (e.g. a CDA document and a set of FHIR resources), and then many different systems - both testing and production clinical systems - can leverage that to automatically transform from one format to the other.

Maps are uni-directional: they map from the source structure to the target structure, and no reverse map is implied. Even if the mapping is simple, and loss-less, it cannot be assumed that there are no conditions that might additionally apply in the reverse direction.

The mapping language, along with a concrete syntax, is defined in detail in the FHIR Mapping Language. The StructureMap resource represents the abstract syntax, and the concrete syntax is the recommended narrative representation for a StructureMap. See also the Tutorial.

7.7.2 Boundaries and Relationships

Note that many mappings between models only establish conceptual equivalence between the structures. These models are useful because they quickly convey how the structures are related to humans, whereas more maps with sufficient detail to support instance transformation are necessarily full of fine detail that can obscure the conceptual relationships. The ConceptMap resource is suitable for representing high level relationships between models, while this StructureMap resource is intended to describe the full details that need to be known in order to transform an instance of data from one structure to another.

The StructureMap resource assumes that both the source and the target models are fully defined using StructureDefinition resources - either resources, or logical models, and is described accordingly. However, there is no direct relationship between the mapping language contained in the StructureMap resource, and the existence of the appropriate structure definitions, so that this mapping language could be used to define a map from an HL7 v2 message to a CDA document. Note, that various implementation contexts may introduce a direct relationship (e.g. see xxxxxxx op to defined).

It's possible to apply the mapping language to structures that do not even have (or cannot have) formally defined types, although the type-related parts of the mapping language cannot be used in these cases.

7.7.3 Background and Context

Each structure map contains, in addition to the standard metadata that all conformance resources contain, the following information:

* A list of the structure definitions referenced by the map

* A list of other structure maps that the map uses

* One or more groups of rules that describe how content in the source is transformed to content in the target

Each group of rules defines a set of input and output variables that must be passed when the group is invoked in a particular context. When a group is invoked, all the rules in the group are checked to see whether they apply.

Each rule may have some or all of the following properties:

* A name - used as the identity in internal references, and traces

* Contexts in both source and target models that define where the rule applies

* A set of source elements that provide data to be mapped

* Conditions that specify if the rule is to apply

* A set of target elements that will be created

* Transform rules that describe how raw data is converted from the source format to the target format (e.g. string manipulation)

* Flags for how instances that can repeat are handled

* Additional rules that apply to the newly created elements (e.g. new contexts)

The mapping language is entirely declarative; there is no imperative or procedural aspects to the definitions.

This resource is referenced by ActivityDefinition and PlanDefinition

7.7.4 Resource Content

* Structure

* UML

* XML

* JSON
* Turtle
* R2 Diff
* All

Structure

Name | Flags | Card. | Type | Description & Constraints
StructureMap | | | DomainResource | A Map of relationships between 2 structures that can be used to transform data Elements defined in Ancestors: id, meta, implicitRules, language, text, contained, extension, modifierExtension
url | ? | 1..1 | uri | Logical URI to reference this structure map (globally unique)
identifier | ? | 0..* | Identifier | Additional identifier for the structure map
version | ? | 0..1 | string | Business version of the structure map
name | ? | 1..1 | string | Name for this structure map (computer friendly)
title | ? | 0..1 | string | Name for this structure map (human friendly)
status | ?!? | 1..1 | code | draft | active | retired | unknown PublicationStatus (Required)
experimental | ?!? | 0..1 | boolean | For testing purposes, not real usage
date | ? | 0..1 | dateTime | Date this was last changed
publisher | ? | 0..1 | string | Name of the publisher (organization or individual)
contact | ? | 0..* | ContactDetail | Contact details for the publisher
description | | 0..1 | markdown | Natural language description of the structure map
useContext | ? | 0..* | UsageContext | Context the content is intended to support
jurisdiction | ? | 0..* | CodeableConcept | Intended jurisdiction for structure map (if applicable) Jurisdiction ValueSet (Extensible)
purpose | | 0..1 | markdown | Why this structure map is defined
copyright | | 0..1 | markdown | Use and/or publishing restrictions
structure | ? | 0..* | BackboneElement | Structure Definition used by this map
url | ? | 1..1 | uri | Canonical URL for structure definition
mode | ? | 1..1 | code | source | queried | target | produced StructureMapModelMode (Required)
alias | ? | 0..1 | string | Name for type in this map
documentation | | 0..1 | string | Documentation on use of structure
import | ? | 0..* | uri | Other maps used by this map (canonical URLs)
group | ? | 1..* | BackboneElement | Named sections for reader convenience
name | ? | 1..1 | id | Human-readable label
extends | ? | 0..1 | id | Another group that this group adds rules to
typeMode | ? | 1..1 | code | none | types | type-and-types StructureMapGroupTypeMode (Required)
documentation | ? | 0..1 | string | Additional description/explaination for group
input | ? | 1..* | BackboneElement | Named instance provided when invoking the map
name | ? | 1..1 | id | Name for this instance of data
type | ? | 0..1 | string | Type for this instance of data
mode | ? | 1..1 | code | source | target StructureMapInputMode (Required)
documentation | | 0..1 | string | Documentation for this instance of data
rule | ? | 1..* | BackboneElement | Transform Rule from source to target
name | ? | 1..1 | id | Name of the rule for internal references
source | ? | 1..* | BackboneElement | Source inputs to the mapping
context | ? | 1..1 | id | Type or variable this rule applies to
min | ? | 0..1 | integer | Specified minimum cardinality
max | ? | 0..1 | string | Specified maximum cardinality (number or *)
type | ? | 0..1 | string | Rule only applies if source has this type
defaultValue[x] | ? | 0..1 | * | Default value if no value exists
element | ? | 0..1 | string | Optional field for this source
listMode | ? | 0..1 | code | first | not_first | last | not_last | only_one StructureMapSourceListMode (Required)
variable | ? | 0..1 | id | Named context for field, if a field is specified
condition | ? | 0..1 | string | FHIRPath expression - must be true or the rule does not apply
check | ? | 0..1 | string | FHIRPath expression - must be true or the mapping engine throws an error instead of completing
target | ?I | 0..* | BackboneElement | Content to create because of this mapping rule + Must have a contextType if you have a context + Can only have a element if you have a context
context | ? | 0..1 | id | Type or variable this rule applies to
contextType | ? | 0..1 | code | type | variable StructureMapContextType (Required)
element | ? | 0..1 | string | Field to create in the context
variable | ? | 0..1 | id | Named context for field, if desired, and a field is specified
listMode | ? | 0..* | code | first | share | last | collate StructureMapTargetListMode (Required)
listRuleId | ? | 0..1 | id | Internal rule reference for shared list items
transform | ? | 0..1 | code | create | copy + StructureMapTransform (Required)
parameter | ? | 0..* | BackboneElement | Parameters to the transform
value[x] | ? | 1..1 | | Parameter value - variable or literal
valueId | | | id
valueString | | | string
valueBoolean | | | boolean
valueInteger | | | integer
valueDecimal | | | decimal
rule | ? | 0..* | see rule | Rules contained in this rule
dependent | ? | 0..* | BackboneElement | Which other rules to apply in the context of this rule

name | ? | 1..1 | id | Name of a rule or group to apply
variable | ? | 1..* | string | Variable to pass to the rule or group
documentation | | 0..1 | string | Documentation for this instance of data
Documentation for this format

UML Diagram (Legend)

XML Template

```
<StructureMap xmlns="http://hl7.org/fhir">
 <!-- from Resource: id, meta, implicitRules, and language -->
 <!-- from DomainResource: text, contained, extension, and modifierExtension -->
 <url value="[uri]"/><!-- 1..1 Logical URI to reference this structure map (globally unique) -->
 <identifier><!-- 0..* Identifier Additional identifier for the structure map --></identifier>
 <version value="[string]"/><!-- 0..1 Business version of the structure map -->
 <name value="[string]"/><!-- 1..1 Name for this structure map (computer friendly) -->
 <title value="[string]"/><!-- 0..1 Name for this structure map (human friendly) -->
 <status value="[code]"/><!-- 1..1 draft | active | retired | unknown -->
 <experimental value="[boolean]"/><!-- 0..1 For testing purposes, not real usage -->
 <date value="[dateTime]"/><!-- 0..1 Date this was last changed -->
 <publisher value="[string]"/><!-- 0..1 Name of the publisher (organization or individual) -->
 <contact><!-- 0..* ContactDetail Contact details for the publisher --></contact>
 <description value="[markdown]"/><!-- 0..1 Natural language description of the structure map -->
 <useContext><!-- 0..* UsageContext Context the content is intended to support --></useContext>
 <jurisdiction><!-- 0..* CodeableConcept Intended jurisdiction for structure map (if applicable) --></jurisdiction>
 <purpose value="[markdown]"/><!-- 0..1 Why this structure map is defined -->
 <copyright value="[markdown]"/><!-- 0..1 Use and/or publishing restrictions -->
 <structure>  <!-- 0..* Structure Definition used by this map -->
  <url value="[uri]"/><!-- 1..1 Canonical URL for structure definition -->
  <mode value="[code]"/><!-- 1..1 source | queried | target | produced -->
  <alias value="[string]"/><!-- 0..1 Name for type in this map -->
  <documentation value="[string]"/><!-- 0..1 Documentation on use of structure -->
 </structure>
 <import value="[uri]"/><!-- 0..* Other maps used by this map (canonical URLs) -->
 <group>  <!-- 1..* Named sections for reader convenience -->
  <name value="[id]"/><!-- 1..1 Human-readable label -->
  <extends value="[id]"/><!-- 0..1 Another group that this group adds rules to -->
  <typeMode value="[code]"/><!-- 1..1 none | types | type-and-types -->
  <documentation value="[string]"/><!-- 0..1 Additional description/explaination for group -->
  <input>  <!-- 1..* Named instance provided when invoking the map -->
   <name value="[id]"/><!-- 1..1 Name for this instance of data -->
   <type value="[string]"/><!-- 0..1 Type for this instance of data -->
   <mode value="[code]"/><!-- 1..1 source | target -->
   <documentation value="[string]"/><!-- 0..1 Documentation for this instance of data -->
  </input>
  <rule>  <!-- 1..* Transform Rule from source to target -->
   <name value="[id]"/><!-- 1..1 Name of the rule for internal references -->
   <source>  <!-- 1..* Source inputs to the mapping -->
    <context value="[id]"/><!-- 1..1 Type or variable this rule applies to -->
    <min value="[integer]"/><!-- 0..1 Specified minimum cardinality -->
    <max value="[string]"/><!-- 0..1 Specified maximum cardinality (number or *) -->
    <type value="[string]"/><!-- 0..1 Rule only applies if source has this type -->
    <defaultValue[x]><!-- 0..1 * Default value if no value exists --></defaultValue[x]>
    <element value="[string]"/><!-- 0..1 Optional field for this source -->
    <listMode value="[code]"/><!-- 0..1 first | not_first | last | not_last | only_one -->
    <variable value="[id]"/><!-- 0..1 Named context for field, if a field is specified -->
    <condition value="[string]"/><!-- 0..1 FHIRPath expression  - must be true or the rule does not apply -->
    <check value="[string]"/><!-- 0..1 FHIRPath expression  - must be true or the mapping engine throws an error instead
     of completing -->
   </source>
   <target>  <!-- 0..* Content to create because of this mapping rule -->
    <context value="[id]"/><!-- 0..1 Type or variable this rule applies to -->
    <contextType value="[code]"/><!-- 0..1 type | variable -->
    <element value="[string]"/><!-- 0..1 Field to create in the context -->
    <variable value="[id]"/><!-- 0..1 Named context for field, if desired, and a field is specified -->
    <listMode value="[code]"/><!-- 0..* first | share | last | collate -->
    <listRuleId value="[id]"/><!-- 0..1 Internal rule reference for shared list items -->
    <transform value="[code]"/><!-- 0..1 create | copy + -->
    <parameter>  <!-- 0..* Parameters to the transform -->
     <value[x]><!-- 1..1 id|string|boolean|integer|decimal Parameter value - variable or literal --></value[x]>
    </parameter>
   </target>
   <rule><!-- 0..* Content as for StructureMap.group.rule Rules contained in this rule --></rule>
   <dependent>  <!-- 0..* Which other rules to apply in the context of this rule -->
    <name value="[id]"/><!-- 1..1 Name of a rule or group to apply -->
    <variable value="[string]"/><!-- 1..* Variable to pass to the rule or group -->
   </dependent>
   <documentation value="[string]"/><!-- 0..1 Documentation for this instance of data -->
  </rule>
 </group>
</StructureMap>
```

JSON Template

```
{
  "resourceType" : "StructureMap",
  // from Resource: id, meta, implicitRules, and language
  // from DomainResource: text, contained, extension, and modifierExtension
  "url" : "<uri>", // R!  Logical URI to reference this structure map (globally unique)
  "identifier" : [{ Identifier }], // Additional identifier for the structure map
```

```
"version" : "<string>", // Business version of the structure map
"name" : "<string>", // R!  Name for this structure map (computer friendly)
"title" : "<string>", // Name for this structure map (human friendly)
"status" : "<code>", // R!  draft | active | retired | unknown
"experimental" : <boolean>, // For testing purposes, not real usage
"date" : "<dateTime>", // Date this was last changed
"publisher" : "<string>", // Name of the publisher (organization or individual)
"contact" : [{ ContactDetail }], // Contact details for the publisher
"description" : "<markdown>", // Natural language description of the structure map
"useContext" : [{ UsageContext }], // Context the content is intended to support
"jurisdiction" : [{ CodeableConcept }], // Intended jurisdiction for structure map (if applicable)
"purpose" : "<markdown>", // Why this structure map is defined
"copyright" : "<markdown>", // Use and/or publishing restrictions
"structure" : [{ // Structure Definition used by this map
  "url" : "<uri>", // R!  Canonical URL for structure definition
  "mode" : "<code>", // R!  source | queried | target | produced
  "alias" : "<string>", // Name for type in this map
  "documentation" : "<string>" // Documentation on use of structure
}],
"import" : ["<uri>"], // Other maps used by this map (canonical URLs)
"group" : [{ // R!  Named sections for reader convenience
  "name" : "<id>", // R!  Human-readable label
  "extends" : "<id>", // Another group that this group adds rules to
  "typeMode" : "<code>", // R!  none | types | type-and-types
  "documentation" : "<string>", // Additional description/explaination for group
  "input" : [{ // R!  Named instance provided when invoking the map
    "name" : "<id>", // R!  Name for this instance of data
    "type" : "<string>", // Type for this instance of data
    "mode" : "<code>", // R!  source | target
    "documentation" : "<string>" // Documentation for this instance of data
  }],
  "rule" : [{ // R!  Transform Rule from source to target
    "name" : "<id>", // R!  Name of the rule for internal references
    "source" : [{ // R!  Source inputs to the mapping
      "context" : "<id>", // R!  Type or variable this rule applies to
      "min" : <integer>, // Specified minimum cardinality
      "max" : "<string>", // Specified maximum cardinality (number or *)
      "type" : "<string>", // Rule only applies if source has this type
      // defaultValue[x]: Default value if no value exists. One of these 23:
      "defaultValueInteger" : <integer>,
      "defaultValueDecimal" : <decimal>,
      "defaultValueDateTime" : "<dateTime>",
      "defaultValueDate" : "<date>",
      "defaultValueInstant" : "<instant>",
      "defaultValueString" : "<string>",
      "defaultValueUri" : "<uri>",
      "defaultValueBoolean" : <boolean>,
      "defaultValueCode" : "<code>",
      "defaultValueBase64Binary" : "<base64Binary>",
      "defaultValueCoding" : { Coding },
      "defaultValueCodeableConcept" : { CodeableConcept },
      "defaultValueAttachment" : { Attachment },
      "defaultValueIdentifier" : { Identifier },
      "defaultValueQuantity" : { Quantity },
      "defaultValueRange" : { Range },
      "defaultValuePeriod" : { Period },
      "defaultValueRatio" : { Ratio },
      "defaultValueHumanName" : { HumanName },
      "defaultValueAddress" : { Address },
      "defaultValueContactPoint" : { ContactPoint },
      "defaultValueSchedule" : { Schedule },
      "defaultValueReference" : { Reference },
      "element" : "<string>", // Optional field for this source
      "listMode" : "<code>", // first | not_first | last | not_last | only_one
      "variable" : "<id>", // Named context for field, if a field is specified
      "condition" : "<string>", // FHIRPath expression  - must be true or the rule does not apply
      "check" : "<string>" // FHIRPath expression  - must be true or the mapping engine throws an error instead of
  completing
    }],
    "target" : [{ // Content to create because of this mapping rule
      "context" : "<id>", // Type or variable this rule applies to
      "contextType" : "<code>", // type | variable
      "element" : "<string>", // Field to create in the context
      "variable" : "<id>", // Named context for field, if desired, and a field is specified
      "listMode" : ["<code>"], // first | share | last | collate
      "listRuleId" : "<id>", // Internal rule reference for shared list items
      "transform" : "<code>", // create | copy +
      "parameter" : [{ // Parameters to the transform
        // value[x]: Parameter value - variable or literal. One of these 5:
        "valueId" : "<id>"
        "valueString" : "<string>"
        "valueBoolean" : <boolean>
        "valueInteger" : <integer>
        "valueDecimal" : <decimal>
      }]
    }],
    "rule" : [{ Content as for StructureMap.group.rule }], // Rules contained in this rule
    "dependent" : [{ // Which other rules to apply in the context of this rule
      "name" : "<id>", // R!  Name of a rule or group to apply
```

```
        "variable" : ["<string>"] // R!  Variable to pass to the rule or group
      }],
      "documentation" : "<string>" // Documentation for this instance of data
    }]
  }]
}
```

Turtle Template

```
@prefix fhir: <http://hl7.org/fhir/> .


[ a fhir:StructureMap;
  fhir:nodeRole fhir:treeRoot; # if this is the parser root

  # from Resource: .id, .meta, .implicitRules, and .language
  # from DomainResource: .text, .contained, .extension, and .modifierExtension
  fhir:StructureMap.url [ uri ]; # 1..1 Logical URI to reference this structure map (globally unique)
  fhir:StructureMap.identifier [ Identifier ], ... ; # 0..* Additional identifier for the structure map
  fhir:StructureMap.version [ string ]; # 0..1 Business version of the structure map
  fhir:StructureMap.name [ string ]; # 1..1 Name for this structure map (computer friendly)
  fhir:StructureMap.title [ string ]; # 0..1 Name for this structure map (human friendly)
  fhir:StructureMap.status [ code ]; # 1..1 draft | active | retired | unknown
  fhir:StructureMap.experimental [ boolean ]; # 0..1 For testing purposes, not real usage
  fhir:StructureMap.date [ dateTime ]; # 0..1 Date this was last changed
  fhir:StructureMap.publisher [ string ]; # 0..1 Name of the publisher (organization or individual)
  fhir:StructureMap.contact [ ContactDetail ], ... ; # 0..* Contact details for the publisher
  fhir:StructureMap.description [ markdown ]; # 0..1 Natural language description of the structure map
  fhir:StructureMap.useContext [ UsageContext ], ... ; # 0..* Context the content is intended to support
  fhir:StructureMap.jurisdiction [ CodeableConcept ], ... ; # 0..* Intended jurisdiction for structure map (if
    applicable)
  fhir:StructureMap.purpose [ markdown ]; # 0..1 Why this structure map is defined
  fhir:StructureMap.copyright [ markdown ]; # 0..1 Use and/or publishing restrictions
  fhir:StructureMap.structure [ # 0..* Structure Definition used by this map
    fhir:StructureMap.structure.url [ uri ]; # 1..1 Canonical URL for structure definition
    fhir:StructureMap.structure.mode [ code ]; # 1..1 source | queried | target | produced
    fhir:StructureMap.structure.alias [ string ]; # 0..1 Name for type in this map
    fhir:StructureMap.structure.documentation [ string ]; # 0..1 Documentation on use of structure
  ], ...;
  fhir:StructureMap.import [ uri ], ... ; # 0..* Other maps used by this map (canonical URLs)
  fhir:StructureMap.group [ # 1..* Named sections for reader convenience
    fhir:StructureMap.group.name [ id ]; # 1..1 Human-readable label
    fhir:StructureMap.group.extends [ id ]; # 0..1 Another group that this group adds rules to
    fhir:StructureMap.group.typeMode [ code ]; # 1..1 none | types | type-and-types
    fhir:StructureMap.group.documentation [ string ]; # 0..1 Additional description/explaination for group
    fhir:StructureMap.group.input [ # 1..* Named instance provided when invoking the map
      fhir:StructureMap.group.input.name [ id ]; # 1..1 Name for this instance of data
      fhir:StructureMap.group.input.type [ string ]; # 0..1 Type for this instance of data
      fhir:StructureMap.group.input.mode [ code ]; # 1..1 source | target
      fhir:StructureMap.group.input.documentation [ string ]; # 0..1 Documentation for this instance of data
    ], ...;
    fhir:StructureMap.group.rule [ # 1..* Transform Rule from source to target
      fhir:StructureMap.group.rule.name [ id ]; # 1..1 Name of the rule for internal references
      fhir:StructureMap.group.rule.source [ # 1..* Source inputs to the mapping
        fhir:StructureMap.group.rule.source.context [ id ]; # 1..1 Type or variable this rule applies to
        fhir:StructureMap.group.rule.source.min [ integer ]; # 0..1 Specified minimum cardinality
        fhir:StructureMap.group.rule.source.max [ string ]; # 0..1 Specified maximum cardinality (number or *)
        fhir:StructureMap.group.rule.source.type [ string ]; # 0..1 Rule only applies if source has this type
        # StructureMap.group.rule.source.defaultValue[x] : 0..1 Default value if no value exists. One of these 38
          fhir:StructureMap.group.rule.source.defaultValueBase64Binary [ base64Binary ]
          fhir:StructureMap.group.rule.source.defaultValueBoolean [ boolean ]
          fhir:StructureMap.group.rule.source.defaultValueCode [ code ]
          fhir:StructureMap.group.rule.source.defaultValueDate [ date ]
          fhir:StructureMap.group.rule.source.defaultValueDateTime [ dateTime ]
          fhir:StructureMap.group.rule.source.defaultValueDecimal [ decimal ]
          fhir:StructureMap.group.rule.source.defaultValueId [ id ]
          fhir:StructureMap.group.rule.source.defaultValueInstant [ instant ]
          fhir:StructureMap.group.rule.source.defaultValueInteger [ integer ]
          fhir:StructureMap.group.rule.source.defaultValueMarkdown [ markdown ]
          fhir:StructureMap.group.rule.source.defaultValueOid [ oid ]
          fhir:StructureMap.group.rule.source.defaultValuePositiveInt [ positiveInt ]
          fhir:StructureMap.group.rule.source.defaultValueString [ string ]
          fhir:StructureMap.group.rule.source.defaultValueTime [ time ]
          fhir:StructureMap.group.rule.source.defaultValueUnsignedInt [ unsignedInt ]
          fhir:StructureMap.group.rule.source.defaultValueUri [ uri ]
          fhir:StructureMap.group.rule.source.defaultValueAddress [ Address ]
          fhir:StructureMap.group.rule.source.defaultValueAge [ Age ]
          fhir:StructureMap.group.rule.source.defaultValueAnnotation [ Annotation ]
          fhir:StructureMap.group.rule.source.defaultValueAttachment [ Attachment ]
          fhir:StructureMap.group.rule.source.defaultValueCodeableConcept [ CodeableConcept ]
          fhir:StructureMap.group.rule.source.defaultValueCoding [ Coding ]
          fhir:StructureMap.group.rule.source.defaultValueContactPoint [ ContactPoint ]
          fhir:StructureMap.group.rule.source.defaultValueCount [ Count ]
          fhir:StructureMap.group.rule.source.defaultValueDistance [ Distance ]
          fhir:StructureMap.group.rule.source.defaultValueDuration [ Duration ]
          fhir:StructureMap.group.rule.source.defaultValueHumanName [ HumanName ]
          fhir:StructureMap.group.rule.source.defaultValueIdentifier [ Identifier ]
          fhir:StructureMap.group.rule.source.defaultValueMoney [ Money ]
          fhir:StructureMap.group.rule.source.defaultValuePeriod [ Period ]
          fhir:StructureMap.group.rule.source.defaultValueQuantity [ Quantity ]
```

```
        fhir:StructureMap.group.rule.source.defaultValueRange [ Range ]
        fhir:StructureMap.group.rule.source.defaultValueRatio [ Ratio ]
        fhir:StructureMap.group.rule.source.defaultValueReference [ Reference ]
        fhir:StructureMap.group.rule.source.defaultValueSampledData [ SampledData ]
        fhir:StructureMap.group.rule.source.defaultValueSignature [ Signature ]
        fhir:StructureMap.group.rule.source.defaultValueTiming [ Timing ]
        fhir:StructureMap.group.rule.source.defaultValueMeta [ Meta ]
      fhir:StructureMap.group.rule.source.element [ string ]; # 0..1 Optional field for this source
      fhir:StructureMap.group.rule.source.listMode [ code ]; # 0..1 first | not_first | last | not_last | only_one
      fhir:StructureMap.group.rule.source.variable [ id ]; # 0..1 Named context for field, if a field is specified
      fhir:StructureMap.group.rule.source.condition [ string ]; # 0..1 FHIRPath expression  - must be true or the rule
   does not apply
      fhir:StructureMap.group.rule.source.check [ string ]; # 0..1 FHIRPath expression  - must be true or the mapping
  engine throws an error instead of completing
    ], ...;
    fhir:StructureMap.group.rule.target [ # 0..* Content to create because of this mapping rule
      fhir:StructureMap.group.rule.target.context [ id ]; # 0..1 Type or variable this rule applies to
      fhir:StructureMap.group.rule.target.contextType [ code ]; # 0..1 type | variable
      fhir:StructureMap.group.rule.target.element [ string ]; # 0..1 Field to create in the context
      fhir:StructureMap.group.rule.target.variable [ id ]; # 0..1 Named context for field, if desired, and a field is
  specified
      fhir:StructureMap.group.rule.target.listMode [ code ], ... ; # 0..* first | share | last | collate
      fhir:StructureMap.group.rule.target.listRuleId [ id ]; # 0..1 Internal rule reference for shared list items
      fhir:StructureMap.group.rule.target.transform [ code ]; # 0..1 create | copy +
      fhir:StructureMap.group.rule.target.parameter [ # 0..* Parameters to the transform
        # StructureMap.group.rule.target.parameter.value[x] : 1..1 Parameter value - variable or literal. One of these
   5
          fhir:StructureMap.group.rule.target.parameter.valueId [ id ]
          fhir:StructureMap.group.rule.target.parameter.valueString [ string ]
          fhir:StructureMap.group.rule.target.parameter.valueBoolean [ boolean ]
          fhir:StructureMap.group.rule.target.parameter.valueInteger [ integer ]
          fhir:StructureMap.group.rule.target.parameter.valueDecimal [ decimal ]
      ], ...;
    ], ...;
    fhir:StructureMap.group.rule.rule [ See StructureMap.group.rule ], ... ; # 0..* Rules contained in this rule
    fhir:StructureMap.group.rule.dependent [ # 0..* Which other rules to apply in the context of this rule
      fhir:StructureMap.group.rule.dependent.name [ id ]; # 1..1 Name of a rule or group to apply
      fhir:StructureMap.group.rule.dependent.variable [ string ], ... ; # 1..* Variable to pass to the rule or group

    ], ...;
    fhir:StructureMap.group.rule.documentation [ string ]; # 0..1 Documentation for this instance of data
  ], ...;
], ...;
]
```

Changes since DSTU2

This resource did not exist in Release 2

This analysis is available as XML or JSON.

Structure

Name | Flags | Card. | Type | Description & Constraints

StructureMap | | | DomainResource | A Map of relationships between 2 structures that can be used to transform data Elements defined in Ancestors: id, meta, implicitRules, language, text, contained, extension, modifierExtension

url | ? | 1..1 | uri | Logical URI to reference this structure map (globally unique)

identifier | ? | 0..* | Identifier | Additional identifier for the structure map

version | ? | 0..1 | string | Business version of the structure map

name | ? | 1..1 | string | Name for this structure map (computer friendly)

title | ? | 0..1 | string | Name for this structure map (human friendly)

status | ?!? | 1..1 | code | draft | active | retired | unknown PublicationStatus (Required)

experimental | ?!? | 0..1 | boolean | For testing purposes, not real usage

date | ? | 0..1 | dateTime | Date this was last changed

publisher | ? | 0..1 | string | Name of the publisher (organization or individual)

contact | ? | 0..* | ContactDetail | Contact details for the publisher

description | | 0..1 | markdown | Natural language description of the structure map

useContext | ? | 0..* | UsageContext | Context the content is intended to support

jurisdiction | ? | 0..* | CodeableConcept | Intended jurisdiction for structure map (if applicable) Jurisdiction ValueSet (Extensible)

purpose | | 0..1 | markdown | Why this structure map is defined

copyright | | 0..1 | markdown | Use and/or publishing restrictions

structure | ? | 0..* | BackboneElement | Structure Definition used by this map

url | ? | 1..1 | uri | Canonical URL for structure definition

mode | ? | 1..1 | code | source | queried | target | produced StructureMapModelMode (Required)

alias | ? | 0..1 | string | Name for type in this map

documentation | | 0..1 | string | Documentation on use of structure

import | ? | 0..* | uri | Other maps used by this map (canonical URLs)

group | ? | 1..* | BackboneElement | Named sections for reader convenience

name | ? | 1..1 | id | Human-readable label

extends | ? | 0..1 | id | Another group that this group adds rules to

typeMode | ? | 1..1 | code | none | types | type-and-types StructureMapGroupTypeMode (Required)

documentation | ? | 0..1 | string | Additional description/explaination for group

input | ? | 1..* | BackboneElement | Named instance provided when invoking the map

name | ? | 1..1 | id | Name for this instance of data

type | ? | 0..1 | string | Type for this instance of data
mode | ? | 1..1 | code | source | target StructureMapInputMode (Required)
documentation | | 0..1 | string | Documentation for this instance of data
rule | ? | 1..* | BackboneElement | Transform Rule from source to target
name | ? | 1..1 | id | Name of the rule for internal references
source | ? | 1..* | BackboneElement | Source inputs to the mapping
context | ? | 1..1 | id | Type or variable this rule applies to
min | ? | 0..1 | integer | Specified minimum cardinality
max | ? | 0..1 | string | Specified maximum cardinality (number or *)
type | ? | 0..1 | string | Rule only applies if source has this type
defaultValue[x] | ? | 0..1 | * | Default value if no value exists
element | ? | 0..1 | string | Optional field for this source
listMode | ? | 0..1 | code | first | not_first | last | not_last | only_one StructureMapSourceListMode (Required)
variable | ? | 0..1 | id | Named context for field, if a field is specified
condition | ? | 0..1 | string | FHIRPath expression - must be true or the rule does not apply
check | ? | 0..1 | string | FHIRPath expression - must be true or the mapping engine throws an error instead of completing
target | ?I | 0..* | BackboneElement | Content to create because of this mapping rule + Must have a contextType if you have a context + Can only have a element if you have a context
context | ? | 0..1 | id | Type or variable this rule applies to
contextType | ? | 0..1 | code | type | variable StructureMapContextType (Required)
element | ? | 0..1 | string | Field to create in the context
variable | ? | 0..1 | id | Named context for field, if desired, and a field is specified
listMode | ? | 0..* | code | first | share | last | collate StructureMapTargetListMode (Required)
listRuleId | ? | 0..1 | id | Internal rule reference for shared list items
transform | ? | 0..1 | code | create | copy + StructureMapTransform (Required)
parameter | ? | 0..* | BackboneElement | Parameters to the transform
value[x] | ? | 1..1 | | Parameter value - variable or literal
valueId | | | id
valueString | | | string
valueBoolean | | | boolean
valueInteger | | | integer
valueDecimal | | | decimal
rule | ? | 0..* | see rule | Rules contained in this rule
dependent | ? | 0..* | BackboneElement | Which other rules to apply in the context of this rule
name | ? | 1..1 | id | Name of a rule or group to apply
variable | ? | 1..* | string | Variable to pass to the rule or group
documentation | | 0..1 | string | Documentation for this instance of data
Documentation for this format

UML Diagram (Legend)

XML Template

```
<StructureMap xmlns="http://hl7.org/fhir">
 <!-- from Resource: id, meta, implicitRules, and language -->
 <!-- from DomainResource: text, contained, extension, and modifierExtension -->
 <url value="[uri]"/><!-- 1..1 Logical URI to reference this structure map (globally unique) -->
 <identifier><!-- 0..* Identifier Additional identifier for the structure map --></identifier>
 <version value="[string]"/><!-- 0..1 Business version of the structure map -->
 <name value="[string]"/><!-- 1..1 Name for this structure map (computer friendly) -->
 <title value="[string]"/><!-- 0..1 Name for this structure map (human friendly) -->
 <status value="[code]"/><!-- 1..1 draft | active | retired | unknown -->
 <experimental value="[boolean]"/><!-- 0..1 For testing purposes, not real usage -->
 <date value="[dateTime]"/><!-- 0..1 Date this was last changed -->
 <publisher value="[string]"/><!-- 0..1 Name of the publisher (organization or individual) -->
 <contact><!-- 0..* ContactDetail Contact details for the publisher --></contact>
 <description value="[markdown]"/><!-- 0..1 Natural language description of the structure map -->
 <useContext><!-- 0..* UsageContext Context the content is intended to support --></useContext>
 <jurisdiction><!-- 0..* CodeableConcept Intended jurisdiction for structure map (if applicable) --></jurisdiction>
 <purpose value="[markdown]"/><!-- 0..1 Why this structure map is defined -->
 <copyright value="[markdown]"/><!-- 0..1 Use and/or publishing restrictions -->
 <structure>  <!-- 0..* Structure Definition used by this map -->
  <url value="[uri]"/><!-- 1..1 Canonical URL for structure definition -->
  <mode value="[code]"/><!-- 1..1 source | queried | target | produced -->
  <alias value="[string]"/><!-- 0..1 Name for type in this map -->
  <documentation value="[string]"/><!-- 0..1 Documentation on use of structure -->
 </structure>
 <import value="[uri]"/><!-- 0..* Other maps used by this map (canonical URLs) -->
 <group>  <!-- 1..* Named sections for reader convenience -->
  <name value="[id]"/><!-- 1..1 Human-readable label -->
  <extends value="[id]"/><!-- 0..1 Another group that this group adds rules to -->
  <typeMode value="[code]"/><!-- 1..1 none | types | type-and-types -->
  <documentation value="[string]"/><!-- 0..1 Additional description/explaination for group -->
  <input>  <!-- 1..* Named instance provided when invoking the map -->
   <name value="[id]"/><!-- 1..1 Name for this instance of data -->
   <type value="[string]"/><!-- 0..1 Type for this instance of data -->
   <mode value="[code]"/><!-- 1..1 source | target -->
   <documentation value="[string]"/><!-- 0..1 Documentation for this instance of data -->
  </input>
  <rule>  <!-- 1..* Transform Rule from source to target -->
   <name value="[id]"/><!-- 1..1 Name of the rule for internal references -->
   <source>  <!-- 1..* Source inputs to the mapping -->
```

```
    <context value="[id]"/><!-- 1..1 Type or variable this rule applies to -->
    <min value="[integer]"/><!-- 0..1 Specified minimum cardinality -->
    <max value="[string]"/><!-- 0..1 Specified maximum cardinality (number or *) -->
    <type value="[string]"/><!-- 0..1 Rule only applies if source has this type -->
    <defaultValue[x]><!-- 0..1 * Default value if no value exists --></defaultValue[x]>
    <element value="[string]"/><!-- 0..1 Optional field for this source -->
    <listMode value="[code]"/><!-- 0..1 first | not_first | last | not_last | only_one -->
    <variable value="[id]"/><!-- 0..1 Named context for field, if a field is specified -->
    <condition value="[string]"/><!-- 0..1 FHIRPath expression  - must be true or the rule does not apply -->
    <check value="[string]"/><!-- 0..1 FHIRPath expression  - must be true or the mapping engine throws an error instead
     of completing -->
   </source>
   <target>  <!-- 0..* Content to create because of this mapping rule -->
    <context value="[id]"/><!-- 0..1 Type or variable this rule applies to -->
    <contextType value="[code]"/><!-- 0..1 type | variable -->
    <element value="[string]"/><!-- 0..1 Field to create in the context -->
    <variable value="[id]"/><!-- 0..1 Named context for field, if desired, and a field is specified -->
    <listMode value="[code]"/><!-- 0..* first | share | last | collate -->
    <listRuleId value="[id]"/><!-- 0..1 Internal rule reference for shared list items -->
    <transform value="[code]"/><!-- 0..1 create | copy + -->
    <parameter>  <!-- 0..* Parameters to the transform -->
     <value[x]><!-- 1..1 id|string|boolean|integer|decimal Parameter value - variable or literal --></value[x]>
    </parameter>
   </target>
   <rule><!-- 0..* Content as for StructureMap.group.rule Rules contained in this rule --></rule>
   <dependent>  <!-- 0..* Which other rules to apply in the context of this rule -->
    <name value="[id]"/><!-- 1..1 Name of a rule or group to apply -->
    <variable value="[string]"/><!-- 1..* Variable to pass to the rule or group -->
   </dependent>
   <documentation value="[string]"/><!-- 0..1 Documentation for this instance of data -->
  </rule>
 </group>
</StructureMap>
```

JSON Template

```
{
  "resourceType" : "StructureMap",
  // from Resource: id, meta, implicitRules, and language
  // from DomainResource: text, contained, extension, and modifierExtension
  "url" : "<uri>", // R!  Logical URI to reference this structure map (globally unique)
  "identifier" : [{ Identifier }], // Additional identifier for the structure map
  "version" : "<string>", // Business version of the structure map
  "name" : "<string>", // R!  Name for this structure map (computer friendly)
  "title" : "<string>", // Name for this structure map (human friendly)
  "status" : "<code>", // R!  draft | active | retired | unknown
  "experimental" : <boolean>, // For testing purposes, not real usage
  "date" : "<dateTime>", // Date this was last changed
  "publisher" : "<string>", // Name of the publisher (organization or individual)
  "contact" : [{ ContactDetail }], // Contact details for the publisher
  "description" : "<markdown>", // Natural language description of the structure map
  "useContext" : [{ UsageContext }], // Context the content is intended to support
  "jurisdiction" : [{ CodeableConcept }], // Intended jurisdiction for structure map (if applicable)
  "purpose" : "<markdown>", // Why this structure map is defined
  "copyright" : "<markdown>", // Use and/or publishing restrictions
  "structure" : [{ // Structure Definition used by this map
    "url" : "<uri>", // R!  Canonical URL for structure definition
    "mode" : "<code>", // R!  source | queried | target | produced
    "alias" : "<string>", // Name for type in this map
    "documentation" : "<string>" // Documentation on use of structure
  }],
  "import" : ["<uri>"], // Other maps used by this map (canonical URLs)
  "group" : [{ // R!  Named sections for reader convenience
    "name" : "<id>", // R!  Human-readable label
    "extends" : "<id>", // Another group that this group adds rules to
    "typeMode" : "<code>", // R!  none | types | type-and-types
    "documentation" : "<string>", // Additional description/explaination for group
    "input" : [{ // R!  Named instance provided when invoking the map
      "name" : "<id>", // R!  Name for this instance of data
      "type" : "<string>", // Type for this instance of data
      "mode" : "<code>", // R!  source | target
      "documentation" : "<string>" // Documentation for this instance of data
    }],
    "rule" : [{ // R!  Transform Rule from source to target
      "name" : "<id>", // R!  Name of the rule for internal references
      "source" : [{ // R!  Source inputs to the mapping
        "context" : "<id>", // R!  Type or variable this rule applies to
        "min" : <integer>, // Specified minimum cardinality
        "max" : "<string>", // Specified maximum cardinality (number or *)
        "type" : "<string>", // Rule only applies if source has this type
        // defaultValue[x]: Default value if no value exists. One of these 23:
        "defaultValueInteger" : <integer>,
        "defaultValueDecimal" : <decimal>,
        "defaultValueDateTime" : "<dateTime>",
        "defaultValueDate" : "<date>",
        "defaultValueInstant" : "<instant>",
        "defaultValueString" : "<string>",
        "defaultValueUri" : "<uri>",
        "defaultValueBoolean" : <boolean>,
        "defaultValueCode" : "<code>",
```

```
        "defaultValueBase64Binary" : "<base64Binary>",
        "defaultValueCoding" : { Coding },
        "defaultValueCodeableConcept" : { CodeableConcept },
        "defaultValueAttachment" : { Attachment },
        "defaultValueIdentifier" : { Identifier },
        "defaultValueQuantity" : { Quantity },
        "defaultValueRange" : { Range },
        "defaultValuePeriod" : { Period },
        "defaultValueRatio" : { Ratio },
        "defaultValueHumanName" : { HumanName },
        "defaultValueAddress" : { Address },
        "defaultValueContactPoint" : { ContactPoint },
        "defaultValueSchedule" : { Schedule },
        "defaultValueReference" : { Reference },
        "element" : "<string>", // Optional field for this source
        "listMode" : "<code>", // first | not_first | last | not_last | only_one
        "variable" : "<id>", // Named context for field, if a field is specified
        "condition" : "<string>", // FHIRPath expression  - must be true or the rule does not apply
        "check" : "<string>" // FHIRPath expression  - must be true or the mapping engine throws an error instead of
    completing
      }],
      "target" : [{ // Content to create because of this mapping rule
        "context" : "<id>", // Type or variable this rule applies to
        "contextType" : "<code>", // type | variable
        "element" : "<string>", // Field to create in the context
        "variable" : "<id>", // Named context for field, if desired, and a field is specified
        "listMode" : ["<code>"], // first | share | last | collate
        "listRuleId" : "<id>", // Internal rule reference for shared list items
        "transform" : "<code>", // create | copy +
        "parameter" : [{ // Parameters to the transform
          // value[x]: Parameter value - variable or literal. One of these 5:
          "valueId" : "<id>"
          "valueString" : "<string>"
          "valueBoolean" : <boolean>
          "valueInteger" : <integer>
          "valueDecimal" : <decimal>
        }]
      }],
      "rule" : [{ Content as for StructureMap.group.rule }], // Rules contained in this rule
      "dependent" : [{ // Which other rules to apply in the context of this rule
        "name" : "<id>", // R!  Name of a rule or group to apply
        "variable" : ["<string>"] // R!  Variable to pass to the rule or group
      }],
      "documentation" : "<string>" // Documentation for this instance of data
    }]
  }]
}
```

Turtle Template

```
@prefix fhir: <http://hl7.org/fhir/> .


[ a fhir:StructureMap;
  fhir:nodeRole fhir:treeRoot; # if this is the parser root

  # from Resource: .id, .meta, .implicitRules, and .language
  # from DomainResource: .text, .contained, .extension, and .modifierExtension
  fhir:StructureMap.url [ uri ]; # 1..1 Logical URI to reference this structure map (globally unique)
  fhir:StructureMap.identifier [ Identifier ], ... ; # 0..* Additional identifier for the structure map
  fhir:StructureMap.version [ string ]; # 0..1 Business version of the structure map
  fhir:StructureMap.name [ string ]; # 1..1 Name for this structure map (computer friendly)
  fhir:StructureMap.title [ string ]; # 0..1 Name for this structure map (human friendly)
  fhir:StructureMap.status [ code ]; # 1..1 draft | active | retired | unknown
  fhir:StructureMap.experimental [ boolean ]; # 0..1 For testing purposes, not real usage
  fhir:StructureMap.date [ dateTime ]; # 0..1 Date this was last changed
  fhir:StructureMap.publisher [ string ]; # 0..1 Name of the publisher (organization or individual)
  fhir:StructureMap.contact [ ContactDetail ], ... ; # 0..* Contact details for the publisher
  fhir:StructureMap.description [ markdown ]; # 0..1 Natural language description of the structure map
  fhir:StructureMap.useContext [ UsageContext ], ... ; # 0..* Context the content is intended to support
  fhir:StructureMap.jurisdiction [ CodeableConcept ], ... ; # 0..* Intended jurisdiction for structure map (if
    applicable)
  fhir:StructureMap.purpose [ markdown ]; # 0..1 Why this structure map is defined
  fhir:StructureMap.copyright [ markdown ]; # 0..1 Use and/or publishing restrictions
  fhir:StructureMap.structure [ # 0..* Structure Definition used by this map
    fhir:StructureMap.structure.url [ uri ]; # 1..1 Canonical URL for structure definition
    fhir:StructureMap.structure.mode [ code ]; # 1..1 source | queried | target | produced
    fhir:StructureMap.structure.alias [ string ]; # 0..1 Name for type in this map
    fhir:StructureMap.structure.documentation [ string ]; # 0..1 Documentation on use of structure
  ], ...;
  fhir:StructureMap.import [ uri ], ... ; # 0..* Other maps used by this map (canonical URLs)
  fhir:StructureMap.group [ # 1..* Named sections for reader convenience
    fhir:StructureMap.group.name [ id ]; # 1..1 Human-readable label
    fhir:StructureMap.group.extends [ id ]; # 0..1 Another group that this group adds rules to
    fhir:StructureMap.group.typeMode [ code ]; # 1..1 none | types | type-and-types
    fhir:StructureMap.group.documentation [ string ]; # 0..1 Additional description/explaination for group
    fhir:StructureMap.group.input [ # 1..* Named instance provided when invoking the map
      fhir:StructureMap.group.input.name [ id ]; # 1..1 Name for this instance of data
      fhir:StructureMap.group.input.type [ string ]; # 0..1 Type for this instance of data
      fhir:StructureMap.group.input.mode [ code ]; # 1..1 source | target
```

```
      fhir:StructureMap.group.input.documentation [ string ]; # 0..1 Documentation for this instance of data
    ], ...;
    fhir:StructureMap.group.rule [ # 1..* Transform Rule from source to target
      fhir:StructureMap.group.rule.name [ id ]; # 1..1 Name of the rule for internal references
      fhir:StructureMap.group.rule.source [ # 1..* Source inputs to the mapping
        fhir:StructureMap.group.rule.source.context [ id ]; # 1..1 Type or variable this rule applies to
        fhir:StructureMap.group.rule.source.min [ integer ]; # 0..1 Specified minimum cardinality
        fhir:StructureMap.group.rule.source.max [ string ]; # 0..1 Specified maximum cardinality (number or *)
        fhir:StructureMap.group.rule.source.type [ string ]; # 0..1 Rule only applies if source has this type
        # StructureMap.group.rule.source.defaultValue[x] : 0..1 Default value if no value exists. One of these 38
          fhir:StructureMap.group.rule.source.defaultValueBase64Binary [ base64Binary ]
          fhir:StructureMap.group.rule.source.defaultValueBoolean [ boolean ]
          fhir:StructureMap.group.rule.source.defaultValueCode [ code ]
          fhir:StructureMap.group.rule.source.defaultValueDate [ date ]
          fhir:StructureMap.group.rule.source.defaultValueDateTime [ dateTime ]
          fhir:StructureMap.group.rule.source.defaultValueDecimal [ decimal ]
          fhir:StructureMap.group.rule.source.defaultValueId [ id ]
          fhir:StructureMap.group.rule.source.defaultValueInstant [ instant ]
          fhir:StructureMap.group.rule.source.defaultValueInteger [ integer ]
          fhir:StructureMap.group.rule.source.defaultValueMarkdown [ markdown ]
          fhir:StructureMap.group.rule.source.defaultValueOid [ oid ]
          fhir:StructureMap.group.rule.source.defaultValuePositiveInt [ positiveInt ]
          fhir:StructureMap.group.rule.source.defaultValueString [ string ]
          fhir:StructureMap.group.rule.source.defaultValueTime [ time ]
          fhir:StructureMap.group.rule.source.defaultValueUnsignedInt [ unsignedInt ]
          fhir:StructureMap.group.rule.source.defaultValueUri [ uri ]
          fhir:StructureMap.group.rule.source.defaultValueAddress [ Address ]
          fhir:StructureMap.group.rule.source.defaultValueAge [ Age ]
          fhir:StructureMap.group.rule.source.defaultValueAnnotation [ Annotation ]
          fhir:StructureMap.group.rule.source.defaultValueAttachment [ Attachment ]
          fhir:StructureMap.group.rule.source.defaultValueCodeableConcept [ CodeableConcept ]
          fhir:StructureMap.group.rule.source.defaultValueCoding [ Coding ]
          fhir:StructureMap.group.rule.source.defaultValueContactPoint [ ContactPoint ]
          fhir:StructureMap.group.rule.source.defaultValueCount [ Count ]
          fhir:StructureMap.group.rule.source.defaultValueDistance [ Distance ]
          fhir:StructureMap.group.rule.source.defaultValueDuration [ Duration ]
          fhir:StructureMap.group.rule.source.defaultValueHumanName [ HumanName ]
          fhir:StructureMap.group.rule.source.defaultValueIdentifier [ Identifier ]
          fhir:StructureMap.group.rule.source.defaultValueMoney [ Money ]
          fhir:StructureMap.group.rule.source.defaultValuePeriod [ Period ]
          fhir:StructureMap.group.rule.source.defaultValueQuantity [ Quantity ]
          fhir:StructureMap.group.rule.source.defaultValueRange [ Range ]
          fhir:StructureMap.group.rule.source.defaultValueRatio [ Ratio ]
          fhir:StructureMap.group.rule.source.defaultValueReference [ Reference ]
          fhir:StructureMap.group.rule.source.defaultValueSampledData [ SampledData ]
          fhir:StructureMap.group.rule.source.defaultValueSignature [ Signature ]
          fhir:StructureMap.group.rule.source.defaultValueTiming [ Timing ]
          fhir:StructureMap.group.rule.source.defaultValueMeta [ Meta ]
        fhir:StructureMap.group.rule.source.element [ string ]; # 0..1 Optional field for this source
        fhir:StructureMap.group.rule.source.listMode [ code ]; # 0..1 first | not_first | last | not_last | only_one
        fhir:StructureMap.group.rule.source.variable [ id ]; # 0..1 Named context for field, if a field is specified
        fhir:StructureMap.group.rule.source.condition [ string ]; # 0..1 FHIRPath expression  - must be true or the rule
     does not apply
        fhir:StructureMap.group.rule.source.check [ string ]; # 0..1 FHIRPath expression  - must be true or the mapping
    engine throws an error instead of completing
      ], ...;
      fhir:StructureMap.group.rule.target [ # 0..* Content to create because of this mapping rule
        fhir:StructureMap.group.rule.target.context [ id ]; # 0..1 Type or variable this rule applies to
        fhir:StructureMap.group.rule.target.contextType [ code ]; # 0..1 type | variable
        fhir:StructureMap.group.rule.target.element [ string ]; # 0..1 Field to create in the context
        fhir:StructureMap.group.rule.target.variable [ id ]; # 0..1 Named context for field, if desired, and a field is
    specified
        fhir:StructureMap.group.rule.target.listMode [ code ], ... ; # 0..* first | share | last | collate
        fhir:StructureMap.group.rule.target.listRuleId [ id ]; # 0..1 Internal rule reference for shared list items
        fhir:StructureMap.group.rule.target.transform [ code ]; # 0..1 create | copy +
        fhir:StructureMap.group.rule.target.parameter [ # 0..* Parameters to the transform
          # StructureMap.group.rule.target.parameter.value[x] : 1..1 Parameter value - variable or literal. One of these
     5
            fhir:StructureMap.group.rule.target.parameter.valueId [ id ]
            fhir:StructureMap.group.rule.target.parameter.valueString [ string ]
            fhir:StructureMap.group.rule.target.parameter.valueBoolean [ boolean ]
            fhir:StructureMap.group.rule.target.parameter.valueInteger [ integer ]
            fhir:StructureMap.group.rule.target.parameter.valueDecimal [ decimal ]
        ], ...;
      ], ...;
      fhir:StructureMap.group.rule.rule [ See StructureMap.group.rule ], ... ; # 0..* Rules contained in this rule
      fhir:StructureMap.group.rule.dependent [ # 0..* Which other rules to apply in the context of this rule
        fhir:StructureMap.group.rule.dependent.name [ id ]; # 1..1 Name of a rule or group to apply
        fhir:StructureMap.group.rule.dependent.variable [ string ], ... ; # 1..* Variable to pass to the rule or group

      ], ...;
      fhir:StructureMap.group.rule.documentation [ string ]; # 0..1 Documentation for this instance of data
    ], ...;
  ], ...;
]
```

Changes since DSTU2

This resource did not exist in Release 2

This analysis is available as XML or JSON.

Alternate definitions: Master Definition (XML, JSON), XML Schema/Schematron (for) + JSON Schema, ShEx (for Turtle)

7.7.4.1 Terminology Bindings

Path | Definition | Type | Reference
StructureMap.status | The lifecycle status of a Value Set or Concept Map. | Required | PublicationStatus
StructureMap.jurisdiction | Countries and regions within which this artifact is targeted for use | Extensible | Jurisdiction ValueSet
StructureMap.structure.mode | How the referenced structure is used in this mapping | Required | StructureMapModelMode
StructureMap.group.typeMode | If this is the default rule set to apply for the source type, or this combination of types | Required | StructureMapGroupTypeMode
StructureMap.group.input.mode | Mode for this instance of data | Required | StructureMapInputMode
StructureMap.group.rule.source.listMode | If field is a list, how to manage the source | Required | StructureMapSourceListMode
StructureMap.group.rule.target.contextType | How to interpret the context | Required | StructureMapContextType
StructureMap.group.rule.target.listMode | If field is a list, how to manage the production | Required | StructureMapTargetListMode
StructureMap.group.rule.target.transform | How data is copied/created | Required | StructureMapTransform

7.7.4.2 Constraints

* smp-1: On StructureMap.group.rule.target: Can only have a element if you have a context (expression on StructureMap.group.rule.target: element.exists() implies context.exists())

* smp-2: On StructureMap.group.rule.target: Must have a contextType if you have a context (expression on StructureMap.group.rule.target: context.exists() implies contextType.exists())

7.7.5 Search Parameters

Search parameters for this resource. The common parameters also apply. See Searching for more information about searching in REST, messaging, and services.

Name | Type | Description | Expression | In Common
date | date | The structure map publication date | StructureMap.date
description | string | The description of the structure map | StructureMap.description
identifier | token | External identifier for the structure map | StructureMap.identifier
jurisdiction | token | Intended jurisdiction for the structure map | StructureMap.jurisdiction
name | string | Computationally friendly name of the structure map | StructureMap.name
publisher | string | Name of the publisher of the structure map | StructureMap.publisher
status | token | The current status of the structure map | StructureMap.status
title | string | The human-friendly name of the structure map | StructureMap.title
url | uri | The uri that identifies the structure map | StructureMap.url
version | token | The business version of the structure map | StructureMap.version

7.7.6 Resource StructureMap - Examples

URL: https://hl7.org/fhir/STU3/structuremap-examples.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.7.6 Resource StructureMap - Examples

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative | Compartments: Not linked to any defined compartments

Example List:

Example | example | XML | JSON | Turtle
Transforms an ActivityDefinition to a SupplyRequest | supplyrequest-transform | XML | JSON | Turtle

Usage note: every effort has been made to ensure that the examples are correct and useful, but they are not a normative part of the specification.

7.7.7 Resource StructureMap - Detailed Descriptions

URL: https://hl7.org/fhir/STU3/structuremap-definitions.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.7.7 Resource StructureMap - Detailed Descriptions

FHIR Infrastructure Work Group | Maturity Level: 2 | Trial Use | Compartments: Not linked to any defined compartments

Detailed Descriptions for the elements in the StructureMap resource.

StructureMap
Definition | A Map of relationships between 2 structures that can be used to transform data.
Control | 1..1
Alternate Names | Template; Profile
StructureMap.url

Definition | An absolute URI that is used to identify this structure map when it is referenced in a specification, model, design or an instance. This SHALL be a URL, SHOULD be globally unique, and SHOULD be an address at which this structure map is (or will be) published. The URL SHOULD include the major version of the structure map. For more information see Technical and Business Versions.
Control | 1..1
Type | uri
Requirements | Allows the structure map to be referenced by a single globally unique identifier.
Alternate Names | url; authoritative-url; destination; identity
Summary | true
Comments | Can be a urn:uuid: or a urn:oid:, but real http: addresses are preferred. Multiple instances may share the same url if they have a distinct version.
StructureMap.identifier
Definition | A formal identifier that is used to identify this structure map when it is represented in other formats, or referenced in a specification, model, design or an instance.
Note | This is a business identifer, not a resource identifier (see discussion)
Control | 0..*
Type | Identifier
Requirements | Allows externally provided and/or usable business identifiers to be easily associated with the module.
Summary | true
Comments | Typically, this is used for identifiers that can go in an HL7 V3 II (instance identifier) data type, e.g., to identify this structure map outside of FHIR, where it is not possible to use the logical URI.
StructureMap.version
Definition | The identifier that is used to identify this version of the structure map when it is referenced in a specification, model, design or instance. This is an arbitrary value managed by the structure map author and is not expected to be globally unique. For example, it might be a timestamp (e.g. yyyymmdd) if a managed version is not available. There is also no expectation that versions can be placed in a lexicographical sequence.
Note | This is a business versionId, not a resource version id (see discussion)
Control | 0..1
Type | string
Summary | true
Comments | There may be different structure map instances that have the same identifier but different versions. The version can be appended to the url in a reference to allow a refrence to a particular business version of the structure map with the format [url]|[version].
StructureMap.name
Definition | A natural language name identifying the structure map. This name should be usable as an identifier for the module by machine processing applications such as code generation.
Control | 1..1
Type | string
Requirements | Support human navigation and code generation.
Summary | true
Comments | The name is not expected to be globally unique. The name should be a simple alpha-numeric type name to ensure that it is computable friendly.
StructureMap.title
Definition | A short, descriptive, user-friendly title for the structure map.
Control | 0..1
Type | string
Summary | true
Comments | This name does not need to be machine-processing friendly and may contain punctuation, white-space, etc.
StructureMap.status
Definition | The status of this structure map. Enables tracking the life-cycle of the content.
Control | 1..1
Terminology Binding | PublicationStatus (Required)
Type | code
Is Modifier | true
Summary | true
Comments | Allows filtering of structure maps that are appropriate for use vs. not. This is labeled as "Is Modifier" because applications should not use a retired structure map without due consideration.
StructureMap.experimental
Definition | A boolean value to indicate that this structure map is authored for testing purposes (or education/evaluation/marketing), and is not intended to be used for genuine usage.
Control | 0..1
Type | boolean
Is Modifier | true
Requirements | Enables experimental content to be developed following the same lifecycle that would be used for a production-level structure map.
Summary | true
Comments | Allows filtering of structure map that are appropriate for use vs. not. This is labeled as "Is Modifier" because applications should not use an experimental structure map in production.
StructureMap.date
Definition | The date (and optionally time) when the structure map was published. The date must change if and when the business version changes and it must change if the status code changes. In addition, it should change when the substantive content of the structure map changes.
Control | 0..1
Type | dateTime
Alternate Names | Revision Date
Summary | true
Comments | Note that this is not the same as the resource last-modified-date, since the resource may be a secondary representation of the structure map. Additional specific dates may be added as extensions or be found by consulting Provenances associated with past versions of the resource.
StructureMap.publisher

Definition | The name of the individual or organization that published the structure map.
Control | 0..1
Type | string
Requirements | Helps establish the "authority/credibility" of the structure map. May also allow for contact.
Summary | true
Comments | Usually an organization, but may be an individual. The publisher (or steward) of the structure map is the organization or individual primarily responsible for the maintenance and upkeep of the structure map. This is not necessarily the same individual or organization that developed and initially authored the content. The publisher is the primary point of contact for questions or issues with the structure map. This item SHOULD be populated unless the information is available from context.
StructureMap.contact
Definition | Contact details to assist a user in finding and communicating with the publisher.
Control | 0..*
Type | ContactDetail
Summary | true
Comments | May be a web site, an email address, a telephone number, etc.
StructureMap.description
Definition | A free text natural language description of the structure map from a consumer's perspective.
Control | 0..1
Type | markdown
Comments | This description can be used to capture details such as why the structure map was built, comments about misuse, instructions for clinical use and interpretation, literature references, examples from the paper world, etc. It is not a rendering of the structure map as conveyed in the 'text' field of the resource itself. This item SHOULD be populated unless the information is available from context (e.g. the language of the profile is presumed to be the predominant language in the place the profile was created).
StructureMap.useContext
Definition | The content was developed with a focus and intent of supporting the contexts that are listed. These terms may be used to assist with indexing and searching for appropriate structure map instances.
Control | 0..*
Type | UsageContext
Requirements | Assist in searching for appropriate content.
Summary | true
Comments | When multiple useContexts are specified, there is no expectation whether all or any of the contexts apply.
StructureMap.jurisdiction
Definition | A legal or geographic region in which the structure map is intended to be used.
Control | 0..*
Terminology Binding | Jurisdiction ValueSet (Extensible)
Type | CodeableConcept
Summary | true
Comments | It may be possible for the structure map to be used in jurisdictions other than those for which it was originally designed or intended.
StructureMap.purpose
Definition | Explaination of why this structure map is needed and why it has been designed as it has.
Control | 0..1
Type | markdown
Comments | This element does not describe the usage of the structure map Instead it provides traceability of ''why'' the resource is either needed or ''why'' it is defined as it is. This may be used to point to source materials or specifications that drove the structure of this structure map.
StructureMap.copyright
Definition | A copyright statement relating to the structure map and/or its contents. Copyright statements are generally legal restrictions on the use and publishing of the structure map.
Control | 0..1
Type | markdown
Requirements | Consumers must be able to determine any legal restrictions on the use of the structure map and/or its content.
Alternate Names | License; Restrictions
StructureMap.structure
Definition | A structure definition used by this map. The structure definition may describe instances that are converted, or the instances that are produced.
Control | 0..*
Summary | true
Comments | It is not necessary for a structure map to identify any dependent structures, though not losting them may restrict its usefulness.
StructureMap.structure.url
Definition | The canonical URL that identifies the structure.
Control | 1..1
Type | uri
Summary | true
StructureMap.structure.mode
Definition | How the referenced structure is used in this mapping.
Control | 1..1
Terminology Binding | StructureMapModelMode (Required)
Type | code
Summary | true
StructureMap.structure.alias
Definition | The name used for this type in the map.
Control | 0..1
Type | string
Summary | true

Comments | This is needed if both types have the same name (e.g. version conversion).
StructureMap.structure.documentation
Definition | Documentation that describes how the structure is used in the mapping.
Control | 0..1
Type | string
StructureMap.import
Definition | Other maps used by this map (canonical URLs).
Control | 0..*
Type | uri
Summary | true
StructureMap.group
Definition | Organizes the mapping into managable chunks for human review/ease of maintenance.
Control | 1..*
Summary | true
StructureMap.group.name
Definition | A unique name for the group for the convenience of human readers.
Control | 1..1
Type | id
Summary | true
StructureMap.group.extends
Definition | Another group that this group adds rules to.
Control | 0..1
Type | id
Summary | true
StructureMap.group.typeMode
Definition | If this is the default rule set to apply for thie source type, or this combination of types.
Control | 1..1
Terminology Binding | StructureMapGroupTypeMode (Required)
Type | code
Summary | true
Comments | Not applicable if the underlying model is untyped. There can only be one default mapping for any particular type combination.
StructureMap.group.documentation
Definition | Additional supporting documentation that explains the purpose of the group and the types of mappings within it.
Control | 0..1
Type | string
Summary | true
StructureMap.group.input
Definition | A name assigned to an instance of data. The instance must be provided when the mapping is invoked.
Control | 1..*
Summary | true
Comments | If no inputs are named, then the entry mappings are type based.
StructureMap.group.input.name
Definition | Name for this instance of data.
Control | 1..1
Type | id
Summary | true
StructureMap.group.input.type
Definition | Type for this instance of data.
Control | 0..1
Type | string
Summary | true
StructureMap.group.input.mode
Definition | Mode for this instance of data.
Control | 1..1
Terminology Binding | StructureMapInputMode (Required)
Type | code
Summary | true
StructureMap.group.input.documentation
Definition | Documentation for this instance of data.
Control | 0..1
Type | string
StructureMap.group.rule
Definition | Transform Rule from source to target.
Control | 1..*
Summary | true
StructureMap.group.rule.name
Definition | Name of the rule for internal references.
Control | 1..1
Type | id
Summary | true
StructureMap.group.rule.source
Definition | Source inputs to the mapping.

Control | 1..*
Summary | true
StructureMap.group.rule.source.context
Definition | Type or variable this rule applies to.
Control | 1..1
Type | id
Summary | true
StructureMap.group.rule.source.min
Definition | Specified minimum cardinality for the element. This is optional; if present, it acts an implicit check on the input content.
Control | 0..1
Type | integer
Summary | true
StructureMap.group.rule.source.max
Definition | Specified maximum cardinality for the element - a number or a "". This is optional; if present, it acts an implicit check on the input content (just serves as documentation; it's the default value).
Control | 0..1
Type | string
Summary | true
StructureMap.group.rule.source.type
Definition | Specified type for the element. This works as a condition on the mapping - use for polymorphic elements.
Control | 0..1
Type | string
Summary | true
StructureMap.group.rule.source.defaultValue[x]
Definition | A value to use if there is no existing value in the source object.
Control | 0..1
Type | *
[x] Note | See Choice of Data Types for further information about how to use [x]
Summary | true
Comments | If there's a default value on an item that can repeat, it will only be used once.
StructureMap.group.rule.source.element
Definition | Optional field for this source.
Control | 0..1
Type | string
Summary | true
StructureMap.group.rule.source.listMode
Definition | How to handle the list mode for this element.
Control | 0..1
Terminology Binding | StructureMapSourceListMode (Required)
Type | code
Summary | true
StructureMap.group.rule.source.variable
Definition | Named context for field, if a field is specified.
Control | 0..1
Type | id
Summary | true
StructureMap.group.rule.source.condition
Definition | FHIRPath expression - must be true or the rule does not apply.
Control | 0..1
Type | string
Summary | true
StructureMap.group.rule.source.check
Definition | FHIRPath expression - must be true or the mapping engine throws an error instead of completing.
Control | 0..1
Type | string
Summary | true
StructureMap.group.rule.target
Definition | Content to create because of this mapping rule.
Control | 0..*
Summary | true
Invariants | Defined on this element smp-1: Can only have a element if you have a context (expression : element.exists() implies context.exists(), xpath: not(f:element) or (f:context)) smp-2: Must have a contextType if you have a context (expression : context.exists() implies contextType.exists(), xpath: not(f:context) or (f:contextType))
StructureMap.group.rule.target.context
Definition | Type or variable this rule applies to.
Control | 0..1
Type | id
Summary | true
StructureMap.group.rule.target.contextType
Definition | How to interpret the context.
Control | 0..1
Terminology Binding | StructureMapContextType (Required)

Type | code
Summary | true
StructureMap.group.rule.target.element
Definition | Field to create in the context.
Control | 0..1
Type | string
Summary | true
StructureMap.group.rule.target.variable
Definition | Named context for field, if desired, and a field is specified.
Control | 0..1
Type | id
Summary | true
StructureMap.group.rule.target.listMode
Definition | If field is a list, how to manage the list.
Control | 0..*
Terminology Binding | StructureMapTargetListMode (Required)
Type | code
Summary | true
StructureMap.group.rule.target.listRuleId
Definition | Internal rule reference for shared list items.
Control | 0..1
Type | id
Summary | true
StructureMap.group.rule.target.transform
Definition | How the data is copied / created.
Control | 0..1
Terminology Binding | StructureMapTransform (Required)
Type | code
Summary | true
StructureMap.group.rule.target.parameter
Definition | Parameters to the transform.
Control | 0..*
Summary | true
StructureMap.group.rule.target.parameter.value[x]
Definition | Parameter value - variable or literal.
Control | 1..1
Type | id|string|boolean|integer|decimal
[x] Note | See Choice of Data Types for further information about how to use [x]
Summary | true
StructureMap.group.rule.rule
Definition | Rules contained in this rule.
Control | 0..*
Type | See StructureMap.group.rule
Summary | true
StructureMap.group.rule.dependent
Definition | Which other rules to apply in the context of this rule.
Control | 0..*
Summary | true
StructureMap.group.rule.dependent.name
Definition | Name of a rule or group to apply.
Control | 1..1
Type | id
Summary | true
StructureMap.group.rule.dependent.variable
Definition | Variable to pass to the rule or group.
Control | 1..*
Type | string
Summary | true
StructureMap.group.rule.documentation
Definition | Documentation for this instance of data.
Control | 0..1
Type | string

7.7.8 Resource StructureMap - Mappings

URL: https://hl7.org/fhir/STU3/structuremap-mappings.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.7.8 Resource StructureMap - Mappings

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative | Compartments: Not linked to any defined compartments

Mappings:

RIM Mapping

Mappings for the structuremap resource.

7.7.8.1 Mappings for RIM Mapping (http://hl7.org/v3)

StructureMap | n/a

7.7.9 Resource StructureMap - Design Notes

URL: https://hl7.org/fhir/STU3/structuremap-explanations.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.7.9 Resource StructureMap - Design Notes

to do

7.7.10 Resource StructureMap - Extensions & Profiles

URL: https://hl7.org/fhir/STU3/structuremap-profiles.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.7.10 Resource StructureMap - Extensions & Profiles

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

This table lists profiles and extensions for the StructureMap resource. For background information, see Profiling Resources and Extensibility. Additional profiles and resources may be found in published Implementation Guides , or in the Conformance resource registry .

7.7.10.1 Profiles

No Profiles defined for this resource

7.7.10.2 Extensions

resource-approvalDate | approvalDate | for Resource HL7 Extensions
resource-effectivePeriod | effectivePeriod | for Resource HL7 Extensions
resource-lastReviewDate | lastReviewDate | for Resource HL7 Extensions

Extensions that reference this resource:

questionnaire-sourceStructureMap | sourceStructureMap | for Core extensions for Questionnaire
questionnaire-targetStructureMap | targetStructureMap | for Core extensions for Questionnaire

7.7.10.3 Search Extensions

No Search Extensions defined for this resource

7.7.11 Resource StructureMap - Operations

URL: https://hl7.org/fhir/STU3/structuremap-operations.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.7.11 Resource StructureMap - Operations

FHIR Infrastructure Work Group | Maturity Level: 2 | Trial Use | Compartments: Not linked to any defined compartments

This resource has 1 operation associated with it:

$transform | Model Instance Transformation

For more information about operations, including how they are invoked, see Operations.

7.7.11.1 Model Instance Transformation

The transform operation takes input content, applies a structure map transform, and then returns the output.

The official URL for this operation definition is

```
http://hl7.org/fhir/OperationDefinition/StructureMap-transform
```

Formal Definition (as a OperationDefinition).

URL: [base]/StructureMap/$transform

URL: [base]/StructureMap/[id]/$transform

This is an idempotent operation

In Parameters:

Name | Cardinality | Type | Binding | Profile | Documentation
source | 0..1 | uri | | | The structure map to apply. This is only needed if the operation is invoked at the resource level. If the $transform operation is invoked on a particular structure map, this will be ignored by the server
content | 1..1 | Resource | | | The logical content to transform
Out Parameters:
Name | Cardinality | Type | Binding | Profile | Documentation
return | 1..1 | Resource | | | The result of the transform

The input and return are specified as 'Resources'. In most usage of the $transform operation, either the input or return content is not a valid FHIR resource. In these cases, the return type is actually a Binary resource. For this operation, the Binary resources may be encoded directly, using a mime-type, as shown in the example. Note: this specification does not yet address the means by which the servers may know the correct mime types for the various content involved

7.7.11.1.1 Examples

Transform input model (Request):

```
POST [base]/StructureMap/example/$tansform
Content-Type: text/xml

<classA xmlns="http://example.org">
  <test>value</test>
</class>
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/xml
[other headers]

<message xmlns="http://example.org/other">
  <testValue value="value"/>
</message>
```

7.8 Common Example Scenarios in FHIR

URL: https://hl7.org/fhir/STU3/usecases.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

FHIR is a framework standard that defines a common way to solve healthcare problems and provides a set of resources that can be used in many different ways. This page describes how certain common usage scenarios are implemented using the capabilities that FHIR defines. The provided scenarios are examples of usage and are not in any way exhaustive. FHIR can and will be used in a wide variety of circumstances.

In addition, to the information on this page, see also Resource Guide.

7.8.1 Personal Health Record (PHR)

In the PHR scenario, an Electronic Medical Record system (EMR, though many other names and acronyms are also used) provides a RESTful API that allows patients to access their own medical record via a common web portal or mobile application, usually provided by a third party. In this scenario, the PHR provider:

* Provides the patient with a login that identifies them (or links the patient record to an external identity provided by OpenID, Facebook, Google, etc.)

* Authenticates the client using an appropriate OAuth server for the login (possibly their own) and restricts the client to viewing records associated with the specific patient (or patients, where appropriate access has been arranged)

The EMR exposes a FHIR server that supports the search and read operations on the following resources:

* the Patient resource in order to provide demographics to the client. When a client searches patients with no search criteria, they get a list of all patients they have access to

* search and read on the Document Reference resource to provide access to general patient documents in the form of PDFs etc. (PDFs are preferred)

* search and read on a set of clinical resources

Here is the Capabilities Statement for this scenario: XML or JSON.

The EMR may also choose to provide additional functionality, such as shared access to patient records by relatives/carers, to allow the patient to upload their own documents, medication statements, observations (e.g. from patient monitoring devices) and/or to allow the patient to make appointments. This additional functionality will involve additional API capabilities to be implemented and exposed. The EMR server may also choose to expose the search, read and history operation on the AuditEvent resource for the patient-specific records to allow patient review of record access. Note that all usage of the RESTful API should be logged in an AuditEvent resource.

7.8.2 Document Sharing (XDS)

One common way to integrate healthcare information from a variety of sources is to build a repository of documents around a patient record. Building a repository of documents allows for less stringent alignment around policy, procedures and

record-keeping/informatics standards.

The most widely adopted framework for sharing documents within institutions, regions, states or countries is IHE Cross-Enterprise Document Sharing (XDS). XDS allows for a federated system of repositories with a registry to provide coordinated access to the documents.

FHIR provides equivalent functionality to XDS that can be used to implement XDS behind the existing XDS.b interface, to provide a simpler mobile-friendly interface to an existing XDS ecosystem, or to link document sharing into other functionality provided through a FHIR interface.

The following FHIR Resources are involved in the XDS functionality:

* The DocumentReference resource describes a document that is located elsewhere. A document registry is a system that maintains a set of Document References

* The Binary support can be used to store the actual documents on a FHIR server. A repository is a system that stores the binary document in addition to Document References (or sometimes without)

* Patient, Practitioner and Organization resources provide support for identifying people and organizations

* The AuditEvent resource tracks usage of the document registry and repository

* The DocumentManifest resource describes a set of documents that are grouped together in original publication

At present, IHE is working with the FHIR project team to use FHIR for Mobile Health Documents (MHD).

7.8.3 Decision Support

One common use of healthcare information systems is to integrate some form of decision support software into clinical systems. Common uses of clinical decision support are:

* Drug-drug interaction checking, and more generally, prescription safety checks

* Suggesting commonly missed diagnostic data interpretations (including delta checking)

* Patient surveillance for early warning of deteriorating patient health (both acute and ambulatory care)

* Identifying candidates for alternative treatment plans for improved efficacy

Note that in addition to clinical decision support, there are also infrastructural uses, such as managing access control.

The various forms of decision support each involve different interaction patterns, so there is no single decision support implementation in the FHIR specification. Generally, the patterns fall into several classes:

* The decision comes from an engine entirely hidden behind a system interface and has no direct impact on the data exchange

* The decision support engine uses existing data and generates alarm messages concerning patient state that are visible on the FHIR interface

* The decision support engine is consulted through a described interface; it accepts a request for, and returns a decision

Any decision support may fall into multiple categories at once, depending on the perspective of a particular system.

* There is no particular support required from the FHIR specification, though there will be ongoing review of the contents of the resources to ensure that they support common decision support practices appropriately

* There is no suitable resource for this use yet. The Flag resource is intended for clinical notes about the patient, and is not intended for this use. A resource called "Alarm" is under preparation for this purpose

* A request for a decision support is understood as a search using a named _query that takes a set of parameters. See below

7.8.3.1 Explicit Requests for Decisions

When a query is initiated in order to get a decision made, the following considerations apply:

Request

* The request for a decision is made using one of the interaction patterns described for search/query: A RESTful search, a query posted to /Mailbox, a query message, or the Asynchronous query pattern

* The request has a _query parameter that identifies the decision that is being requested

* The request also has a set of parameter values. These parameter values may be the data that describes the decision being made or they may be references to specific resources that contain the request. In general, the more complex the decision request is, the more likely it is that a full resource is appropriate, particularly since this provides a ready made way to record and manage the requests themselves.

* In some of the query interaction patterns, the resources identified in the parameter value can be bundled up with the request. In others, only the references can be passed

* Which of the query patterns is most appropriate depends on the complexity of the decision support input, and the length of time the decision is expected to take. As either of these increases, the more complex query patterns become more appropriate

Response

* If the decision support engine is unable to provide the requested decision, it returns an OperationOutcome Resource describing the issue

* Otherwise it returns a resource that represents the decision, along with other resources as supporting information, as described by the resource, or applicable profiles

* In principle, the decision provider can choose to make a copy of the returned decision resource available through a normal RESTful interface, or it can choose not to. This decision may be constrained by applicable profiles, policy decision, or the innate nature of the query

* If either the decision provider or the requester choose to retain a copy of the decision, they must ensure that the (lack of) currency of the decision is appropriately considered when it is used

It follows from this then, that decisions that may be requested need at least a response resource defined, and possibly a request resource. This table summarizes known decisions for which resources have been defined.

Decision | Resources | Invocation
What immunizations should this patient have? | Response: ImmunizationRecommendation | The exact way to invoke this decision is not yet defined

Implementers are allowed to use existing resources for decisions not documented here, but there is no guarantee that they will be suitable. Improving decision support will be a focus for ongoing development during the Trial Use period.

On This Page:

PHR

Document Sharing (XDS)

Decision Support

7.9 Clinical Safety

URL: https://hl7.org/fhir/STU3/safety.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.9 Clinical Safety

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

This specification defines data elements, resources, formats, methods and APIs for exchanging healthcare data between different participants in the healthcare process. As such, Clinical Safety is a key concern with regard to the specification and its many and various implementations.

STU Note: This page, and the concept of safety in an API specification, needs further development.

Feedback is welcome here .

7.9.1 Implementer's Safety Check List

FHIR is as simple to implement as we know how to make it. However, due to the nature of healthcare, and healthcare processes, and cultural concerns, there are a number of features in FHIR that implementers are obliged to consider in order to implement safe systems.

This section is a check list to help implementers be sure that they've considered all the parts of FHIR that impact on their system design with regard to safety.

* Production exchange of patient or other sensitive data will always use some form of encryption on the wire

* For each resource that my system handles, I've reviewed the Modifier elements

* My system checks for modifierExtension elements

* My system supports elements labeled as "must-support" in the profiles that apply to my system

* For each resource that my system handles, my system handles the full Life cycle (status codes, currency issues, and erroneous entry status)

* My system can render narratives properly (where they are used)

* My system has documented how distributed resource identification works in its relevant contexts of use, and where (and why) contained resources are used

* My system manages lists of current resources correctly

* My system makes the right Provenance statements and AuditEvent logs, and uses the right security labels where appropriate

* My system checks that the right Patient consent has been granted (where applicable)

* When other systems return http errors from the RESTful API and Operations (perhaps using Operation Outcome), my system checks for them and handles them appropriately

* My system caters for parameters that have missing values when doing search operations, and responds correctly to the client with regard to erroneous search parameters

* My system ensures that system clocks are synchronised using a protocol like NTP or SNTP, or my server is robust against clients that have the wrong clock set

* My system ensures checks for patient links (and/or merges) and handles data that is linked to patients accordingly

* My system publishes a Capability Statement with StructureDefinitions, ValueSets, and OperationDefinitions, etc., so other implementers know how the system functions

Obviously this list is only a small part of the overall safety check list for an application, which will have checks regarding jurisdictionally mandated policies, internal integrity, etc.

In addition, server developers should check these specific additional checks for client convenience:

* Server: CORS (cross-origin resource sharing) is enabled (many clients are javascript apps running in a browser)

* JSON is supported (many clients are javascript apps running in a browser; XML is inconvenient at best)

* JSON is returned correctly when errors happen (clients often don't handle HTML errors well)

* the _format header is supported correctly

* Errors are trapped and an OperationOutcome returned

7.10 Appendix: HL7 EHR System Functional Model and FHIR

URL: https://hl7.org/fhir/STU3/ehr-fm.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.10 Appendix: HL7 EHR System Functional Model and FHIR

Electronic Health Records Work Group | Maturity Level: N/A | Ballot Status: Informative

IDO/HL7 10781 EHR System Functional Model Release 2 provides a reference list of functions that may be present in an Electronic Health Record System. While FHIR is an implementation focused on exchange of information in healthcare, this often happens in the context of an EHR system and EHR record. This table briefly describes one way that FHIR can be used to meet the requirements described in the EHR-S FM and is provided to help readers of the FHIR specification understand how FHIR can be used. There are many other equally valid ways to implement the EHR-S FM and to make use of FHIR.

EHR Function | FHIR Implementation Notes

TI.1 | Security | FHIR defines parts of the security infrastructure, and delegates others to standard web based security frameworks

TI.1.1 | Entity Authentication | FHIR assumes that the users are authenticated. OAuth is the preferred mechanism

TI.1.2 | Entity Authorization | FHIR does not currently provide any resources to describe or manage access-control permissions. By default, underlying web frameworks such as SAML would be used. See the security section for a discussion of binding between FHIR and SAML

TI.1.3 | Entity Access Control | See above about SAML / OAuth

TI.1.4 | Patient Access Management | See Security Labels

TI.1.5 | Non-Repudiation | The provenance resource tracks the timestamps, actors, and digital signatures associated with resources

TI.1.6 | Secure Data Exchange | TLS (https:) should be used for all production exchange of data. All conformant FHIR RESTful implementations SHALL be able to use TLS

TI.1.7 | Secure Data Routing | FHIR allows for brokers and various forms of messaging that support assured destinations and delivery (also see IN.2.2 below)

RI.1.1.4 | Information Attestation | See the provenance resource

TI.1.8 | Patient Privacy and Confidentiality | FHIR does not include functionality related to this requirement, though implementations would be expected to provide this

RI.1.1 | Health Record Information and Management | This is a core application of the FHIR capabilities

RI.1.22 | Data Retention, Availability and Destruction | A FHIR RESTful server gives precise and fine-grained control of retention, availability and destruction of resources, all clearly described by the capability statement

RI.1.1.x.1 | Auditable Records | FHIR provides the AuditEvent resource for auditable records.

RI.2 | Synchronization | FHIR supports synchronization using standard web publication/subscription methods via Bundles. Bundle-based pub/sub may be push or pull based, and can include all resources of a particular type, or selected subsets of the resources. In addition, groups of resources can be exchanged in bundles, keeping a set of related resources in synchronization

RI.1.1.13 | Extraction of Health Record Information | FHIR does not provide report formats, but does provide extensive search and retrieval functions to assist with building such reports

RI.1.1.1 | Store and Manage Health Record Information | A FHIR RESTful server can store and manage health information persistently - see below for further information.

RI.1.2.1 | Manage Structured and Unstructured Health Record Information | The dual contents of FHIR resources - structured data and XHTML narrative - provide seamless support for dealing with a mix of structured and unstructured information

TI.3 | Registry and Directory Services | The FHIR Administration resources provide registry-based access to patients, providers, etc.

TI.4 | Standard Terminologies and Terminology Services | FHIR encourages the use of standard terminologies wherever possible, and provides full support for their use through a variety of terminology related data types. FHIR defines a terminology service infrastructure. Also, see profiling, which discusses how terminology is used in a FHIR context

TI.5 | Standards-based Interoperability | FHIR is a definition of a standard on which to base interoperability

TI.5.1 | Interchange Standards | This is the core focus of FHIR. See below for discussion of interaction modes

TI.5.2 | Interchange Standards Versioning and Maintenance | FHIR version maintenance is described here

TI.5.3 | Standards-based Application Integration | FHIR enables simple integration through use of an easy to understand, use and debug web-based infrastructure. The same framework used within an EHR for persistence can also offer a simple way to implement exchange

TI.5.4 | Interchange Agreements | The FHIR Conformance Statement and Resource Profile resources provide a registry based infrastructure for individual trading partner agreements, as well as for community based ones

TI.6 | Business Rules Management | FHIR does not address this requirement at this point in time

TI.7 | Workflow Management | FHIR does not address this requirement at this point in time, though the resources and services exist to support this functionality

The EHR system functional model describes several modes for interaction between systems. Each of these can be implemented in several different ways using FHIR

Interaction Modes | FHIR Options

Unsolicited Notifications e.g. a patient has arrived for a clinic appointment | Create/update new resource via HTTP Push resources using Bundle Send FHIR Message (if appropriate event is defined)

Query/Response e.g. Is Adam Everyman known to the system? Yes, MRN is 12345678. | Search with parameters A query message (though not defined yet)

Service Request and Response e.g. Laboratory Order for Fasting Blood Sugar and a response containing the results of the test. | Could be supported either through Messaging or SOA solutions. Request/Response support is not yet defined

Information Interchange between organizations (e.g. in a RHIO, or in a National Health System) | Pub/sub using bundles (push or pull) RESTful interface FHIR messaging

Structured / Unstructured clinical document, e.g. dictated surgical note | See the Documents

The combination of a properly secured and managed FHIR server, along with enforced use of the AuditEvent and Provenance resources ensures that the core record management functions defined in the EHR-S FM are met (as follows). See the FHIR Record Lifecycle Event Implementation Guide for additional details.

* Lifespan/Lifecycle tracking, including capturing source, origination and authorship information, along with tracking of views and exchanges

* Attestation for accuracy and completeness, along with digital signature

* A full version history with content retention

* Retention and persistence

Additional functionality, not defined at this point in time in FHIR, is required to ensure non-repudiation, access control, and consent tracking.

7.11 Managing Resource Identity

URL: https://hl7.org/fhir/STU3/managing.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.11 Managing Resource Identity

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

Each resource has a known identity, which is a URL. The identity is not stored inside the resource, but must be tracked by systems handling resources. For RESTful systems, the resource identity is the same as the URL by which it is found. When a resource is packaged in a bundle (e.g. for messages, documents, or exchanged in a service), the id is included along with the resource. When systems using FHIR resources are deployed in an implementation context where there are more than two systems exchanging resources, the way that resources are identified with URLs needs to be managed carefully.

Note that many resources also have an identifier element, and usually there can be multiple identifiers. This identifier is a logical identifier for the concept that the resource describes, and is different from the identity discussed in this section, which is the literal identity of the resource. Note that there are often multiple resource instances that describe the same concept across multiple systems, e.g. each application keeps its own copy of the patient information in a Patient resource. This can even exist within a single system, such as in the case of patient duplicates).

7.11.1 Using Resources

Resources are used in a variety of circumstances. Generally, these can be categorized into 3 different scenarios:

* Closed Trading System: the resources are only ever exchanged between fixed systems in a tightly controlled community, such as a single EHR system with multiple modules. There is only one master server for each resource type, and resources are managed by that server. In this context, the logical id of a resource is sufficient to fully identify the resource

* Open system: there are many peer servers, each managing a set of resources of different types. In order to identify resources, a full URL reference to the origin server is required. The grand example of an open system is the World Wide Web

* Partially closed, inter-linked systems: a mixture of both closed and open systems - trading communities that are tightly managed, but have managed interactions with other closed trading systems, or with the world-wide web, or both. In fact, this combination appears to be the most likely scenario for current real-world healthcare business solutions

These combinations are why either relative (logical) or absolute references are allowed, and why a logical id is always required, in order to enable seamless exchange amongst partially closed trading systems.

7.11.2 Copying Resources and Re-identification

When resources are exchanged between systems, they may need to be re-identified (i.e. assigned a new logical resource id). When a resource is re-identified, nothing in the resource changes, but any references that point to the resource need to be updated. Whether re-identification is required or not depends on the context, as does how resource references are updated.

The normal case is that a client/receiving system accepts the server/sender's identification of a resource at face value, whether it is a relative or absolute reference. When the client/receiver wants to follow resource references, they are done using the server id (typically either by http calls or locating them in a bundle), but other arrangements are possible. In such cases, there is no need for re-identification.

Another scenario is for a client to retrieve a resource from a server, and make its own local persistent copy. If the local resource has a life-cycle of its own (i.e. it is not just a cached resource), then it needs to have its own identity; i.e. the resource must be re-identified. The simplest case is that the client is only keeping local copies of resources from a single

server. In these cases, the client can simply replace the Base URL and keep the logical id of the resource the same. In fact, if the server is using relative references, then this change doesn't involve any actual changes to the resources, it only means a re-interpretation of the references.

In some cases, however, the client may deal with multiple servers. In this case, the logical id of the resource is not guaranteed to be unique (unless all resources have a UUID for the logical id, which is allowed but not required). When the client cannot be sure that the resource identities are unique, it will have to re-identify the resources. In practice this means that the client needs to keep some kind of identity translation table, and update references to the resources it has copied locally when other resources are received.

The case of a gateway system that migrates resources from one ecosystem to another is very similar. In some limited cases, it can leave the logical id of the resources unchanged as resources are copied from one closed system to another. However, in more complicated cases, it will have to modify the resource references as resources pass across the gateway.

STU Note: In its current form, FHIR allows various convenient implementer practices around resource identification that make integration across eco-system boundaries more difficult. In particular, either only allowing UUIDs for logical IDs or only allowing absolute references would make the boundary management problem easier but remove useful and convenient flexibility for other uses of FHIR.

Feedback is welcome here .

7.12 Managing Push and Pull

URL: https://hl7.org/fhir/STU3/pushpull.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

7.12 Managing Push and Pull

The primary focus of FHIR is exchanging healthcare data between systems. Whenever one system has information that another needs or wants, one question that automatically arises is whether the source system will push the data, or whether the destination system will pull the data.

Push | As new information becomes available, or whenever it deems appropriate, the source system sends the information to the destination. It expects the destination system to maintain and index the information on receipt. The source system must trust the destination system to manage access/security appropriately

Pull | The source system maintains and indexes data. When the destination system wants or needs the information, it must retrieve it form the source system. Either the source or the destination (or both) manages access/security

Note that it is possible to mix these modes by adding an interface repository into the mix. For example, to connect a push-based source with a pull-based destination, simply have the source push information at the repository, and the destination can pull data from it as required. If the source is pull based, the interface repository can query it for new data on a regular basis, and when it finds any, it can pull it into the repository and/or push it to the destination.

Which combination of push, pull and interface repositories is appropriate for any given context depends on a host of factors around deployment architecture, network and business topology, security policies, etc. However, it is often constrained by the capabilities of the various systems and the standards used in the exchange.

The FHIR specification supports and enables both push and pull:

* REST: The REST API can be used in either fashion - this is discussed in more depth below

* Messages: FHIR defines message events for both push and pull

* Services: Services may be defined to support either model

* Documents: This question doesn't apply, because documents are bundles that are exchanged using either REST, messages, or services

7.12.1 Push and Pull on a RESTful interface

* Push: The source is a client, and when new data is available, it uses create, update, and transaction to push data to the destination, which is a server

* Pull: The source is a server, and the destination is a client. When the client wants to use the data, it uses a combination of search and read so the users can find the data they need

* Push/Pull: The source is a server, and the destination is a client. On a regular basis, the destination queries the server for its history, and replicates the changes in its own copy of the database

7.13 Variations between Submitted data and Retrieved data

URL: https://hl7.org/fhir/STU3/updates.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.13 Variations between Submitted data and Retrieved data

FHIR Infrastructure Work Group | Maturity Level: 5 | Ballot Status: Trial Use

Some of the time when using a FHIR interface, requests to create or update resource instances will behave exactly as the initiator requested. The desired record(s) will be created or revised within the target system and a subsequent query of the data would show the exact same information as was submitted. However, FHIR systems are not guaranteed to behave this way. Without any other agreement between exchange partners, FHIR systems are not obligated to store and return data as it was received. In fact, for some interoperability paradigms, they're not obligated to store any data at all. This page discusses some of the considerations around system behavior, including differences in expectations for systems interoperating using REST, messaging, documents and services.

7.13.1 Data element support

The most common reason for differences between what data is submitted to a system and what data can be extracted from it is that the system doesn't support all of the data elements present in the instance received. In the base resource, no systems are required to support any particular set of extensions or even any particular subset of core elements.

For elements that are part of the resource, the expectation is that "most" systems will support the element. i.e. most systems will support capturing a patient's name, gender and date of birth. But "most" does not mean "all". It is possible to be fully FHIR conformant and claim to support the Patient resource, but be incapable of storing any of those data elements. There are use-cases where names may be unnecessary (e.g. agricultural veterinary systems, anonymized reporting, etc.) and similar use-cases for almost every data element. No FHIR resource elements start off as mustSupport and very few resource elements start off with a minimum cardinality other than "0".

In the case of extensions, the very nature of a concept being an extension means that the designers of the specification expected that fewer than "most" systems would support the element, though support might vary widely by context. An extension might be used by 100% of systems in some country, discipline, etc. and not used by any systems in another context. In any event, there is no guarantee that an arbitrary receiver will recognize and be able to persist any given extension.

In order to know whether a particular data element is likely to be stored by a given server, a client should check the Capability statement of that server. If, for a given resource, the StructureDefinition pointed to indicates that the element or extension is "mustSupport=true", and the server is capable of storing and returning data in general, then it would be expected that the system will be capable of storing and returning that data element. (Some servers such as decision support systems might not be capable of storing or returning any received data.)

All of these concerns around possibly not storing resource elements or extensions can hold whether the data is sent using REST, Messaging or Services. However, with documents, a consuming system is expected to accept the entire contents of the document without losing any information or altering it in any way.

7.13.2 System behavior

Even if a system supports all of the data elements provided, not all systems will actually persist the data received or be capable of returning it in response to a query. mustSupport indicates that a system supports an element but does not prescribe exactly what the system must do with supported elements. Data might be persisted, displayed, relayed, analyzed, tabulated or used in a variety of other fashions. The behavior of a given system should be unsurprising given its context, but it is still important to recognize that not all systems will persist the data they receive.

7.13.3 Access permissions

Even if a system stores a given data element, that does not mean it will always include that element when responding to queries. Systems will have access permission rules that restrict who can see a given resource instance and, occasionally, who can see a particular data element within a resource. Systems responding to queries might suppress records or may adjust the content of resource instances to exclude data elements the querying system is not permitted to see.

7.13.4 Generated and inferred data

Some servers may add additional data elements (or more commonly, extensions) based on information they have generated or inferred from data in the resource, from other resources or other information of which the server is aware. i.e. an instance queried after being created or updated might have more information present than was included on the originally submitted record

7.13.5 Data integration

Servers receiving updates from multiple sources may choose to be selective about what sources they choose to trust for updates to certain information. For example, a patient registry system might choose to only allow updates to name, gender and date of birth from administrative systems but not clinical systems. If a system filters out patient address information from being disclosed to a system when it queries, it will likely choose to not replace or eliminate the addresses it has on file when it receives an update from that system.

One approach commonly followed by HL7 version 2 messaging interfaces is to not update any elements not included in an instance. For example, if a Patient instance were received with no telecom or contact information, all existing telecom and contact information would be retained and only those elements included in the instance would be updated. This same approach can be followed with FHIR, however, unlike HL7 V2 , FHIR does not have a defined construct to use to indicate that a particular data element should explicitly be set to empty. Systems wishing to fully emulate HL7 v2 behavior will need to use an extension to mirror this behavior.

7.13.6 Ramifications of storage/retrieval variations

While changing data prior to storage or prior to returning query results is possible, it should not be considered "normal" behavior. It creates several challenges:

* Digital signatures will no longer be valid. Any change to submitted data will break standard signatures. In theory, this could be mitigated by using a custom canonicalization for the signature that excludes malleable elements. However, this would require both sender and receiver to agree on the alternate signature canonicalization. The signature would still be considered invalid by systems that were not party to the agreement.

* Workflows can be made more complex. A system that sends a create and then sends an update presuming that it has seen the most "current" version, could accidentally overwrite information set by the server as part of the create process.

* Audit becomes more complicated. The AuditEvent resource points to a specific resource version when tracking what data was returned by a query. If the record is modified when it is returned, then the audit record will likely need to be supplemented with information about what elements were filtered.

* Multiple views of the same information by different people (or even by the same people in different contexts) may cause confusion, particularly when attempting to coordinate care across a team. Providing OperationOutcome instances containing warnings if information has been suppressed may be helpful, though doing this may not be possible, depending on security policy rules.

* If data is lost as part of the transmission process, then multiple stages of transmission (or even round-tripping of data) could result in significant degradation as information passes through multiple systems.

7.13.7 Mitigation approaches for storage/retrieval variations

FHIR does provide a couple of mechanisms that can help with the issue of a system that has received only partial data overwriting data that was filtered from its record:

* For systems that enforce the use of ETags for updates, the server can choose to not include an ETag when returning resource instances that have been modified in some way. This will prevent updates from being performed.

* The SUBSETTED Security Label can be used to flag data that has had information removed.

Note that using either of the above mechanisms may constitute a security breach in circumstances where the individual or system accessing the data should not know that the record being reviewed has been modified in any way. (The presence of digital signatures would present a similar issue.)

7.14 Integrated Examples

URL: https://hl7.org/fhir/STU3/integrated-examples.html

Bronchoscopy; several biopsy specimen were taken from pathological mucosa, right main bronchus specimen send for pathologic analysis. Bronchoscopy because of atelectasis right. X-thorax; increase in atelectasis and pleural liquid. Bronchoscopy; -fluids drained from right main bronchus -pathalogic mucous membrane right bronchus, easily bleeding -left bronchial system open Antiobiotic policy with retropharyngeal abscess with prolapse to the mediatinum. No surgical possibility to fully drain the mediatinum. Neck; swelling and redness pretracheal extending to chest. No fluctation, however induration is present. Swelling back pharynx, also present in postcricoid area. Light stridor sound when breathing. Overall condition is good. Scoop; little supraglottic swelling, vocal chords not judgable. CT thorax: no mediastinal/retropharyngal collering of the fluid collections. Diffuse edema retro/parapharyngeal and mediastinitis. Preoperative culture shows; S pyogenes, sensitivity for peneciline and E. cloacae, sensitivity for vancomycin. Stop vancomycin prescription. Continue penicilin and start vancomycin supplementation. Total treatment time is approximatly 6 weeks. In consultation with the IC/anesthesia; intubation. Analyse lower respiratory tract. Extubation on OR. Normal voice, no stridor. However, hypoxic at low respiratory tract obstruction with 84% O2 and rapid breathing. Roel Bor is in his mid-fifties, works for an IT-company and has two healthy children and a wife. His uncle unfortunately was less healthy and died of cancer. At the end of 2012, Roel was also diagnosed with a tumor in the Erasmus Medical Center. His tumor is located in the head-neck area. The standard treatment for that is TPF-chemotherapy. The tumor is not fully curable due to its position, but it was optimally minimized with the therapy in the AUMC. Two severe complications followed the treatment, namely bacterial sepsis (streptococcus aureus) and renal failure. Both were stabilized within a short period through medication (see care plan). Roel thereafter had his ups and downs with the last known condition of a severe fever, for which he received among other treatment (paracetamol). On a side-note and as a final statement: the patient suffers from house dust allergy.

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.14 Integrated Examples

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

These integrated examples are the result of choosing several real world healthcare records from a Dutch healthcare institution and representing them using FHIR resources. They are made published here for several reasons:

* To test that the resources are fit for purpose using real cases

* So that the available resource examples include real cases

* So that applications have real world clinical cases to use in development and prototyping

7.14.1 Patient case 1

The 69 year old Pieter van den Heuvel is diagnosed with several serious diseases over the last few years. In the spring and summer of 2011, Pieter had complaints of fatigue, dyspnea and even fainting. After visiting his physician, Pieter was referred to the cardiologist for further examination. At the hospital a malfunction of the heart valve was diagnosed and eventually replaced. Pieter recovered well in the next 2 months, but probably needs a life time prescription on blood pressure reducing medication. In the summer of 2012, Pieter again visited his physician for a chest injury after he fell from the stairs. By coincidence, the doctor noticed a suspicious stain on the X-thorax and performed some additional tests on Pieter. After the results, Pieter was diagnosed with early stage non-small cell lung cancer. Fortunately, the tumor didn't spread to other parts of the body. Tumor type and location made it possible to perform a partial lobectomy in the right lung. After his recovery, Pieter was submitted to a high risk control group for yearly screening.

In 2013, Pieter was charged with an increasing neck swelling, a stiff neck and difficulty swallowing. Because of breathing problems, Pieter was urgently admitted to the hospital. An upper respiratory infection caused a retropharyngeal abscess in the back of the throat. Surgery was needed to remove the abscess. The surgeons recommended a tracheotomy during the surgery, so there was less change of complications during the abscess removal. A tracheotomy was intubated before the surgery. After the removal of the retropharyngeal abscess, the tracheotomy was extubated as the swelling was reduced and normal breathing became reinstated.

7.14.1.1 Patient demographic information

```
Patient name: Pieter van den Heuvel
Patient number: 0108173
Date of birth: 17-11-1944
Gender: Male
```

7.14.1.2 Patient contact information

```
Address: van Egmondkade 23
Zip code: 1024 RJ
City: Amsterdam
Phone: +31648352638
email: p.heuvel@gmail.com
```

intake date | surgery date | practitioner | specialism | diagnose | procedure | care plan | surgery code | SNOMED CT code | related encounter
26-6-2011 | 27-6-2011 | P. Voigt | CTC (Cardio-thoracale Chirurgie) | Heart valve disorder | Heart valve replacement | CP2903 | 1000263502 | 64915003 | v1451
6-7-2012 | 7-7-2012 | M.I.M. Versteegh | CTC (Cardio-thoracale Chirurgie) | NSCLC - Non-small cell lung cancer | partial lobectomy | CP2934 | 1000263813 | 173171007 | v3251
22-3-2013 | 22-3-2013 | A.P.M. Langeveld | ENT | Retropharyngeal abscess | Trachea-tracheotomy | CP2938 | 1000050465 | 48387007 | v6751
24-3-2013 | 24-3-2013 | E.M.J.M. van den broek | ENT | Retropharyngeal abscess | retropharyngeal abscess | CP3953 | 1000049161 | 172960003 | v6751
27-3-2013 | 27-3-2013 | R.J.P. Briet | ENT | Retropharyngeal abscess | Trachea-extubatie | CP1283 | 1000050159 | 309812005 | v6751

device | hopital/home | dose | admission route | start date | presciber | SNOMED CT code
Feeding tube | hopital | N.A. | tube | 10-4-2013 | R.A. van den Berk | 61420007

date | substance | value | status
2013-04-02 | glucose | 6,3 mmol/l | abnormal
2013-04-02 | base excess | 12,6 mmol/l | abnormal
2013-04-02 | carbon dioxide | 6,2 mm[Hg] | abnormal
2013-04-02 | erytrocyten | 18,7 g/dl | abnormal
2013-04-02 | Hemoglobin | 7,5 g/dl | abnormal

7.14.1.3 Anamnese

date: 8-6-2012

date: 18-3-2013

7.14.1.4 Physical investigation

date: 18-3-2013

7.14.1.5 Additional research

7.14.1.6 Policy

7.14.1.7 Conclusion

7.14.2 Patient case 2

```
Patient name: Roel Bor
Patient number: 123456789
Date of birth: 1960-03-13
Gender: Male
```

7.14.2.1 Patient contact information

```
Address: Bos en Lommerplein 280
Zip code: 1055 RW
City: Amsterdam
Phone: +31612345678
```

practitioner | organization | careplan | start date | substance | condition | other participant | procedure | encounter ID | surgery code | SNOMED CT code

A. Bronsig | Artis University Medical Center | CP3928 | 2013-03-11 | potassium | fever | Carla Espinosa | | 20130404 | 100028475 | 64915003
A. Bronsig | Artis University Medical Center | CP7364 | | | malignant tumor | | chemotherapy | 20130128 | 100028475 | 363346000
A. Bronsig | Artis University Medical Center | CP8766 | 2013-03-11 | | Bacterial sepsis | Luigi Maas | observation | 20130311 | 100028475 | 363346000

practitioner | date | encounter | reason | condition | medication | medication prescription | procedure | diagnostic report | issued by
A. Bronsig | 28-1-2013 | 20130128 | chemotherapy | malignant tumor | | | chemotherapy | BUMC
A. Bronsig | 4-3-2013 | 20130404 | fever | fever
A. Bronsig | 11-3-2013 | 20130311 | sepsis | Bacterial sepsis | | AUMC
A. Bronsig | 11-3-2013 | 20130311 | renal | Renal insufficiency | | AUMC

practitioner | date | observation | value | status | SNOMED CT code
Luigi Maas | 11-3-2013 | Blood culture for bacteria | Gram-positive bacteria | positive | 8745002
A. Bronsig | 4-4-2013 | temperature taken | 39 degrees Celsius1 | high | 89003005
A. Bronsig | 4-4-2013 | Serum bicarbonate measurement | 28mmol/L | normal | 271239003
Luigi Maas | 4-4-2013 | Serum creatinine raised | 122 umol/L | high | 166717003
Luigi Maas | 4-4-2013 | Epidermal Growth Factor Receptor | Negative for EGF receptor expression (Non-small cell lung cancer) | negative | 427038005

7.15 Appendix: The Relationship between FHIR and other HL7 Standards

URL: https://hl7.org/fhir/STU3/comparison.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.15.0 Appendix: The Relationship between FHIR and other HL7 Standards

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

Health Level Seven (HL7) has been developing healthcare information exchange and related standards since 1987. In that time, the organization has produced a number of standards families - many used throughout the world to automate healthcare data sharing and improve patient care. FHIR has been written to be implementable without any knowledge of these other specifications. However, FHIR does leverage this prior experience, both in terms of applying best practices learned from experience and attempting to avoid some of the pitfalls of earlier work.

This appendix describes the relationship of FHIR to some of HL7's other standard families. It may be of interest to those coming to FHIR with previous experience with other HL7 standards as well as those who may need to support interoperability between FHIR solutions and implementations of other HL7 standards.

In this appendix:

* HL7 v2

* v3 (RIM / messaging)

* CDA & CCDA etc.

* Other HL7 standards

Notes:

* In addition to the major standards families identified below, HL7 produces numerous implementation guides, some of which have earned as much prominence as a standard family themselves. The general guidance given below for each standard family should hold for all implementation guides based on that standard. For example, the guidance listed for CDA would apply to Consolidated CDA (CCDA) and other CDA Implementation Guides.

* While this appendix focuses on the relationship between FHIR and other HL7 standards, relationships also exist to non-HL7 standards. Some resources provide direct implementation of functionality from other standards including DICOM (see the ImagingStudy resource) and IHE (e.g. the AuditEvent and DocumentReference resources).

* Many FHIR resources draw requirements from or provide mappings to other standards. Some resources also provide additional guidance on how to use them with external specifications as part of their implementation notes. As well, a wiki page to capture additional guidance on the use of FHIR with other specifications can be found here

* FHIR can satisfy the needs covered by all of the previous primary HL7 interoperability standards (HL7 v2 , HL7 v3 and CDA). In many cases, it also provides additional benefits in terms of ease of interoperability. Therefore, the possibility exists that FHIR could gradually replace some or all of these standards. However, it is unclear how rapidly (or even if) the market will make such a migration. It is likely that most of these standards will exist in parallel for quite some time. HL7 has committed to ongoing maintenance of existing standards for as long as the HL7 membership requires.

7.15.1 HL7 Version 2

URL: https://hl7.org/fhir/STU3/comparison-v2.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.15.1 HL7 Version 2

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

HL7 v2 was HL7's first information exchange standard and is one of its most widely adopted, being prominent in in-patient settings throughout the world, though also used in a variety of other contexts as well. HL7 v2 uses messages composed of re-usable segments to communicate healthcare-related information between a sending and receiving system as well as to invoke particular behavior (patient transfers, lab orders, etc.) It also supports one-way communication through notifications, provides support for queries and other workflows.

7.15.1.1 HL7 v2 Similarities and Differences

Event-based: FHIR supports an event-based messaging paradigm similar to the HL7 v2 messaging structure (though unlike HL7 v2, FHIR supports other paradigms as well including documents, REST and other service models). Refer to the Message Header resource.

Granularity: HL7 v2's "Segment" structure provides re-usable chunks of data that roughly correspond to FHIR's idea of resources. However, HL7 v2 segments can't be independently manipulated. Additionally, not all segments have the characteristics of independent identity held by FHIR resources. Due to differences in scope and approach to extensibility, HL7 v2 segments and data types are frequently cluttered with data elements that are not used by (or even understood by) the majority of implementations.

Segments can be composed into repeating and/or optional collections called "groups" to represent full healthcare business objects. For example, the "Order" component of an OMP (Pharmacy/Treatment Order Message) includes:

* an ORC segment dealing with workflow aspects of the order

* an RXO segment dealing with pharmacy-specific order aspects

* optional TQ1 and TQ2 segments describing the timing of the order

* optional NTE segments dealing with supplemental notes or rendering of the order

* optional RXR segments describing route information

* etc.

The HL7 v2 approach to granularity emphasizes re-use of "patterns" of information. For example, timing and route information are not useful on their own, but they are useful in many circumstances. Due to the 3-level nesting limit, separate segments are also required for data structures that would otherwise nest too deeply. FHIR takes a different approach to reusability, focusing on objects that can be maintained independently. The MedicationRequest resource encompasses all of the aspects of the above segments, with the exception of some of the workflow aspects of ORC which is handled by the Task resource. The MedicationRequest resource is itself complex, having nested structures for dosage instructions, dispensing instructions, etc. that are not simple data types.

Extensibility: HL7 v2 provides an extensibility mechanism through the use of "Z-segments". The meaning of these extensions is opaque without prior manual explanation by the sender. Extensions are supposed to be restricted to data elements that do not affect the meaning of the "standard" segments. FHIR Extensions, on the other hand, can appear at any level (including within data types). ModifierExtensions may be used in circumstances where an extension can change the meaning of other elements (e.g. the introduction of a negation indicator on a record). Finally, the meaning of FHIR extensions is discoverable by resolving the URI that defines the extension. The URI approach also ensures that extensions created by independent systems won't collide. (This can be an issue with Z-segments.)

Inter-version compatibility: HL7 version 2 has strict processes for maintaining forward and backward compatibility. Content can only be added to the end of existing fields, components, etc. Applications are expected to ignore unexpected content or repetitions. FHIR promises similar compatibility rules. The path to an element within a FHIR instance will remain unchanged in future versions. Specific rules on handling "new" elements (ignoring, checking for "must understand" indicators, etc. will be developed during the STU period.

Human readability: In general, HL7 v2 instances do not provide for human readable versions of the content exchanged. While some systems may make use of NTE segments to provide a human-readable rendering of all or part of a message payload, the rules for when or if this occurs is site-specific. FHIR requires human readable content to be provided for each resource.

Update behavior: HL7 v2 data is typically exchanged in "snapshot" mode - updates are communicated by sending a complete copy of the instance with the new data filled in. However, some segments and messages in HL7 v2 support more sophisticated exchanges where only changed data is sent and codes or special values indicate what sort of change is to occur (e.g. add this address, remove this name). Out-of-the-box, FHIR only functions using snapshot mode. While the use of ModifierExtensions to introduce equivalent behavior to HL7 v2 is possible, doing so would create interoperability issues and would make use of the resources difficult outside the messaging paradigm.

Optionality & Profiles: Both HL7 v2 and FHIR provide a similar degree of flexibility at the international standard level. Most data elements are optional. However, there are two differences. FHIR resources are much more limited in terms of what elements are included in the core specification - only those elements that the vast majority of systems will support. HL7 v2 tends to include many elements that are used in only very limited circumstances. FHIR uses extensions for those circumstances. HL7 v2 and FHIR both provide formal mechanisms for defining profiles to give guidance on the use of the specification. However, the HL7 v2 mechanism has not been widely used. FHIR Profiles form an essential component of the methodology and are built into tooling, increasing the likelihood of their use.

7.15.1.2 HL7 v2 Interoperability Considerations

Mapping: One of the biggest challenges with HL7 v2 interoperability is the variation of implementation. Even when identical scenarios are being handled in similar business environments, the data elements supported can vary and even the place

where a given data element is placed in an instance can vary. As a result, defining consistent mapping rules between HL7 v2 and FHIR at an international or even regional level is not terribly realistic. The FHIR mappings provided give a starting point for consideration, but mappings will generally need to be done on an implementation by implementation basis.

Extensions: While some HL7 v2 elements will map to FHIR core, a large percentage will not. Where a HL7 v2 element is not supported by core, an extension will be needed to share the information. Where there is interest, HL7 may choose to publish and maintain extensions for HL7 v2 elements that are not supported as part of the core FHIR specification. The FHIR extension registry should be searched prior to defining local extensions. If time permits, the relevant HL7 WG should be contacted with a request to define additional HL7 v2 extensions if needed ones are not present. If time does not permit, applications can define their own extensions, but should have a migration plan for if/when HL7 defines it later. For Z-segments, URIs should be defined to be specific to the system/environment that defined the Z-segment (e.g. http://acme.org/fhir/extensions/consent), not based on the name of the Z-segment itself (given that Z-segments with the same name but different meaning may exist) (e.g. http://hl7.org/ZAC).

Resource identification: HL7 v2 messages will often reference objects that have already been referred to in previous messages. When converting the messages to FHIR, these references will need to point to the same resource URI. Given that not all HL7 v2 message objects have identifiers in the message, this can be somewhat problematic. An approach to handling this issue exists for FHIR transactions. However, the ramifications of using this approach in a messaging environment have not yet been resolved. Implementers will need to explore their own strategies as part of early adoption.

Merging references and resources: HL7 v2 message instances may well reference the same "object" numerous times. For example, a message containing a patient's medication history is likely to include references to the same clinicians and clinics/hospitals many times. While in some cases, the data captured for a given object might be identical in all uses, in other cases the information might vary. For example, the sending system might convey historical phone numbers for old records and current phone numbers for newer records. Alternatively, the message design might allow expression of different amounts of detail in different portions of the message or the sending application might simply be designed to convey different amounts of detail in different portions of the message (e.g. conveying phone number for an ordering clinician, but not for a data-entry clinician). When converting to FHIR, all references to the same "object" will generally have a single resource identifier and be referenced only once in the instance - with the complete set of information needed/available. This creates two challenges:

* How does the conversion software recognize when two portions of a message are referencing the same object? While some references may have unique identifiers or names that are sufficient to confirm "same object", others may not - though some other combination of attributes may be sufficient. The specific rules will need to be determined by the implementer performing the conversion

* If multiple versions of data are present, what set of data should be used - or should multiple versions be sent with distinct history ids? (And if the latter, what is the 'order' of the versions? If the order of the versions can be determined by data in the message (e.g. assuming older order dates have "older" demographics), dates can be specified on the entry updated element to indicate relative ordering. If the ordering can't be determined, it will be difficult to merge the data into a single resource or represent it using multiple resources.

Identified vs. Contained resources: Each HL7 v2 message will be mapped to multiple resource instances - often 10s or even 100s of resource instances. To maintain consistency with the HL7 v2 messaging paradigm, all resource data will typically be sent over the wire as part of the FHIR message rather than being sent by reference as would be typical in a RESTful implementation. However, FHIR provides two different ways of communicating the resources as part of the message bundle: they can either be sent as "fully identified" resources (direct entries in the bundle with their own identity, and able to be the subject of independent transactions), or they can be sent as contained resources, meaning they are only identified relative to another resources and cannot be retrieved or otherwise manipulated on their own. A HL7 v2 to FHIR conversion process will need to make the determination of what data elements are or must be present, for a resource to be fully identified. In some cases, the determination will be done at the time of mapping. In other cases, it may depend on the content of a particular instance. As an example, an XCN containing just a name (|^Smith^John|) doesn't contain enough information to identify the physician from any other John Smith, so will need to be contained resource, whereas an XCN of |12345^Smith^John| generally does, though the conversion process will need to be aware of the scope and management processes around the identifier.

Generating human-readable content: FHIR requires that every resource have a human readable narrative that contains all information relevant to human decision-making. When converting from HL7 v2, developers (likely with guidance from clinicians) will need to determine what information from the message should be rendered and how to generate this content.

Nulls and update modes: In HL7 v2, "action" codes can determine whether particular segments represent information to be added, updated or deleted. Fields can be populated with "null" (two consecutive double-quotes with no other characters) to note a field is to be deleted. An omitted element or repetition is generally interpreted as "retain existing data unchanged". This contrasts with the FHIR approach of requiring all data to be present as a snapshot. Systems will either need to build in logic to generate a full snapshot of each resource or consider using the Patch Operation instead.

7.15.2 HL7 v3 (and ISO 21090)

URL: https://hl7.org/fhir/STU3/comparison-v3.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.15.2 HL7 v3 (and ISO 21090)

HL7 v3 was intended to be the next generation of HL7's messaging standards. It introduced a common Reference Information Model (RIM), data type model and set of vocabulary as well as a formal standards development methodology. In addition, it introduced the use of "documents" as an alternative architecture to messaging for sharing healthcare information (see the CDA comparison). While nominally covering both, the term "v3" is typically used to refer to "v3 messaging". The data types used as a basis for v3 have also been adopted by ISO as ISO 21090. The HL7 RIM has also been adopted as an ISO standard.

v3 messaging has been adopted by a number of large projects, particularly in the electronic health record area, though it has not achieved the market penetration of HL7 v2 . The HL7 RIM and the ISO 21090 data types have also been used by other SDOs and projects that have not leveraged the full HL7 v3 methodology. Most of the comments and guidance provided here will apply to those solutions as well.

7.15.2.1 v3 Similarities and Differences

Reference model: The use of the HL7 RIM is a core aspect of the HL7 v3 methodology and it is front and center in the specification and the wire format. All data elements in HL7 v3 instances are derived from either the RIM or the ISO data types. In FHIR, this is true of most resources and data type elements, but not all. Some resources (StructureDefinition, CapabilityStatement, ValueSet, etc.) deal with content that is outside the RIM's scope. And in a few circumstances, adjustments have been made in the FHIR data types that are not yet supported in the HL7 v3 data types model. The expectation is that these changes will be supported in the next version of the v3 data types model. The main difference is that the serialization format of FHIR is not driven by the RIM mappings. This results in considerably more concise and intuitive instances. It is possible to implement FHIR with absolutely no knowledge of the HL7 RIM.

Codes: v3 places considerable reliance on coded attributes to convey the meaning of instances. Examples include classCode, moodCode, determinerCode, etc. The allowed codes for these attributes are strictly controlled by HL7. FHIR also has attributes that are limited to codes defined in the FHIR specification - those using the code data type. However, these are generally limited to attributes with business meaning - status, contact types, etc.

Both FHIR and v3 make use of value sets to define the sets of codes that can be used for attributes within particular contexts. However, in FHIR, a ValueSet is just another type of resource, meaning it can be sent as part of an instance just like any other piece of data. (The same is true of StructureDefinition, CapabilityStatement and other meta-level resources.)

Granularity & referencing: HL7 v3 models are broken into 3 main types - wrappers, payloads and Common Message Element Types (CMETs). These are combined into interactions to define the set of content that can be sent over the wire at one time. In some cases, the granularity of each of these models will exactly align with the granularity of FHIR resources, but not always. v3 models are divided based on the expectation of re-use. FHIR models are divided based on whether the objects they represent can be considered to "stand alone". In HL7 v3, numerous models can exist to represent the same essential underlying healthcare information construct. For example, at the HL7 International level, there are 10 different CMETs for the concept of "Patient". In addition, some payload models represent patient directly without using CMETs. Further variation exists in the v3 models created by HL7 affiliates and other v3 implementers. Each of these different CMETs has their own schema and may use different element names, different levels of nesting and different constraints. With FHIR, there is only one Patient resource. Many profiles can be created on that resource, but all of them will use the same schema and support the same serialization format.

Design by constraint: The design methodology in v3 is one of "design by constraint". The idea is that all data needed for any sort of healthcare communication is represented in the HL7 RIM. All other data models simply constrain the RIM to reflect the needs of particular domain spaces. This starts at the international level with further refinement happening in individual countries, projects and finally specific implementations. As models become closer to the implementer, they become less abstract. The result is a tendency for v3 models to be extremely broad in their coverage and capability and somewhat abstract. They need to be this way to ensure that all possible implementations in the space covered by that model can be properly constrained. As well, each model produces its own schema and, in most cases, constrained schemas are not strictly wire-compatible with the schemas of the model being constrained.

FHIR takes a different approach. FHIR resources do not attempt to represent all data elements that could possibly be used in a space. Instead, only those data elements that are expected to be used by "most" implementations within the scope of the resource are considered part of the core resource definition. (This is sometimes referred to as "The 80% rule" - if approximately 80% of systems maintaining the resource will support the element, then it is part of core). All other data elements are expected to be handled using extensions. Profiles are used both to constrain resources and to define extensions appropriate to narrower implementation spaces. Serialization format interoperability is retained across all profiles on a given resource.

Context conduction: When conveying healthcare information between humans, much data can be inferred from context. For example, if a report has an "author" noted on a cover page, it is generally inferred that each statement within the report is authored by that same person. This inference grows more challenging when data needs to be analyzed by computers, whether for query, decision support or other analysis. Thus far, the HL7 v3 methodology has provided three distinct mechanisms to allow data models to define how "context" should propagate through models, making explicit for computers what humans would normally understand intuitively. FHIR has chosen a different path. In FHIR, no context is conducted - everything is explicit. If a report about a patient contains 100 observations all about that same patient, each observation will include a reference to the patient. However, this is relatively painless because it's only a reference - an id and possibly a short display value. One of the benefits of this approach is that each resource can be safely consumed and examined without concern for the context in which that resource was communicated. The meaning of each resource instance is fully

self-contained.

Null flavors: In healthcare, it's quite common for data to be unknown, unavailable, have an exceptional value or otherwise fall outside the bounds of a "normal" value. To deal with this, v3 introduced the concept of "null flavor" on almost every attribute and data type property in its models. These coded null flavors could be sent in place of or in addition to the data that would typically be sent for the attribute, association or data type property. Examples include the ideas of "Unknown", "Not asked", "Positive infinity", "Trace amount", "Masked", "Other", etc. Unless an element was explicitly marked as "mandatory" - meaning no null flavors were permitted, these null flavors could appear anywhere.

FHIR approaches the problem differently. Null flavors are only introduced in the core specification in those circumstances where it is expected that most systems will need them. Where needed, the flavors are constrained to those relevant to that element.

7.15.2.2 v3 Interoperability Considerations

Using RIM mappings: Most resource elements and data type properties include mappings to the RIM. These mappings serve two purposes. They help to define FHIR semantics in terms of HL7's reference models, helping to ensure that the Work Groups defining the data elements have a good and consistent understanding of the meaning of every element. They also provide guidance for implementers of v3 specifications that may be looking to migrate to or map between v3 and FHIR. However, for the latter use it's important to understand some limitations on the RIM mappings. The RIM is a language which allows the same "idea" to be conveyed in a several ways with varying granularity and expressiveness. Thus, it's entirely possible for a RIM element to map to a core FHIR element even though its RIM representation is somewhat different than described in the mapping. In addition, not all v3 models adhere to good modeling practices, so some data elements that would appear to map to a FHIR element might not map if the information has not been well represented. Therefore, RIM mappings should be taken as a guide, not an absolute, and mappings must be done in the context of the v3 specification being mapped. (Also, see Abstract models below.)

v3 extensions: While the core of the v3 methodology is "design by constraint", it still makes provision for the use of extensions - either in a foreign namespace or denoted by a special attribute. When converting between v3 and FHIR, the use of such extensions will need to be taken into account. As a rule, most v3 extensions will map to FHIR extensions, as the v3 design-by-constraint principle suggests that anything that would qualify as "core" in FHIR would already have been part of the base v3 specification.

Abstract models: As previously noted, many of the v3 models created at the HL7 International level are quite abstract. As a result, the models can be used to say a wide variety of things, often in a wide variety of different ways. This makes defining a mapping between those specifications and FHIR (or any other specification) quite tricky. For practical v3 <-> FHIR interoperability, mappings will need to be created at the level of message specifications, implementation guides and/or templates that are more concrete and closer to the implementation level. For example, mapping all of CDA to FHIR would be impossible given the expressive capability of the right-hand-side of the CDA model. However, mapping the Consolidated CDA (CCDA) templates to FHIR is quite possible.

Context conduction: As discussed above, HL7 v3 models rely on context conduction - either implicitly or explicitly controlled. When converting to FHIR, the context will need to be propagated into each resource.

Update mode: In HL7 v3 instances, updates are generally handled in snapshot mode, similar to the FHIR approach - if any information changes, the entire record is sent, including the modified data elements. However, the v3 methodology does support the introduction of an "updateMode" property to allow only the changes to be sent for all or part of an instance. Each element repetition is flagged with an updateMode to indicate whether the element is to be added, removed, updated, etc. Additional updateModes allow further control over updates. As with the V2 discussion above, implementers will need to generate a full snapshot of each resource or consider using the Patch Operation instead.

Additional considerations: Most of the implementation considerations for interoperating between FHIR and HL7 v2 also hold with v3. Specifically: Extensions, Independent vs. Contained resources, Resource Identification, Merging references and resources and Generating human-readable content.

7.15.3 Clinical Document Architecture (CDA)

URL: https://hl7.org/fhir/STU3/comparison-cda.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.15.3 Clinical Document Architecture (CDA)

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

CDA is HL7's most widely adopted HL7 v3 standard. It provides both a standardized header containing metadata about the document as well as the ability to convey a wide variety of clinical content organized into various sections. The document content can be un-encoded, such as a PDF through to a fully encoded HL7 v3 instance.

NOTE: While FHIR can be used to create documents using the Composition Resource, FHIR can also be used to exchange traditional CDA R2 documents making use of the DocumentReference resource, and handling the CDA document itself as a binary attachment (as XDS does).

7.15.3.1 CDA Similarities and Differences

Clinical document focus: As its name implies, Clinical Document Architecture is limited to "clinical" use cases. The CDA model does not support exchange of content not deemed to have clinical relevance, such as financial information and is limited to documents that deal with patients. (In some cases, such as the HL7 Structured Product Labeling standard, non-patient-specific CDA-like specifications are created to get around this limitation.) FHIR documents have no limitation on their content and can have subjects other than patients.

Human readability approach: CDA and FHIR both require that content be human-readable and define specific rules for how the human readable text is presented.

Clinical Statement vs. resources: In CDA, the "content" of the document is expressed using a complex and extremely abstract model based on HL7's "Clinical Statement" project. Its purpose is to allow implementers to express pretty much any clinical concept in any degree of rigor and granularity. (In practice, there are limitations built into the CDA model that make the expression of certain clinical concepts challenging). This model provides significant power, but also presents challenges. The first is that RIM modeling expertise is required in order to express any particular piece of clinical information. It isn't obvious how to represent things like allergies or surgery or blood pressure "out of the box". Templates are required to support interoperability. The second is that common clinical concepts can be (and frequently are) modeled differently in different circumstances. With FHIR, all clinical (and non-clinical) content in a message is handled by referencing existing resource definitions. These resources make it clear how to represent common structures like allergies and blood pressure "out of the box" and ensure that there's only one way for core content to be represented. It does however create the limitation that an appropriate resource must have been defined in order to share content. In the early stages of FHIR development, it may be necessary to make use of the Basic resource if an appropriate standard resource has not yet been defined.

Templates and Profiles: As discussed above, CDA relies on the presence of templates in order to understand the meaning of instances. While the meaning can theoretically be determined by looking at RIM attributes and codes, the reality is that this is often not safe or sufficient. As such, pretty much every CDA instance includes templateId attribute values scattered throughout the instance to define meaning. With FHIR, meaning is defined by the resource. Profiles can be used to define extensions, but they never refine the meaning of core elements. While the profiles used in constructing a particular instance can be declared within the instance via tags. However, such declaration is not required.

Mark-up language: CDA defines its own XML syntax for narrative content, loosely based on HTML. FHIR makes use of a constrained set of XHTML which is somewhat more expressive than the CDA markup. Conversions from FHIR to CDA will need to take these constraints into account (or alternatively provide a fully rendered version of the document).

7.15.3.2 CDA Interoperability Considerations

CDA is a type of HL7 v3 specification. Therefore, all considerations that apply to v3 messaging also apply to CDA. In addition, the following topics are specific to CDA implementations.

What to map: The right-hand side (clinical content) portion of the CDA model qualifies as an abstract model as discussed above. While the CDA header can reasonably be mapped to the HL7 Composition resource and related resources, mappings between FHIR and CDA should be done at the template level rather than the CDA specification itself.

Human readable granularity: With FHIR, narrative only exists for the resources at the root of each section. With CDA, narrative exists for each section. Usually this means the narrative in CDA and FHIR will correspond. However, in some cases, a section will contain other sub-sections. In CDA, these "container" sections can have narrative. In FHIR, they cannot. Applications will need to have some way of managing this if converting.

Discrete to human-readable linkages: To ensure semantic traceability, both FHIR and CDA allow establishing linkages between text in the narrative and specific discrete elements in the encoded part of a document. If converting between FHIR and CDA, these linkages need to be converted as well. However, this is complicated by the fact that the granularity at which linkages can occur is different between the two specifications. In CDA, linkages can only occur at the level of a section or one of a couple of the entry types. With FHIR, linkages can occur at any level at all, including individual data type components or even portions of extensions. Converting from CDA to FHIR will be straight-forward, however there will be information loss when converting the other way.

7.15.4 Clinical Document Architecture (CDA) on FHIR

URL: https://hl7.org/fhir/STU3/cda-intro.html

A CDA document on FHIR is a defined and complete information object that can include text, images, sounds, and other multimedia content. A number of design principles follow from consideration of the above goals: These principles and requirements have led to the current approach, where the material to be rendered is placed into the Section.content. [EDITORS: The current design doesn't make it clear where to consistently find narrative.]

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.15.4 Clinical Document Architecture (CDA) on FHIR

Structured Documents Work Group | Maturity Level: N/A | Ballot Status: Informative

7.15.4.1 What is CDA on FHIR?

CDA on FHIR specifies how to implement CDA R2 with the FHIR Composition resource. The original HL7 Clinical Document Architecture (CDA) defined the structure and semantics of "clinical documents" for the purpose of exchange. A clinical document is a documentation of clinical observations and services, with the following characteristics:

* Persistence - A clinical document continues to exist in an unaltered state, for a time period defined by local and regulatory requirements (NOTE: There is a distinct scope of persistence for a clinical document, independent of the persistence of any XML-encoded CDA document instance).

* Stewardship - A clinical document is maintained by an organization entrusted with its care.

* Potential for authentication - A clinical document is an assemblage of information that is intended to be legally authenticated.

* Context - A clinical document establishes the context for its contents.

* Wholeness - Authentication of a clinical document applies to the whole and does not apply to portions of the document without the full context of the document.

* Human readability - A clinical document is human readable.

7.15.4.2 Scope of the CDA on FHIR

The scope of CDA on FHIR is the standardization of clinical documents for exchange. The data format of clinical documents outside of the exchange context (e.g. the data format used to store clinical documents) is not addressed in this specification. CDA on FHIR does not specify the creation or management of documents, only their exchange markup. While it may be possible to directly use the CDA Schema in a document authoring environment, such use is not the primary purpose of the CDA specification. Document management is critically interdependent with the CDA specifications, but the specification of document management messages is outside the scope of the CDA.

7.15.4.3 Goals and Design Principles

The goals of the CDA on FHIR are:

* Give priority to delivery of patient care.

* Allow cost effective implementation across as wide a spectrum of systems as possible.

* Support exchange of human-readable documents between users, including those with different levels of technical sophistication.

* Promote longevity of all information encoded according to this architecture.

* Enable a wide range of post-exchange processing applications.

* Be compatible with a wide range of document creation applications.

* Promote exchange that is independent of the underlying transfer or storage mechanism.

* Prepare the design reasonably quickly.

* Enable policy-makers to control their own information requirements without extension to this specification.

* This architecture must be compatible with XML and JSON.

* This architecture must be compatible with representations of clinical information arising from other HL7 committees.

* Technical barriers to use of the architecture should be minimized.

* The architecture specifies the representation of instances required for exchange.

* The architecture should impose minimal constraints or requirements on document structure and content required for exchange.

* The architecture must be scalable to accommodate fine-grained markup such as highly structured text and coded data.

* Document specifications based on this architecture should accommodate such constraints and requirements as supplied by appropriate professional, commercial, and regulatory agencies.

* Document specifications for document creation and processing, if intended for exchange, should map to this exchange architecture.

* CDA documents must be human readable using widely-available and commonly-deployed XML-aware browsers and print drivers and a generic CDA style sheet written in a standard style sheet language.

* Use open standards.

7.15.4 General CDA on FHIR Concepts

7.15.4.1 Major Components of a CDA on FHIR Document

This section serves as a high-level introduction to the major components of a CDA document, all of which are described again and in greater detail later on. The intent here is to familiarize the reader with the high-level concepts to facilitate an understanding of the sections that follow. [EDITORS: in CDA r2 there is a bunch of detail about how CDA is wrapped - and an example. Consider whether the discussion is relevant here: "A CDA document is wrapped by the <ClinicalDocument> element, and contains a header..."]

7.15.4.2 Human Readability and Rendering CDA Documents

The CDA requirement for human readability guarantees that a receiver of a CDA document can algorithmically display the clinical content of the note on a standard Web browser.

* There must be a deterministic way for a recipient of an arbitrary CDA document to render the attested content.

* Human readability shall not require a sender to transmit a special style sheet along with a CDA document. It must be possible to render all CDA documents with a single style sheet and general-market display tools.

* Human readability applies to the authenticated content. There may be additional information conveyed in the document that is there primarily for machine processing that is not authenticated and need not be rendered.

* When structured content is derived from narrative, there must be a mechanism to describe the process (e.g. by author, by human coder, by natural language processing algorithm, by specific software) by which machine-processable portions were derived from a block of narrative.

* When narrative is derived from structured content, there must be a mechanism to identify the process by which narrative was generated from structured data.

7.15.5 Other HL7 Standards

URL: https://hl7.org/fhir/STU3/comparison-other.html

This page is part of the FHIR Specification (v3.0.2: STU 3). The current version which supercedes this version is 4.0.1. For a full list of available versions, see the Directory of published versions

7.15.5 Other HL7 Standards

FHIR Infrastructure Work Group | Maturity Level: N/A | Ballot Status: Informative

HL7 has produced a number of other standards that don't overlap with FHIR as closely as those listed above, primarily because they aren't focused solely on information exchange. However, they deserve a brief mention:

7.15.5.1 EHR Functional Model (EHR-FM)

This specification defines a number of functional behaviors for Electronic Health Record systems. FHIR is a healthcare information exchange standard that can be used to satisfy some of these functional behaviors. Details on how FHIR fits into the EHR-FM can be found here

7.15.5.2 Context Management Specifications (CCOW)

CCOW is a standard for allowing independent systems to synchronize context on a single workstation, providing a seamless interface for the user of that workstation (e.g. ensuring consistent user authentication, display of the same patient, display of the same order, etc.) In theory, FHIR resources could be used as an alternative CCOW implementation technology, however the business case for doing this is not clear. CCOW profiles include HL7 v2 mappings. These mappings can be used to help identify the equivalent FHIR data elements when establishing CCOW linkages in FHIR-based systems.

7.15.5.3 Arden Syntax

Arden Syntax is a language for defining decision support rules. These rules reference data elements that are used as part of the decision making process. However, the specification does not define how these data elements are identified. FHIR element and extension identifiers would provide one mechanism for identifying the relevant data elements.

7.15.5.4 Virtual Medical Record

The Virtual Medical Record is a draft specification under development by HL7 that also serves the decision support space. It defines a logical medical record that decision support rules can be constructed against. At present, this model is a custom model created specifically for VMR. However, the Decision Support work group is evaluating the possibility of using FHIR as a structure for future versions of the specification.